任务五技术报告

1.技术任务详情

1.1技术要求：

1.2提交要求：

2.常用激光SLAM算法的建图测试

2.1gmapping：

2.1.1建图操作视频：

2.1.2建图结果：

2.2hector：

2.2.1建图操作视频：

2.2.2建图结果：

2.3karto：

2.3.1建图操作视频：

2.3.2建图结果：

2.4cartographer:

2.5建图效果的对比总结：

2.5.1初始状态不同：

2.5.2几种算法的建图效果及相应理解：

2.5.2.1gmapping：

2.5.2.2hector：

2.5.2.3karto：

3.cartographer建图环境搭建

3.1技术实现：

4.cartographer建图测试

4.1技术实现：

4.1.1用官网上的地图包：

4.1.2用比赛资料中的地图包：

4.2实现效果展示：

4.2.1用官网上的地图包：

哈工大威海创新平台 - 718—刘渝婉(22971967)

4.2.2用比赛资料中的地图包：
5.对算法原理的理解
5.1总体部分：
5.1.1cartographer：
5.1.1.1Local SLAM部分（Frontend）：
5.1.1.2Global SLAM部分（Backend）：
5.1.2cartographer_ros：
5.2回环检测部分理解：
6.拓展应用——在turtlebot3上使用cartographer仿真建图
6.1技术实现：
6.1.1使用gazebo建立一个仿真环境：
6.1.2下载turtlebot3的仿真包，修改turtlebot3_cartographer.launch
6.1.3启动gazebo：
6.1.4给仿真环境加房子：
6.1.5启动键盘控制小车：
6.1.6运行cartographer算法：
6.1.7保存地图：
6.1.8导航功能的实现：
6.2仿真建图效果演示：
6.3用建好的图导航效果演示：
7.对cartographer源码的优化
7.0优化整体思路：
7.1调参优化：
7.1.1sampling_ratio：
7.1.2min_score：
7.1.3num_subdivisions_per_laser_scan：
7.1.4num_accumulated_range_data:
7.1.5missing_data_ray_length：
7.1.6voxel_filter_size：
7.1.7resolution：
7.1.8TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight：
7.1.9TRAJECTORY_BUILDER_2D.ceres_scan_matcher.rotation_weight：

哈工大威海创新平台 - 718—刘渝婉(22971967)

7.2代码部分优化：
7.2.1分支定界min_score部分：
7.2.1.1思路来源：
占据栅格地图：
分支定界中的min_score：
7.2.1.2实现方法：
主要思路：
解决问题1：
解决问题2：
7.2.1.3修改步骤：
第一步
第二步
第三步
第四步
7.2.1.4效果
7.2.1.5反思不足：
7.2.1.6代码压缩文件：
7.2.2分支界定部分的贪心算法优化
7.2.2.1思路来源：
7.2.2.2实现方法：
7.2.2.3修改步骤：
7.2.2.4代码压缩文件：
7.2.3遇到对称环境处理的改进：
7.2.3.1思路来源：
7.2.3.2实现方法：
7.2.3.3修改步骤：
第一步
第二步
第三步
7.2.3.4效果：
7.2.3.5反思不足：
7.2.3.6代码压缩文件：

哈工大威海创新平台 - 718—刘渝婉(22971967)

7.2.4分支定界源码筛枝阈值设定优化：
7.2.4.1思路来源：
7.2.4.2实现方法：
7.2.4.3修改步骤：
7.2.4.4效果：
7.2.4.5代码压缩文件：
7.2.5已排除的错误的优化思路：
建立一个存储每个节点的pose_estimate的数组避免重复计算pose_estimate：
7.2.5.1初步想法：
7.2.5.2错误原因：
8.视频及优化代码压缩包网盘链接
8.1视频压缩包：
8.2代码压缩包：

1.技术任务详情

1.1技术要求：

- 查找建图相关资料，了解常用的激光SLAM算法；
- 安装Linux系统，建议Ubuntu18.04；
- 安装ROS环境并学习其基本操作；
- 完成谷歌Cartographer算法建图环境搭建；
- 使用Cartagrapher算法进行建图测试；
- 学习算法原理，对算法进行优化及测试。

1.2提交要求：

- 根据资料包提供的数据集完成建图测试，录制视频；
- 撰写技术实现和改进说明。

2.常用激光SLAM算法的建图测试

说明：这里测试的激光slam算法均是在turtlebot3机器人的基础上进行的仿真建图，详尽技术实现步骤见“六、拓展应用——在turtlebot3上使用cartographer仿真建图”

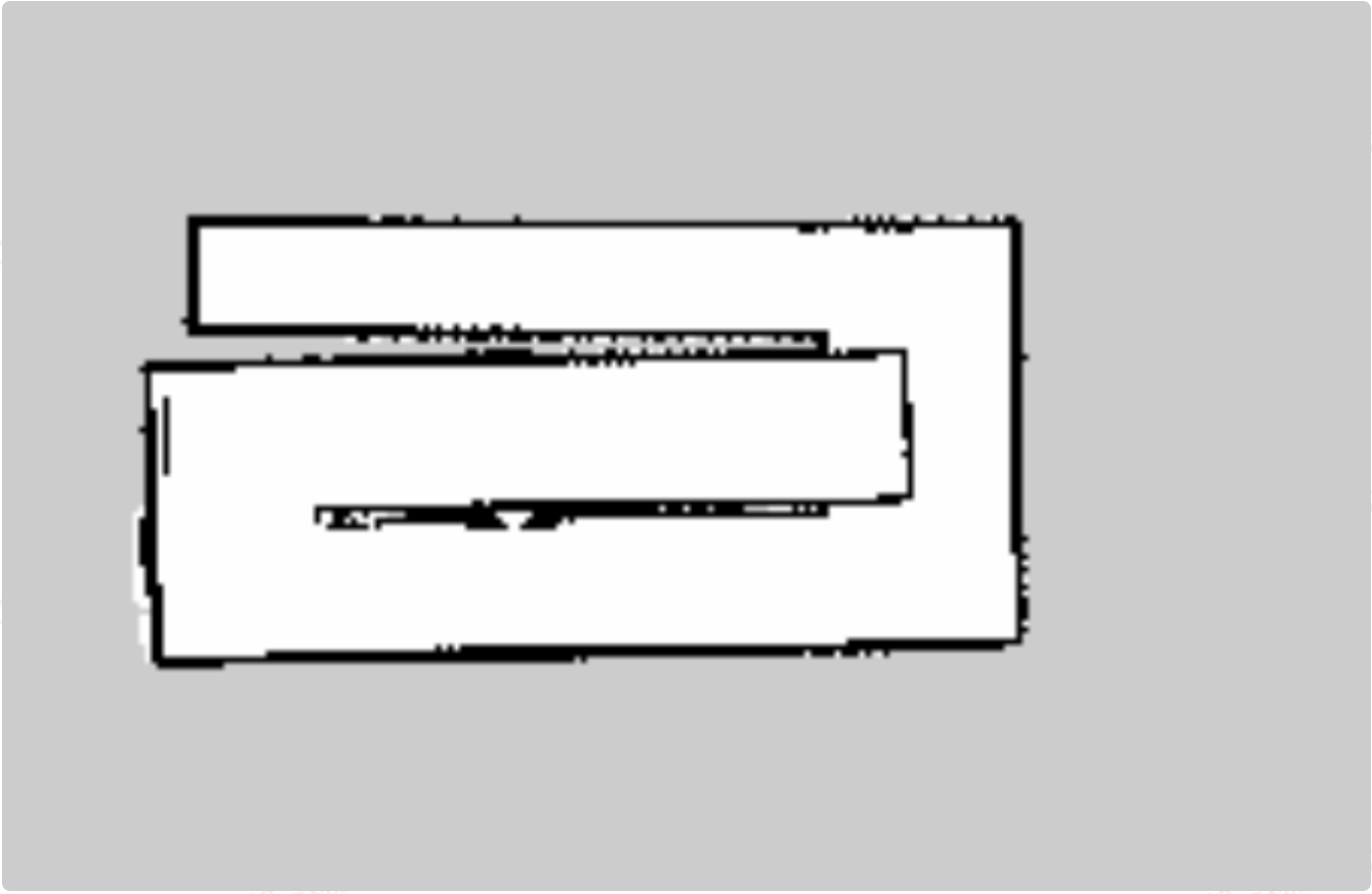
运行不同的slam算法只需在已经搭建了不同算法的建图环境的基础上，将“roslaunch turtlebot3_slam turtlebot3_slam.launch slam_methods:=cartographer”命令中的参数slam_methods设置为不同slam算法的名字即可。

2.1gmapping：

2.1.1建图操作视频：

见视频1

2.1.2建图结果：

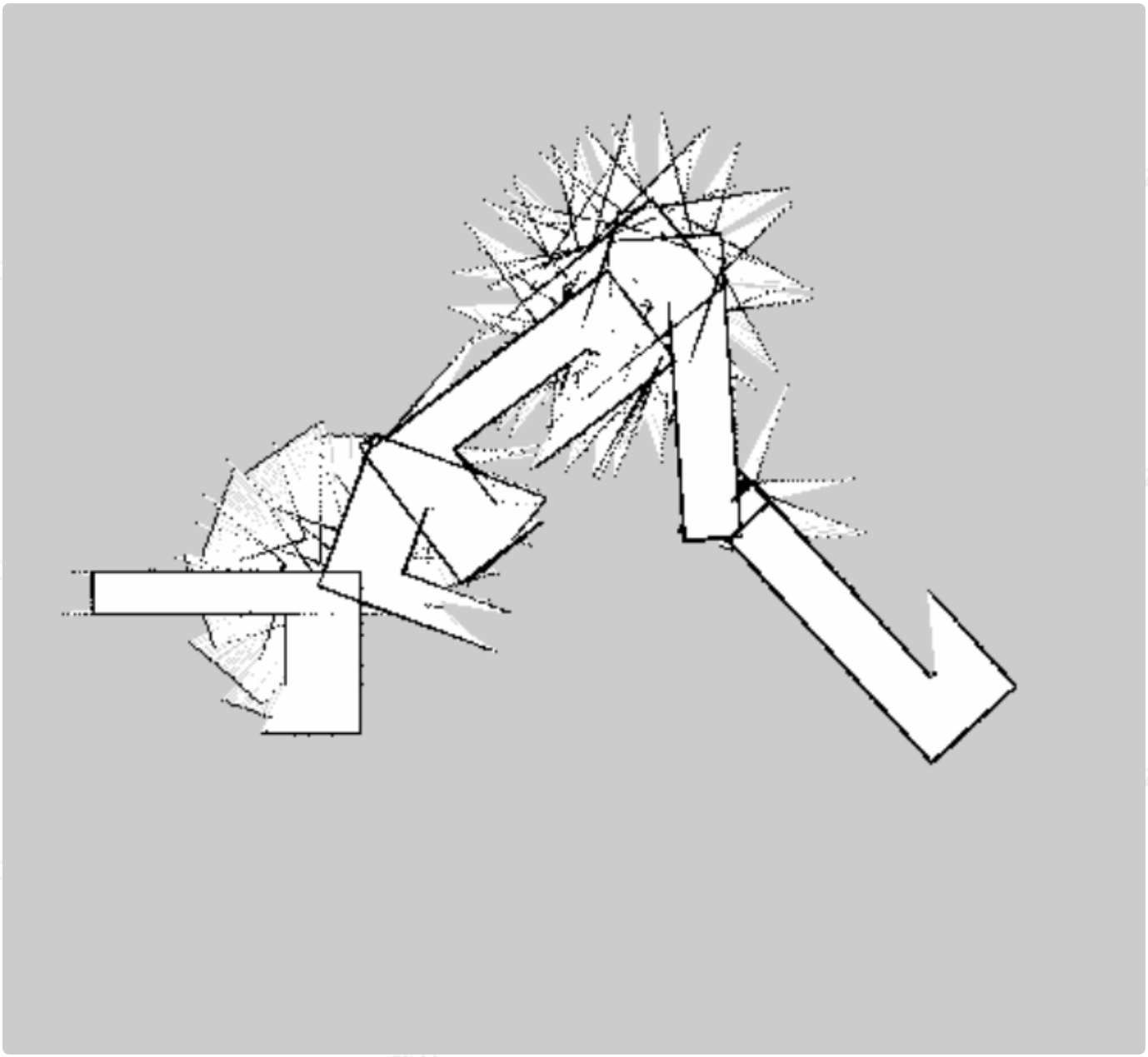


2.2hector：

2.2.1建图操作视频：

见视频2

2.2.2建图结果：



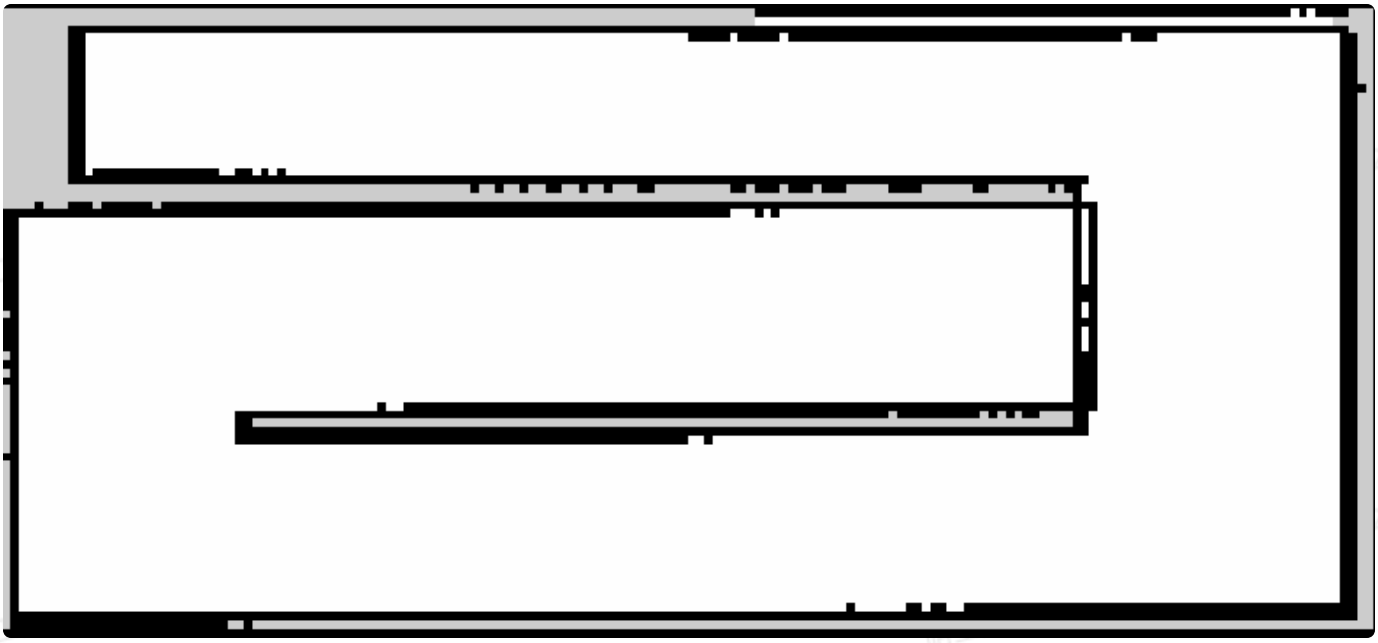
2.3karto：

2.3.1建图操作视频：

见视频3

2.3.2建图结果：

哈工大威海创新平台 - 718—刘渝婉(22971967)



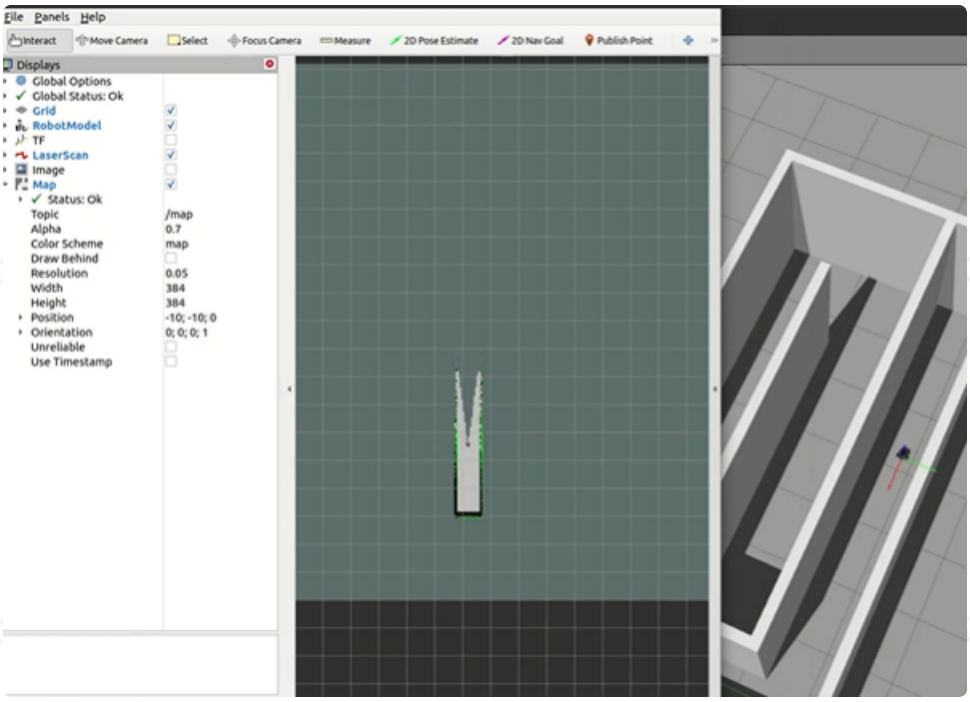
2.4cartographer:

见“六、拓展应用——在turtlebot3上使用cartographer仿真建图”

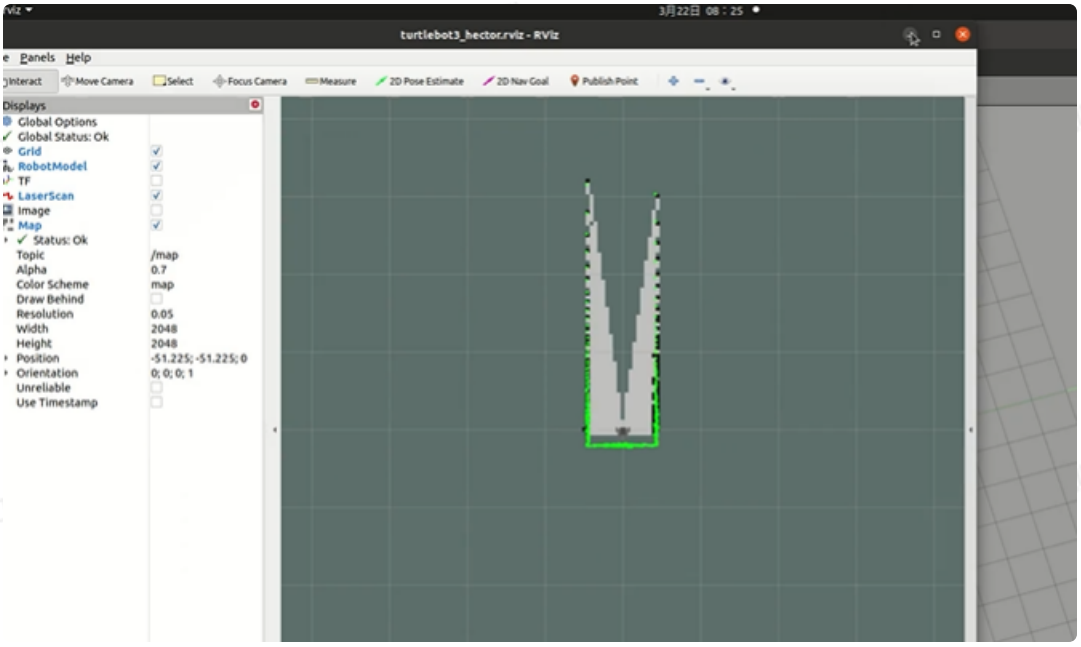
2.5建图效果的对比总结：

2.5.1初始状态不同：

gmapping：

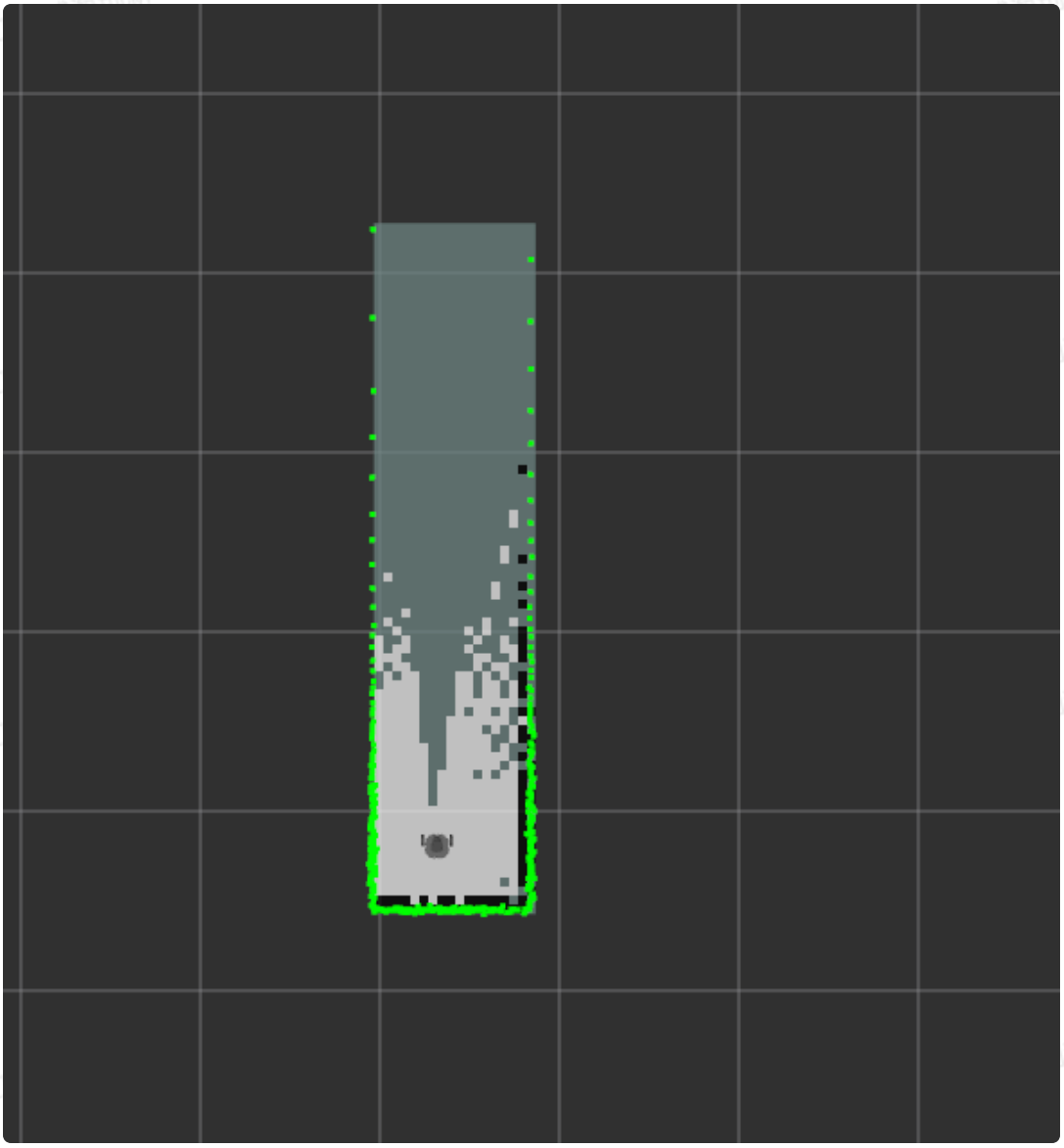


hector：



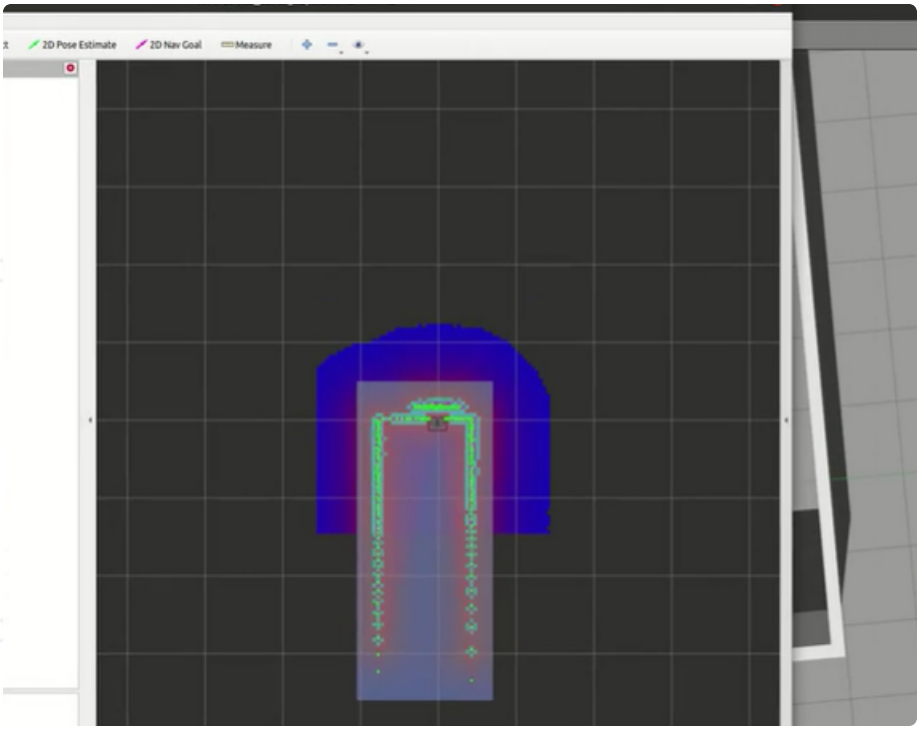
karto:

哈工大威海创新平台 - 718—刘渝婉(22971967)



cartographer：

哈工大威海创新平台 - 718—刘渝婉(22971967)



2.5.2几种算法的建图效果及相应理解：

2.5.2.1gmapping：

gmapping是基于粒子滤波的，它没有回环检测，比较依赖里程计信息。它的里程计模型在传播时，将所有粒子同等对待，优的粒子在传播时可能变成差的粒子，粒子退化问题严重。

它在较小的环境中能实现较好的建图效果，但是难以应对大场景，大场景建图比较消耗资源，在长回廊的环境中容易把图建歪。如上面用gmapping建的图，从上到下的第一条长廊与第二条长廊形成了一个小角度。

2.5.2.2hector：

hector无需使用里程计，完全基于scan-matching，使用迭代优化的方法来求匹配最佳位置，采用多分辨率的地图匹配。

要求雷达的测量精度较高、角度范围大，robot的速度要控制在较低的情况下才会有比较理想的建图效果，噪声多、边角特征点少的场景就很容易失败。上面用hector建的图，在拐弯的情况下建的图发生了偏离，个人感觉hector建图的时候在直线长廊建图时效果不错，但是在机器人旋转的时候图就会发生漂移，感觉该算法可能不太适合拐弯多的环境。

下面是让机器人旋转的时候的建图效果，地图漂移效果很明显：

见视频4

2.5.2.3karto：

karto是基于图优化的方法，采用的SPA方法与扫描匹配和闭环检测相关，它的扫描匹配效果更好，在大环境下制图优势更大。
如上面用karto建的图，地图线条清晰，未发生明显的漂移和错位，效果比较好。

3.cartographer建图环境搭建

3.1技术实现：

我用的Linux系统为Ubuntu20.04，ROS版本是noetic。搭建建图环境命令如下：

C++ 复制代码

```
sudo apt-get update
sudo apt-get install -y python3-wstool python3-rosdep ninja-build stow
mkdir catkin_ws
cd catkin_ws
wstool init src
wstool merge -t src https://raw.githubusercontent.com/cartographer-project/cartographer_ros/master/cartographer_ros.rosinstall
wstool update -t src
sudo rosdep init//如果安装ros的时候进行过了就不用再进行，不然会报错
rosdep update
rosdep install --from-paths src --ignore-src --rosdistro=${ROS_DISTRO} -y
src/cartographer/scripts/install_abseil.sh
catkin_make_isolated --install --use-ninja

```

4.cartographer建图测试

4.1技术实现：

4.1.1用官网上的地图包：

如果是用官网上的地图包，前两步为从网上下载地图包

```C++
wget -P ~/Downloads https://storage.googleapis.com/cartographer-public-data/bags/backpack_2d/b2-2016-04-05-14-44-52.bag
wget -P ~/Downloads https://storage.googleapis.com/cartographer-public-data/bags/backpack_2d/b2-2016-04-27-12-31-41.bag
//生成地图
roslaunch cartographer_ros offline_backpack_2d.launch bag_filenames:=${HOME}/Downloads/b2-2016-04-05-14-44-52.bag
```

4.1.2用比赛资料中的地图包：

在把比赛资料里的地图包放到Downloads文件夹后，运行如下命令生成地图：

```C++
roslaunch cartographer_ros offline_backpack_2d.launch bag_filenames:=${HOME}/Downloads/cartographer_paper_deutsches_museum.bag
```

4.2实现效果展示：

官网上的包速度比较快，比赛给的包速度比较慢

4.2.1用官网上的地图包：

见视频5

4.2.2用比赛资料中的地图包：

（建图用了50分钟，所以对中间的建图部分进行了倍速处理，视频中间有几次闪烁的黑屏，是因为建图时间较长电脑休眠熄屏）

见视频6

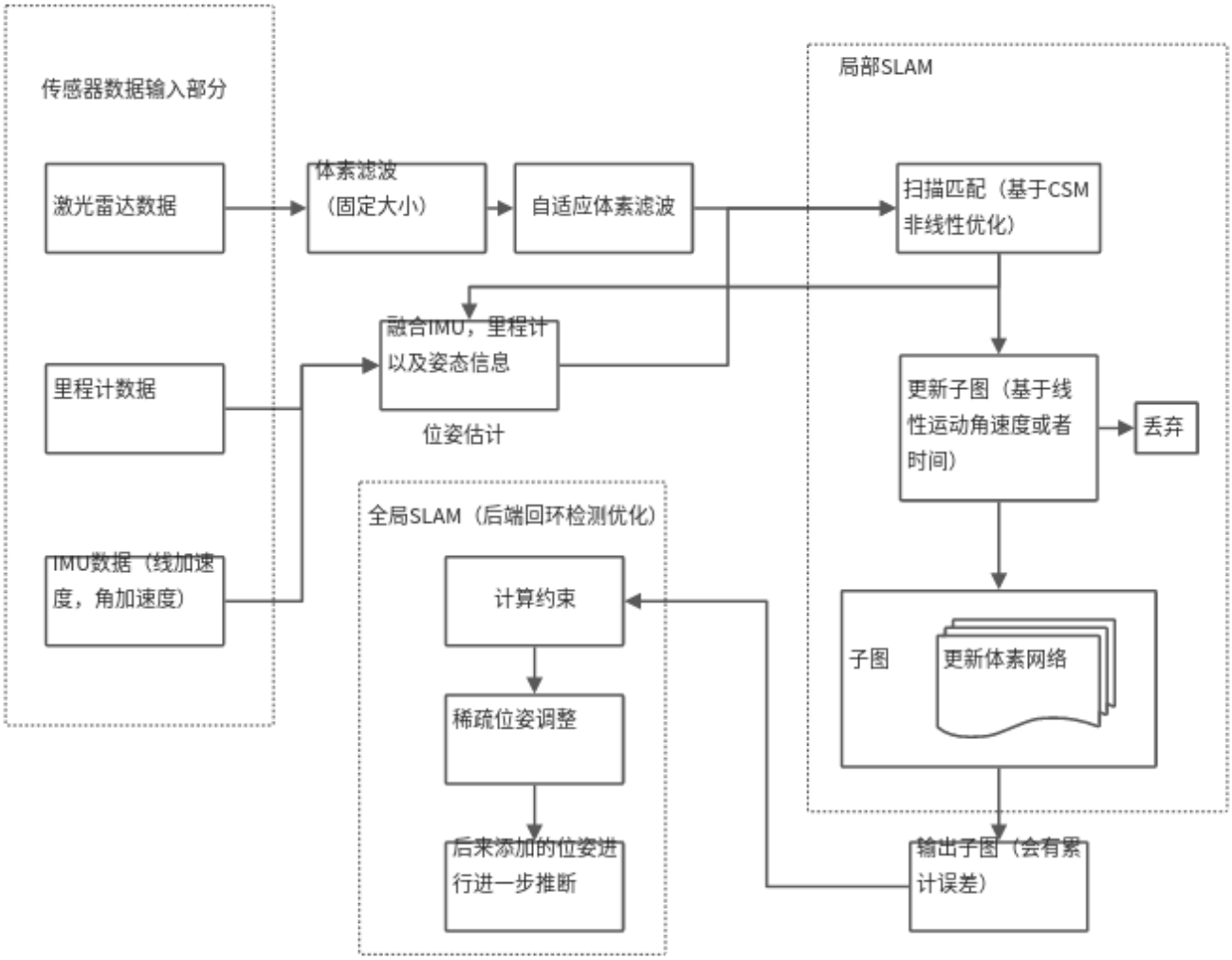
5.对算法原理的理解

说明：主要学习了算法中的回环检测部分，所以对回环检测部分搭配自己的理解进行了一定的梳理

5.1总体部分：

cartographer的代码主要包括两个部分：cartographer和cartographer_ros。

（ceres-solver是一个非线性优化的库，当我们解决优化问题的时候可以调用这个库来求解）



5.1.1cartographer：

主要负责处理来自Laser、IMU、Odemetry等传感器的数据并基于这些数据进行地图的构建。

5.1.1.1Local SLAM部分（Frontend）：

建立、维护Submaps,但问题是该部分建图误差会随着时间累积

5.1.1.2Global SLAM部分（Backend）：

主要进行Loop Closure（闭环检测）（采用Branch-and-Bound Approach （BBA）的方法来解决，将不同的子图以最匹配的位姿tie在一起），如果是3D情况下，该部分还负责根据IMU数据找出重力的方向。

5.1.2cartographer_ros：

基于ros的通信机制获取传感器的数据并将他们转换成cartographer中定义的格式传递给cartographer处理，与此同时也将cartographer的处理发布用于显示或保存（相当于是对cartographer进行了一层ROS接口的封装）

5.2回环检测部分理解：

回环检测主要应该发生在PoseGraph2D::ComputeConstraintsForNode中，其中调用了PoseGraph2D::ComputeConstraint，在这个函数中进行了**遍历当前已完成的所有子图和当前获得的激光帧（节点）分支定界找到的位置（那个pose_estimate）建立约束**

C++ 复制代码

```cpp
for (const auto& submap_id : finished_submap_ids)
  {
    ComputeConstraint(node_id, submap_id);
  }
```

以及遍历过去的所有节点，和新建立的子图计算约束

C++ 复制代码

```cpp
  if (newly_finished_submap)
  {
    const SubmapId newly_finished_submap_id = submap_ids.front();
    for (const auto& node_id_data : optimization_problem_->node_data())
    {
      const NodeId& node_id = node_id_data.id;
      if (newly_finished_submap_node_ids.count(node_id) == 0)
      {
        ComputeConstraint(node_id, newly_finished_submap_id);
      }
    }
  }
```

上面调用的PoseGraph2D::ComputeConstraint应该是个优化中总的计算约束的函数（是类PoseGraph2D的成员函数，与下面的另一个ComputeConstraint不同属一个类）

C++ 复制代码

```cpp
void PoseGraph2D::ComputeConstraint(const NodeId& node_id,
                                    const SubmapId& submap_id) {

  CHECK(submap_data_.at(submap_id).state == SubmapState::kFinished);

  const common::Time node_time = GetLatestNodeTime(node_id, submap_id);
  const common::Time last_connection_time =
      trajectory_connectivity_state_.LastConnectionTime(
          node_id.trajectory_id, submap_id.trajectory_id);
    //此种情况下进行局部窗口搜索
  if (node_id.trajectory_id == submap_id.trajectory_id ||
      node_time <
          last_connection_time +
              common::FromSeconds(
                  options_.global_constraint_search_after_n_seconds())) {

    const transform::Rigid2d initial_relative_pose =
        optimization_problem_->submap_data()
            .at(submap_id)
            .global_pose.inverse() *
        optimization_problem_->node_data().at(node_id).global_pose_2d;
    constraint_builder_.MaybeAddConstraint(
        submap_id, submap_data_.at(submap_id).submap.get(), node_id,
        trajectory_nodes_.at(node_id).constant_data.get(),
        initial_relative_pose);
      //此种情况下进行全局窗口搜索
  } else if (global_localization_samplers_[node_id.trajectory_id]->Pulse()) {
    constraint_builder_.MaybeAddGlobalConstraint(
        submap_id, submap_data_.at(submap_id).submap.get(), node_id,
        trajectory_nodes_.at(node_id).constant_data.get());
  }
}

```

这个函数中根据节点和子图所在轨迹的情况分为了局部窗口搜索和全局窗口搜索。

如果是局部窗口搜索，则调用了MaybeAddConstraint函数：

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```cpp
void ConstraintBuilder2D::MaybeAddConstraint(
    const SubmapId& submap_id, const Submap2D* const submap,
    const NodeId& node_id, const TrajectoryNode::Data* const
constant_data,
    const transform::Rigid2d& initial_relative_pose) {
  if (initial_relative_pose.translation().norm() >
      options_.max_constraint_distance()) {
    return;
  }
  if (!sampler_.Pulse()) return;

  common::MutexLocker locker(&mutex_);
  if (when_done_) {
    LOG(WARNING)
        << "MaybeAddConstraint was called while WhenDone was scheduled.";
  }
  constraints_.emplace_back();
  kQueueLengthMetric->Set(constraints_.size());
  auto* const constraint = &constraints_.back();

  const auto* scan_matcher =
      DispatchScanMatcherConstruction(submap_id, submap->grid());

  auto constraint_task = common::make_unique<common::Task>();
  constraint_task->SetWorkItem([=]() EXCLUDES(mutex_) {
    ComputeConstraint(submap_id, submap, node_id, false, /*
match_full_submap */
                      constant_data, initial_relative_pose,
*scan_matcher,
                      constraint);
  });
  constraint_task->AddDependency(scan_matcher->creation_task_handle);
  auto constraint_task_handle =
      thread_pool_->Schedule(std::move(constraint_task));
  finish_node_task_->AddDependency(constraint_task_handle);
}

```

这个函数第一步首先通过判断某些节点相对于子图的初始相对位姿的范数是否过大（使用一个阈值max_constraint_distance判断）（POSE_GRAPH设置 max_constraint_distance = 15），不计算他们的约束。

函数中计算约束的关键部分应该是ConstraintBuilder2D::ComputeConstraint

（这个是类ConstraintBuilder2D的成员函数，虽然与之前提到的PoseGraph2D::ComputeConstraint函数名相同，但函数内容不同），该函数被调用的时候参数表中的一个参数match_full_submap被设置为了false，表示是在局部窗口搜索。
由于是在局部窗口搜索，所以此函数关键部分用到了：

C++ 复制代码

```
else {
    kConstraintsSearchedMetric->Increment();
    if (submap_scan_matcher.fast_correlative_scan_matcher->Match(
            initial_pose, constant_data->filtered_gravity_aligned_point_cloud,
            options_.min_score(), &score, &pose_estimate)) {
      // We've reported a successful local match.
      CHECK_GT(score, options_.min_score());
      kConstraintsFoundMetric->Increment();
      kConstraintScoresMetric->Observe(score);
    } else {
      return;
    }
  }
```

这里面的if条件调用了扫描匹配的Match，其中：

C++ 复制代码

```


  return MatchWithSearchParameters(search_parameters, initial_pose_estimate,
                                   point_cloud, min_score, score,
                                   pose_estimate);


```

return部分调用了MatchWithSearchParameters，其中：

C++ 复制代码

```cpp

  const Candidate2D best_candidate = BranchAndBound(
      discrete_scans, search_parameters, lowest_resolution_candidates,
      precomputation_grid_stack_->max_depth(), min_score);
if (best_candidate.score > min_score) {
    *score = best_candidate.score;
    *pose_estimate = transform::Rigid2d(
        {initial_pose_estimate.translation().x() + best_candidate.x,
         initial_pose_estimate.translation().y() + best_candidate.y},
        initial_rotation *
Eigen::Rotation2Dd(best_candidate.orientation));
    return true;
  }
  return false;
```

这部分调用了BranchAndBound（分支定界函数）：

（因为优化点涉及到分支定界函数，所以在下述源码中注释了自己的理解）

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```cpp
Candidate2D FastCorrelativeScanMatcher2D::BranchAndBound(
    const std::vector<DiscreteScan2D>& discrete_scans,
    const SearchParameters& search_parameters,
    const std::vector<Candidate2D>& candidates, const int candidate_depth,
    float min_score) const {
    //首先判断解的深度是否为0，如果深度为0，则说明已经搜索到最后一层
  if (candidate_depth == 0) {
    // Return the best candidate.
    //则直接返回当前储存解的数组中的首个元素，由于在进入分支定界前，储存解的数组都会经过打分函数ScoreCandidates的处理，会使得数组中元素根据每个解的得分由大到小排列，因此数组中首个元素即为得分最高的元素
    return *candidates.begin();
  }
 //初始化表示最优解的变量，并将其得分初始化为用于筛枝的阈值min_score
  Candidate2D best_high_resolution_candidate(0, 0, 0, search_parameters);

    best_high_resolution_candidate.score = min_score;
  //进入for循环开始分支定界的核心部分
  for (const Candidate2D& candidate : candidates) {
     //如果父节点的得分就小于筛枝的阈值min_score，则跳出for循环直接舍弃这个父节点及其下属所有子节点
    if (candidate.score <= min_score) {
      break;
    }
     //创建一个用于储存将父节点分支后产生的子节点的数组
    std::vector<Candidate2D> higher_resolution_candidates;
     //设定步长的值
    const int half_width = 1 << (candidate_depth - 1);
      //开始将父节点分支，每个父节点下分为4个子节点
    for (int x_offset : {0, half_width}) {
        //如果分支得到的子节点的位置超出搜索框范围则舍弃
      if (candidate.x_index_offset + x_offset >
          search_parameters.linear_bounds[candidate.scan_index].max_x) {
        break;
      }
      for (int y_offset : {0, half_width}) {
          //如果分支得到的子节点的位置超出搜索框范围则舍弃
        if (candidate.y_index_offset + y_offset >
            search_parameters.linear_bounds[candidate.scan_index].max_y) {
          break;
        }
          //将分支得到的子节点加入之前创建的数组
        higher_resolution_candidates.emplace_back(
```

```
41                 candidate.scan_index, candidate.x_index_offset + x_offset,
42                 candidate.y_index_offset + y_offset, search_parameters);
43           }
44         }
45         //为刚刚得到的子节点数组的所有子节点打分
46         ScoreCandidates(precomputation_grid_stack_->Get(candidate_depth - 1),
47                         discrete_scans, search_parameters,
48                         &higher_resolution_candidates);
49           //不断进行递归，同时根据分数高低不断更新最优解
50         best_high_resolution_candidate = std::max(
51             best_high_resolution_candidate,
52             BranchAndBound(discrete_scans, search_parameters,
53                            higher_resolution_candidates, candidate_depth - 1,
54                            best_high_resolution_candidate.score));
55       }
56         //返回分支定界得到的最优解
57       return best_high_resolution_candidate;
58     }
```

机器人每向前走一定距离得到一帧激光帧，激光帧为点云的形式，这个点云会经过预处理：
将点云按照initial_pose旋转成rotated_point_cloud,初始化search_parameter(也就是搜索窗,将位置和姿态的变化分成多个区间,比如x和y轴的范围是-40~40,角度的变化是5°,然后把他们分别分成30份,则得到了位姿变化的30*30*30种可能性).
将已经旋转的点云rotated_point_cloud按照角度变化的份数旋转成n份(rotated_scans)，再将rotated_scans中的所有点转换成栅格地图中的点（称为discrete_scans），然后构造candidates，将例如30*30*30种可能性对应的位姿在栅格地图中的表示全部记录下来作为解储存到candidates数组中。
新得到的激光帧（节点）需要在搜索框中选个最佳位姿，分支定界中就是将这些candidate，每个candidate先进行筛枝操作，根据设置的阈值min_score初步减掉一些枝，再往下细分子节点，（这些子节点默认是父节点的右、下、右下还有本身），四个子节点先加入higher_resolution_candidates的可变长数组中作为分支定界的下一层，然后对子节点进行打分，得到分值最高的暂时作为best_high_resolution_candidate，继续进行搜索，直至得到规定的搜索框中的最优解（分值最大的）。
再回到MatchWithSearchParameters函数，分支定界完成后会返回一个最优解，赋值给best_candidate，MatchWithSearchParameters中会再次比较一下该最优解的得分是否大于设置的阈值min_score,如果确实大于，则把该解得分赋值给*score，并计算出要用于约束计算的*pose_estimate，最后使整个函数返回值为true，即使得调用它的Match函数返回值也是true。
再回到ConstraintBuilder2D::ComputeConstraint函数中：

C++ 复制代码

```
else {
    kConstraintsSearchedMetric->Increment();
    if (submap_scan_matcher.fast_correlative_scan_matcher->Match(
            initial_pose, constant_data->filtered_gravity_aligned_point_cloud,
            options_.min_score(), &score, &pose_estimate)) {
      // We've reported a successful local match.
      CHECK_GT(score, options_.min_score());
      kConstraintsFoundMetric->Increment();
      kConstraintScoresMetric->Observe(score);
    } else {
      return;
    }
  }
```

由于Match则函数返回值为true，则会进行if中的语句，会再次检查一下得到的最优解的得分是否大于min_score。

ConstraintBuilder2D::ComputeConstraint函数中进行完上面步骤还会：

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```
ceres::Solver::Summary unused_summary;
  ceres_scan_matcher_.Match(pose_estimate.translation(), pose_estimate,
                            constant_data->filtered_gravity_aligned_point_cloud,
                            *submap_scan_matcher.grid, &pose_estimate,
                            &unused_summary);


  const transform::Rigid2d constraint_transform =
      ComputeSubmapPose(*submap).inverse() * pose_estimate;

  //根据计算的位姿,重新设置约束的值.--可以认为增加了新的约束.
  constraint->reset(new Constraint{submap_id,
                                   node_id,
{transform::Embed3D(constraint_transform),
 options_.loop_closure_translation_weight(),
 options_.loop_closure_rotation_weight()},
                                   Constraint::INTER_SUBMAP});
```

和优化相关，其中的constraint_transform应该是算出来的约束。

如果是全局窗口搜索，则调用了MaybeAddGlobalConstraint：

C++ 复制代码

```cpp
void ConstraintBuilder2D::MaybeAddGlobalConstraint(
    const SubmapId& submap_id, const Submap2D* const submap,
    const NodeId& node_id, const TrajectoryNode::Data* const
constant_data) {
  absl::MutexLock locker(&mutex_);
  if (when_done_) {
    LOG(WARNING)
        << "MaybeAddGlobalConstraint was called while WhenDone was
scheduled.";
  }
  constraints_.emplace_back();
  kQueueLengthMetric->Set(constraints_.size());
  auto* const constraint = &constraints_.back();
  const auto* scan_matcher =
      DispatchScanMatcherConstruction(submap_id, submap->grid());
  auto constraint_task = absl::make_unique<common::Task>();
  constraint_task->SetWorkItem([=]() LOCKS_EXCLUDED(mutex_) {
    ComputeConstraint(submap_id, submap, node_id, true, /*
match_full_submap */
                      constant_data, transform::Rigid2d::Identity(),
                      *scan_matcher, constraint);
  });
  constraint_task->AddDependency(scan_matcher->creation_task_handle);
  auto constraint_task_handle =
      thread_pool_->Schedule(std::move(constraint_task));
  finish_node_task_->AddDependency(constraint_task_handle);
}
```

这里面调用ConstraintBuilder2D::ComputeConstraint时把 match_full_submap设置成了true代表进行全局窗口搜索：

哈工大威海创新平台 - 718—刘渝婉(22971967)

```cpp
if (match_full_submap) {
    kGlobalConstraintsSearchedMetric->Increment();
    if (submap_scan_matcher.fast_correlative_scan_matcher->MatchFullSubmap(
            constant_data->filtered_gravity_aligned_point_cloud,
            options_.global_localization_min_score(), &score, &pose_estimate)) {
      CHECK_GT(score, options_.global_localization_min_score());
      CHECK_GE(node_id.trajectory_id, 0);
      CHECK_GE(submap_id.trajectory_id, 0);
      kGlobalConstraintsFoundMetric->Increment();
      kGlobalConstraintScoresMetric->Observe(score);
    } else {
      return;
    }
```

if条件里调用了MatchFullSubmap：

```cpp

  return MatchWithSearchParameters(search_parameters, center, point_cloud,
                                   min_score, score, pose_estimate);

```

return部分也是调用了MatchWithSearchParameters函数，和之前局部窗口搜索的区别应该是搜索框有所不同，并且MatchWithSearchParameters里传入的参数局部窗口搜索时是initial_pose_estimate，全局窗口搜索换成了center。剩下的过程应该和局部窗口搜索差不多。

6.拓展应用——在turtlebot3上使用cartographer仿真建图

6.1技术实现：

6.1.1使用gazebo建立一个仿真环境：

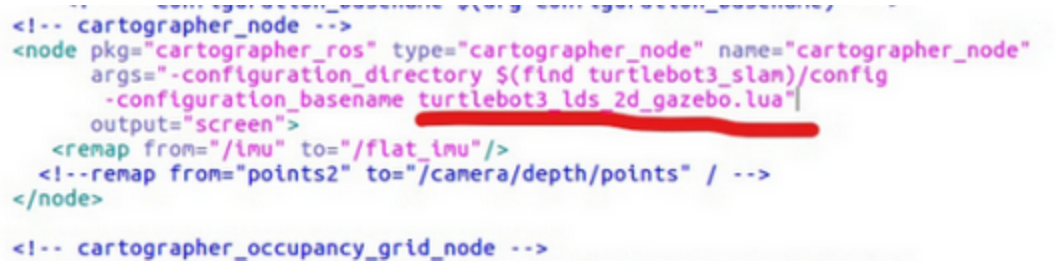
在gazebo环境下，点击上方菜单栏中的“Edit–>Building Editor”，选择“Wall”，进行墙的绘画，完成绘制后快捷键“Ctrl+s”保存绘制的建筑

见视频7

6.1.2下载turtlebot3的仿真包，修改turtlebot3_cartographer.launch

该launch文件在目录

Robot_Turtlebot3/src/turtlebot3/turtlebot3_slam/launch



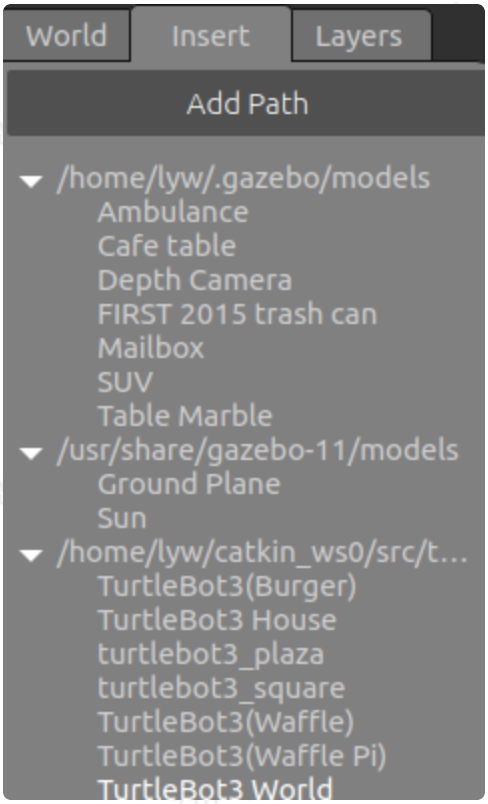
6.1.3启动gazebo：

开启一个终端

Python 复制代码

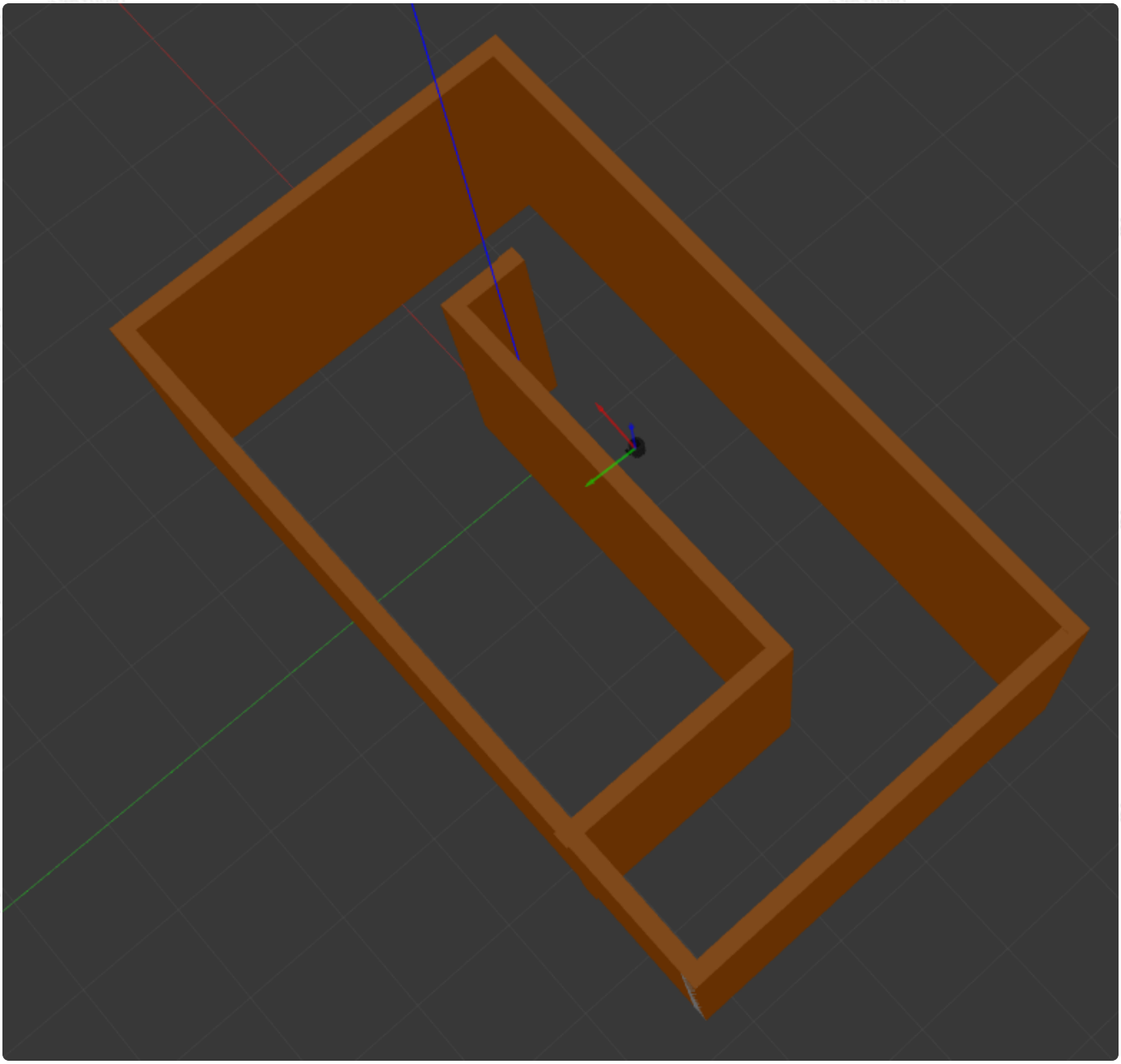
```
echo "export TURTLEBOT3_MODEL=burger" >> ~/.bashrc
//catkin_ws0是turtlebot3所在的workspace
source ~/catkin_ws0/devel/setup.bash

roslaunch turtlebot3_gazebo turtlebot3_empty_world.launch
```

6.1.4给仿真环境加房子：



可用 中的turtlebot3_house,也可用自己已建好的building

哈工大威海创新平台 - 718—刘渝婉(22971967)



6.1.5启动键盘控制小车：

再开一个终端

```python
source ~/catkin_ws0/devel/setup.bash
roslaunch turtlebot3_teleop turtlebot3_teleop_key.launch
```

在英文状态下用wasdx控制：

w:沿z轴正方向前进的线速度

a、d：调整角速度

s：停止——线速度角速度都降为0

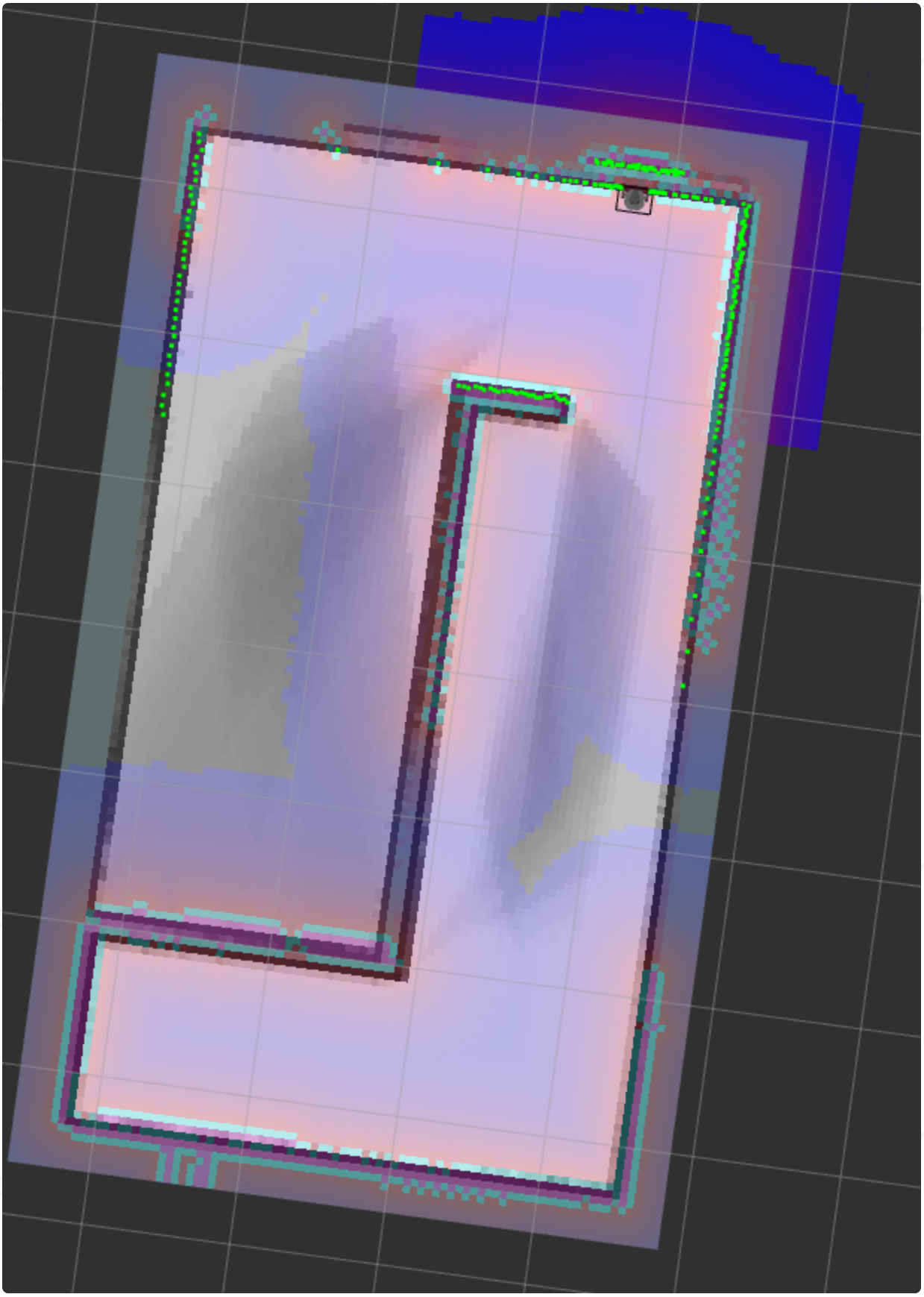
x：沿z轴负方向前进的线速度

6.1.6运行cartographer算法：

再开一个终端

```python
source ~/catkin_ws0/devel/setup.bash
//cartographer_tur_ws是cartograhpher-turtlebot所在的工作空间
source ~/cartographer_tur_ws/devel_isolated/setup.bash
roslaunch turtlebot3_slam turtlebot3_slam.launch
slam_methods:=cartographer
```

哈工大威海创新平台 - 718—刘渝婉(22971967)

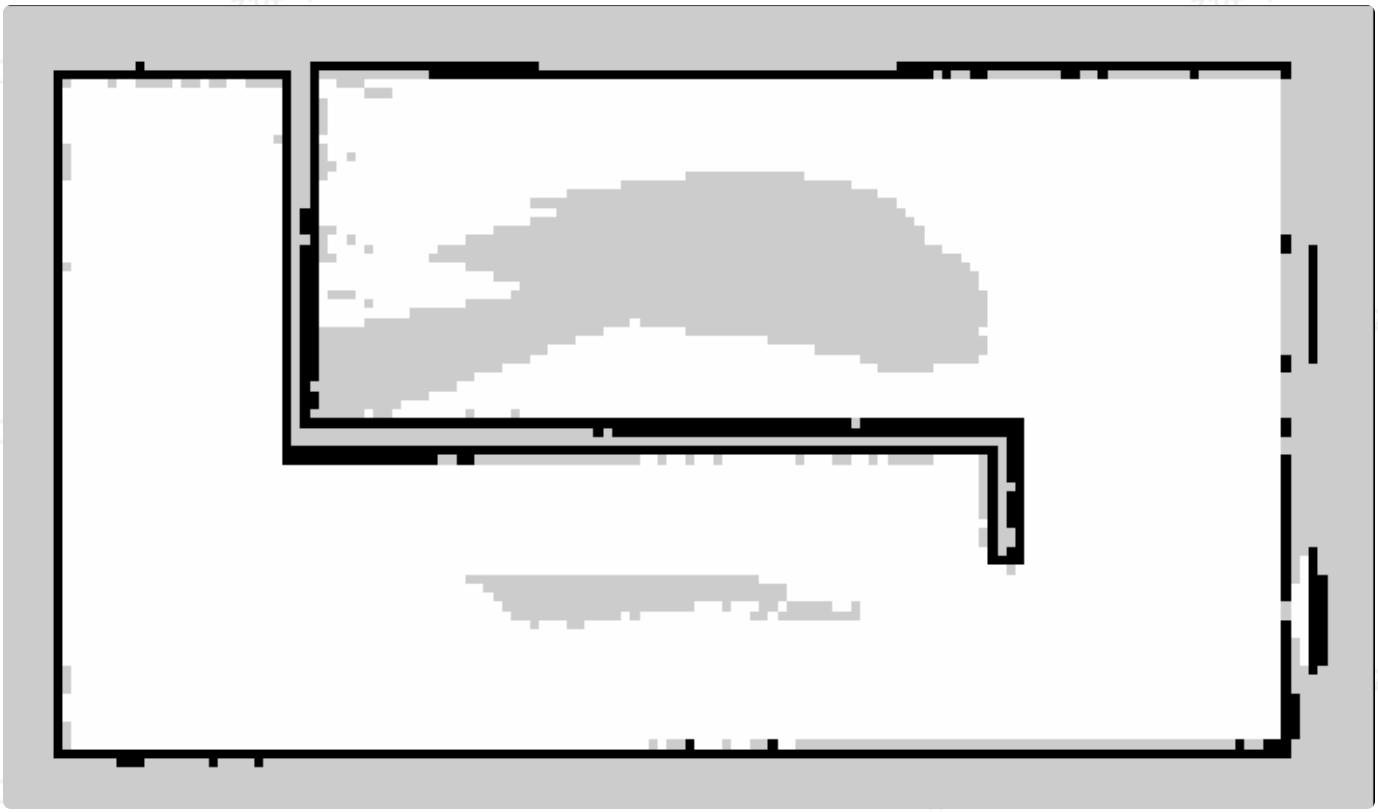


6.1.7保存地图：

718—刘渝婉(22971967)

再开一个终端：

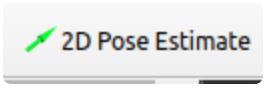
```
rosrun map_server map_saver -f ~/map
```

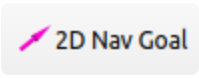


6.1.8导航功能的实现：

先按上述（1）—（4）步骤启动gazebo并加房子

```
source ~/catkin_ws0/devel/setup.bash
roslaunch turtlebot3_navigation turtlebot3_navigation.launch
map_file:=/home/lyw/map3.yaml
//map3是之前建好的地图保存的
```

在打开的rviz中先 

在地图中确定目前gazebo中车的位姿，再用 

确定目标地点，即可自主规划路径到达

6.2仿真建图效果演示：

见视频8

6.3用建好的图导航效果演示：

见视频9

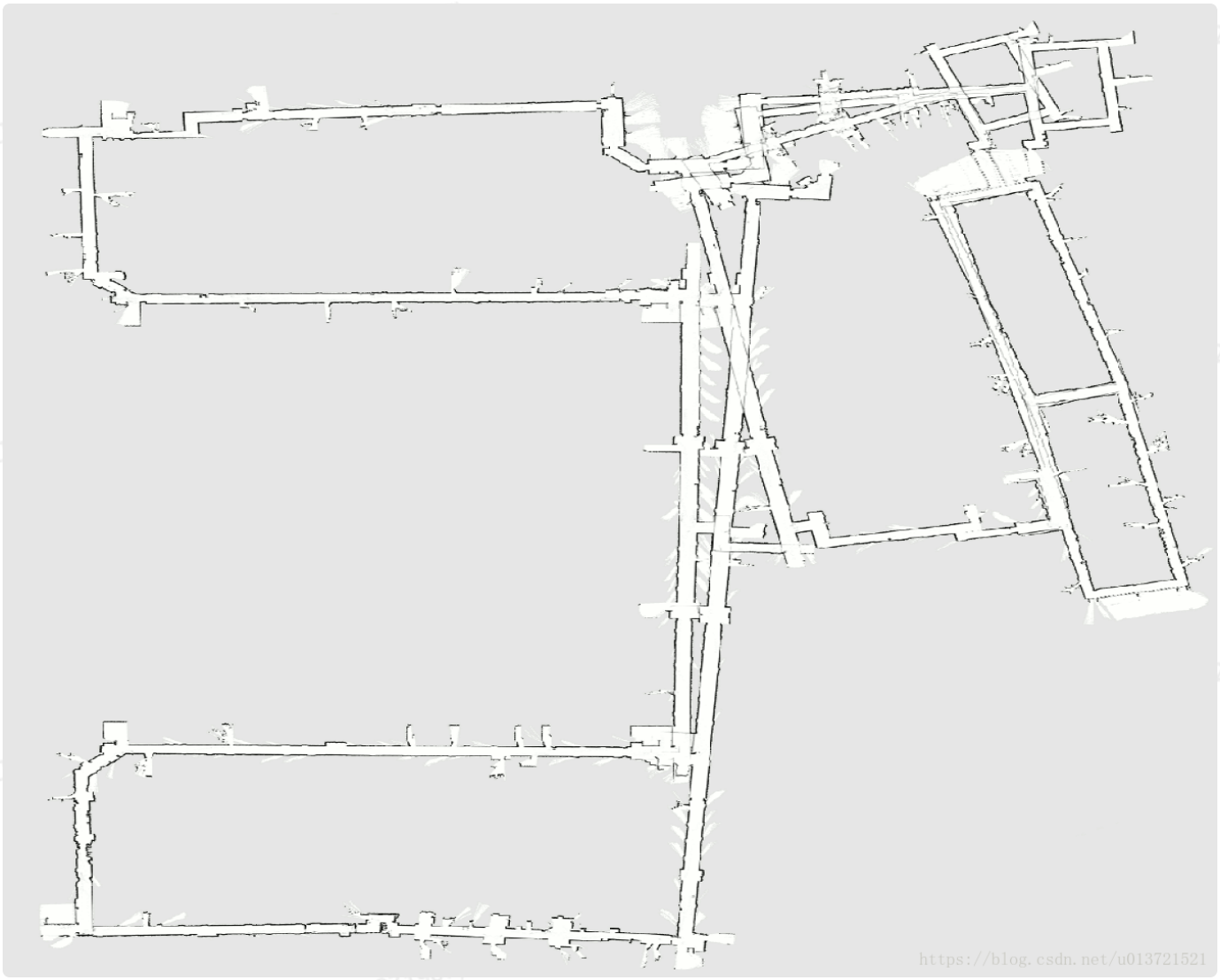
7.对cartographer源码的优化

7.0优化整体思路：

首先，cartographer算法大致可以分为两个比较大的模块，一个是localSLAM部分，也就是submap的建立，这部分也就是常说的Frontend，该部分主要完成建图的从物理到数字的部分工作，但是，submap的建立会伴随着误差的积累。

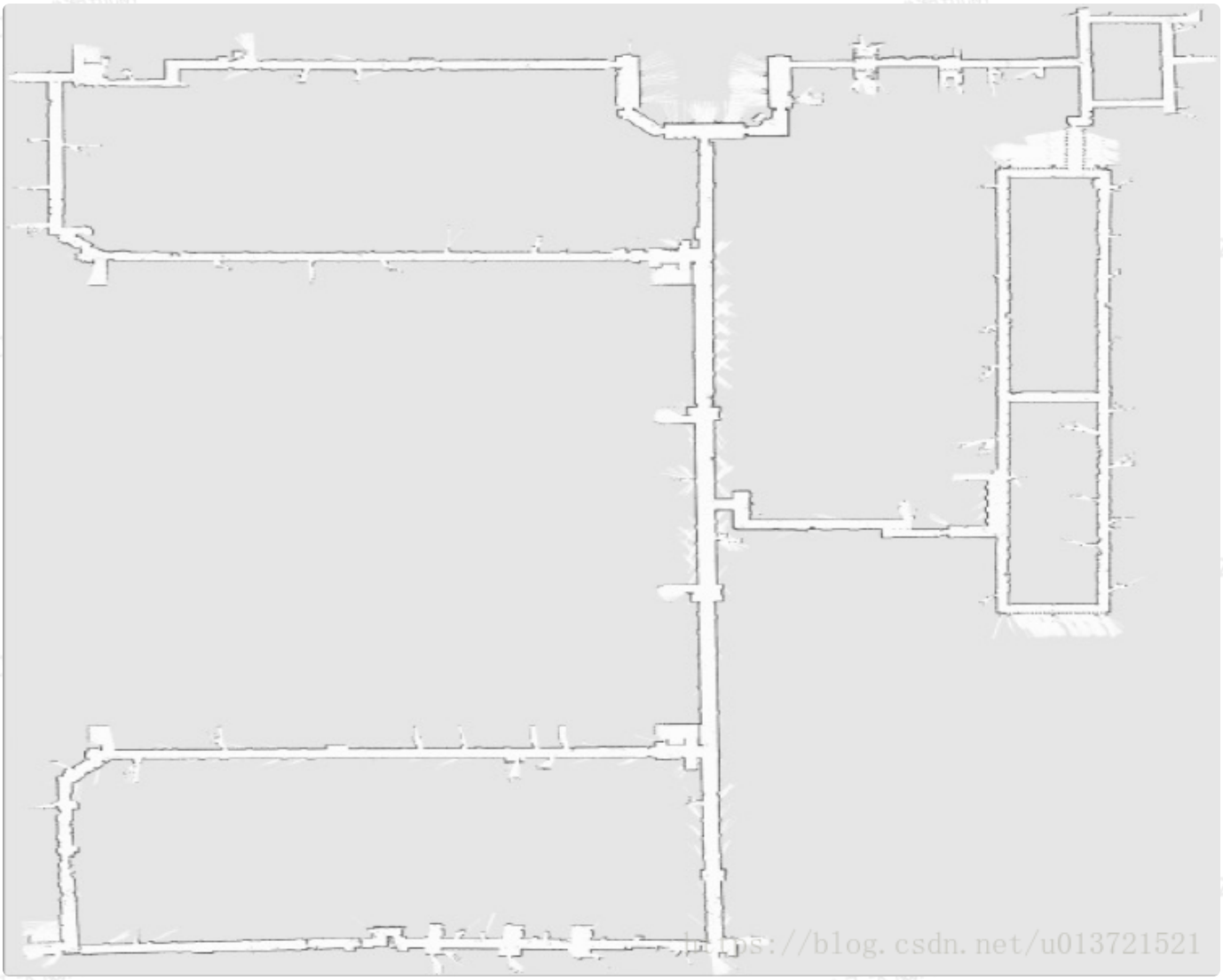
第二部分是GlobalSLAM,也被叫做Backend，既然在前端有了误差的积累，后端的算法就是用来闭环检测并且调整算法误差，尽可能精准的完善建图的过程。

哈工大威海创新平台 - 718—刘渝婉(22971967)



https://blog.csdn.net/u013721521

哈工大威海创新平台 - 718—刘渝婉(22971967)

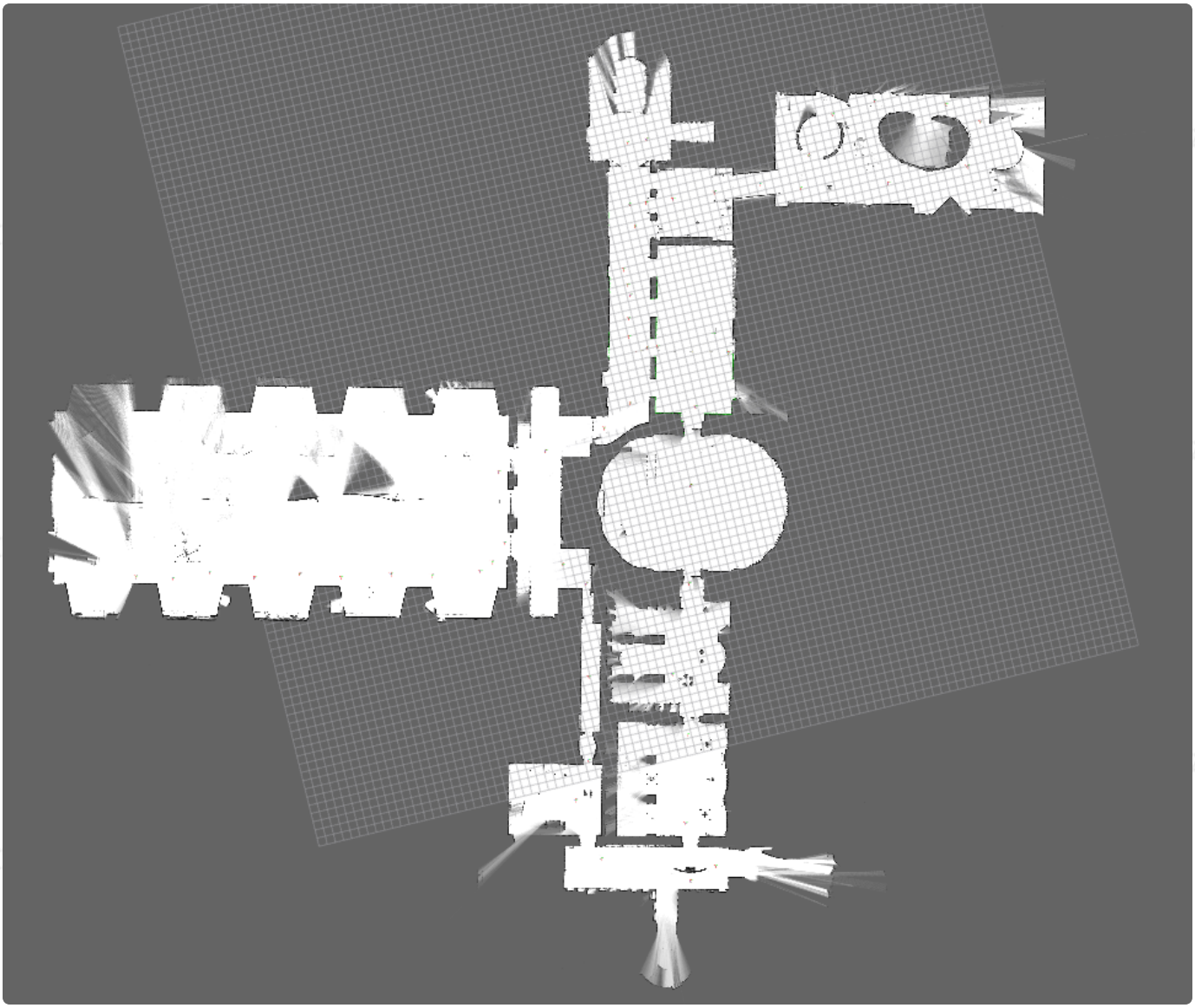


总体而言，前端部分负责建立和生成子图，这部分主要运用的是数学原理，后端部分主要负责维护子图，进行全局的优化，将不同的子图以一个最匹配的位姿结合在一起，这部分主要运用的算法原理。由于对于数学底层原理能够优化的比较少，所以大体重心就放在了后端部分。

7.1调参优化：

调参前：

哈工大威海创新平台 - 718—刘渝婉(22971967)



调参注意事项总结：

参数比较多，要注意调某一个参数的时候其他参数已调为正常值

调某一个参数的时候其他参数要保持数值不变

注意点：

POSE_GRAPH.optimize_every_n_nodes设置成0把后端优化去掉，相当于把前端和后端区分开来，这样如果建图有问题，说明是前端的参数需要调整，先优化前端而如果前端没有问题，再把此数值改回去，调整后端的参数

说明：

以下调参过程中记录的各个参数值的数值间隔不规律，是因为一开始不知道怎样的参数值会使地图发生肉眼可见的变化，所以一开始参数值的数值间隔比较小，发现地图没什么变化后，就增大参数值间隔进行调参；

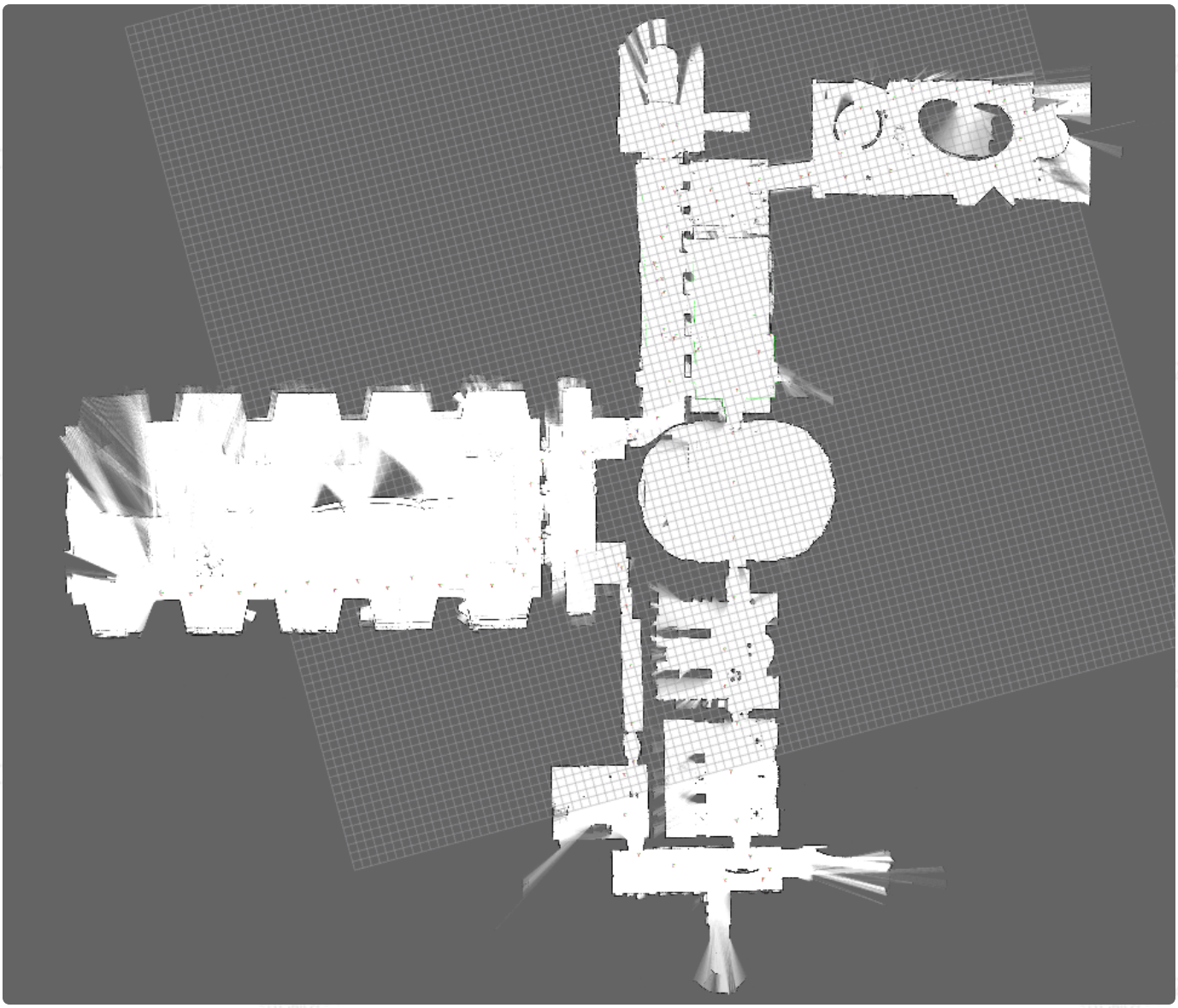
因为比赛给的包建图时间过长，因此使用的是官网上的地图包进行的调参

7.1.1sampling_ratio：

参数介绍：

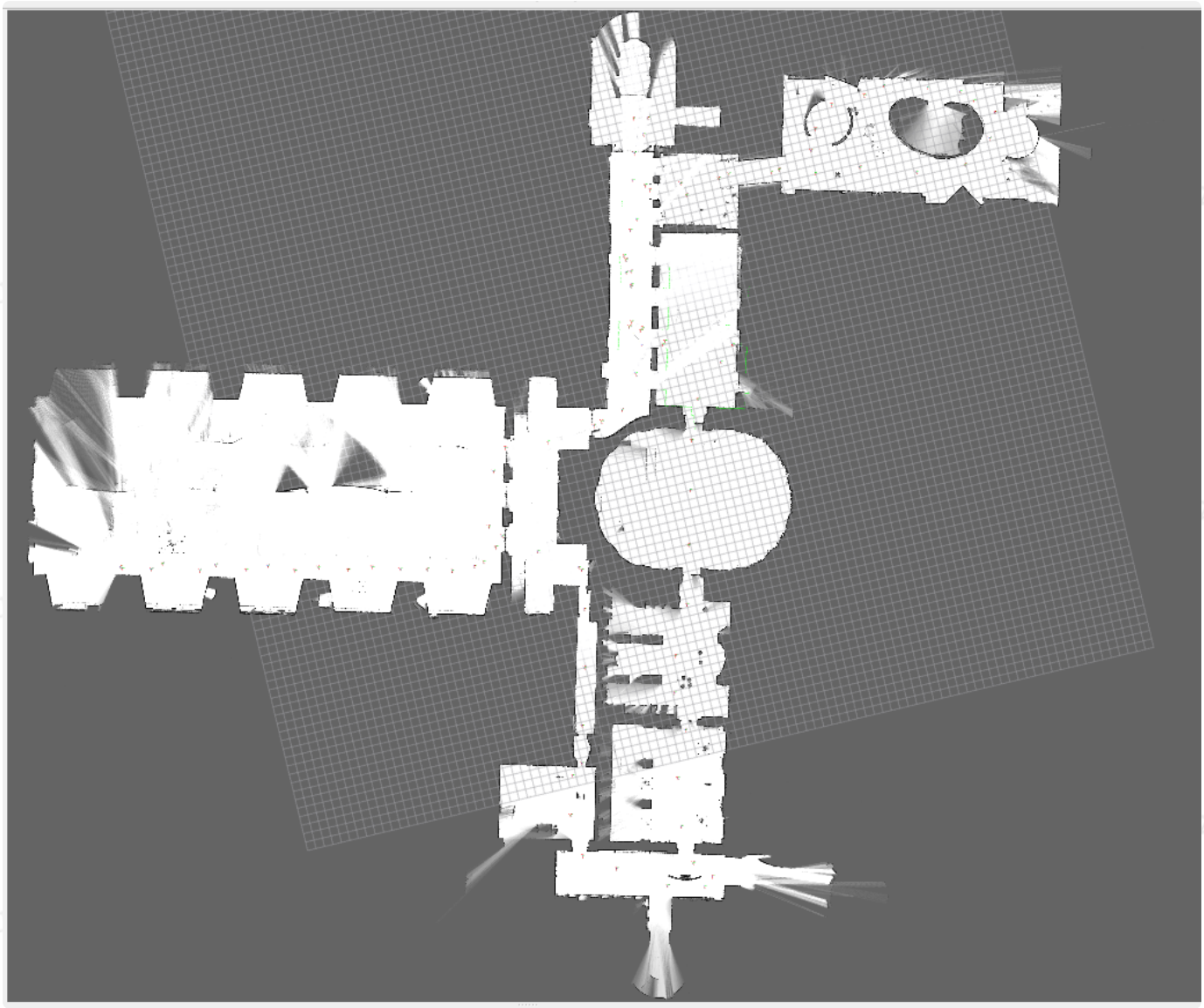
- 参数在cartographer的pose_graph.lua（与全局优化有关）文件中
- 该参数表示采样率，通过将采样实现限制约束条件的数量和计算量，此值过低会使得约束失效，无法正确回环检测，此值过高或增加计算负荷，影响实时性。

sampling_ratio=0.0005



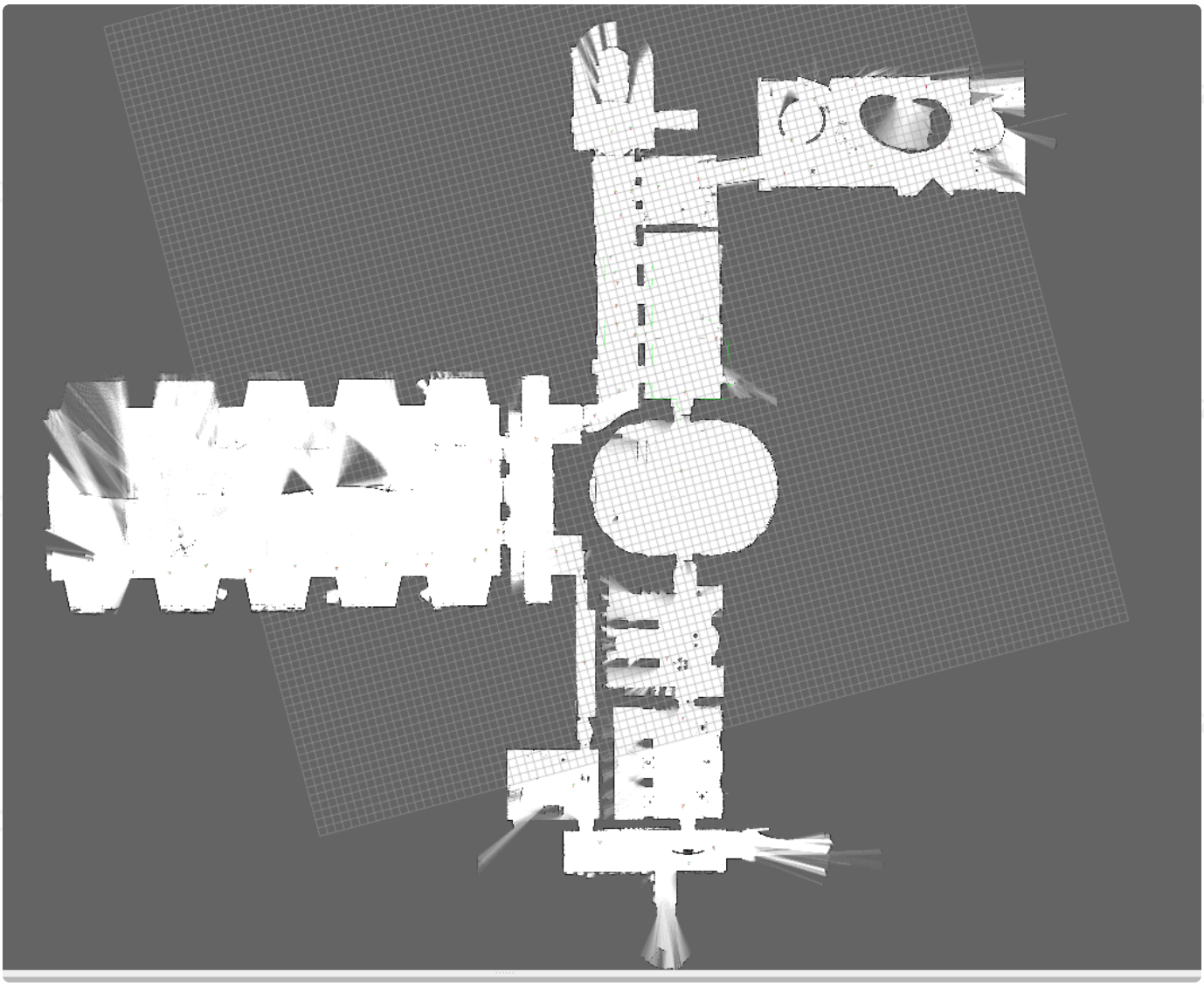
哈工大威海创新平台 - 718—刘渝婉(22971967)

sampling_ratio=0.005



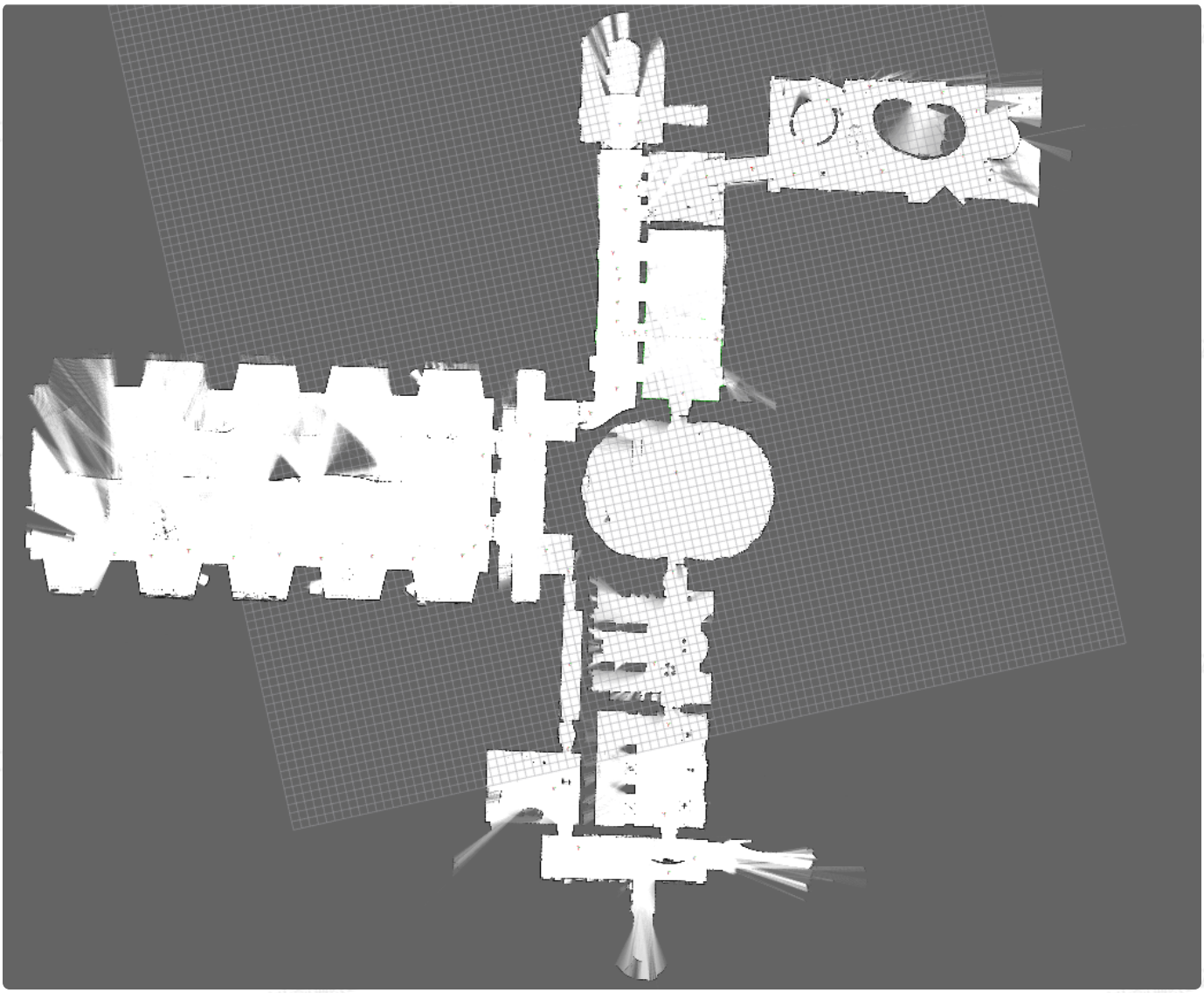
哈工大威海创新平台 - 718—刘渝婉(22971967)

sampling_ratio=0.01



sampling_ratio=0.3（原始值）

哈工大威海创新平台 - 718—刘渝婉(22971967)



调参结果：

将此值在保证未出现约束失效的情况下进行了减小，减小了计算负荷，保证实时性

筛选最佳参数值：

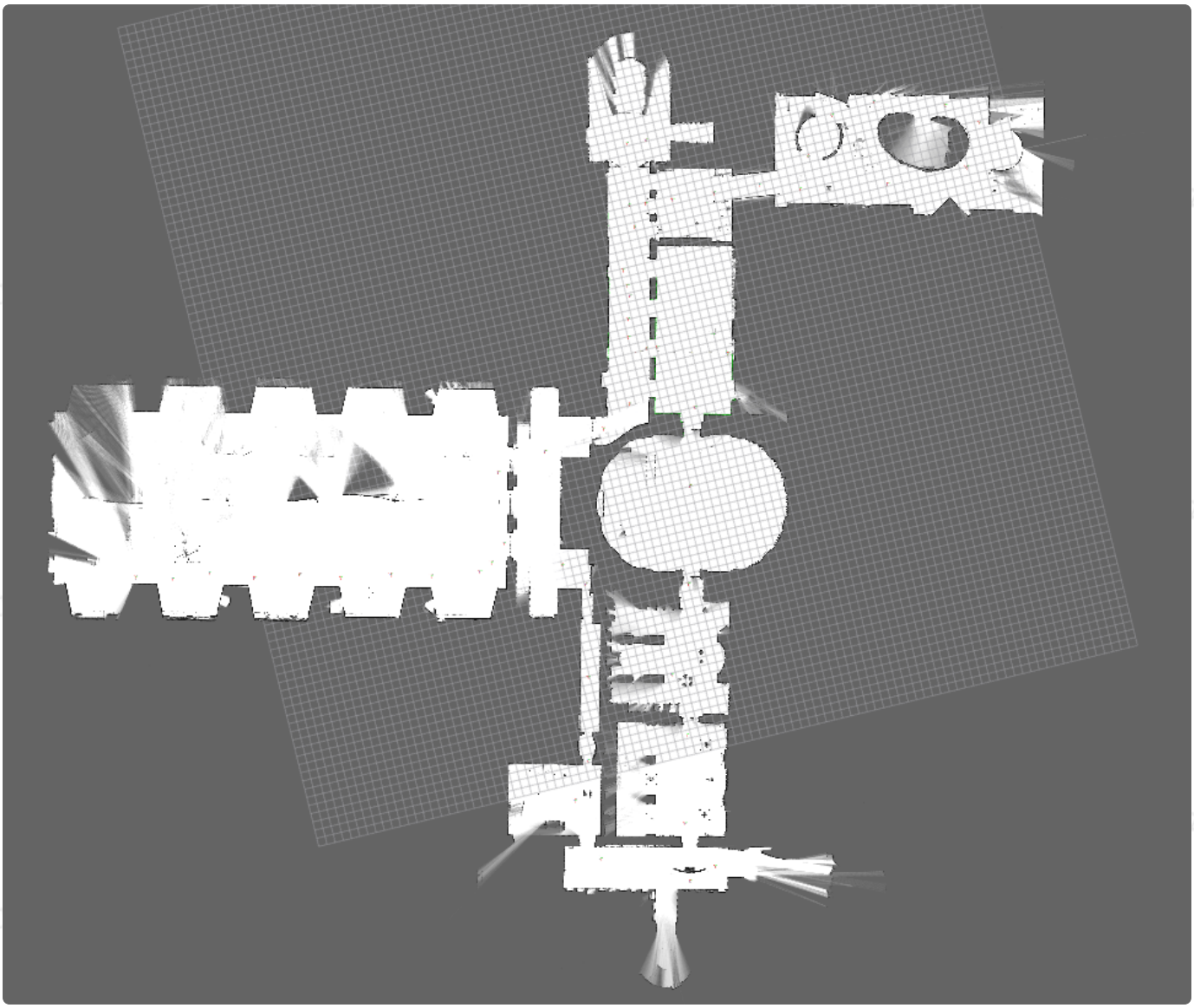
sampling_ratio=0.005

7.1.2min_score：

参数介绍：

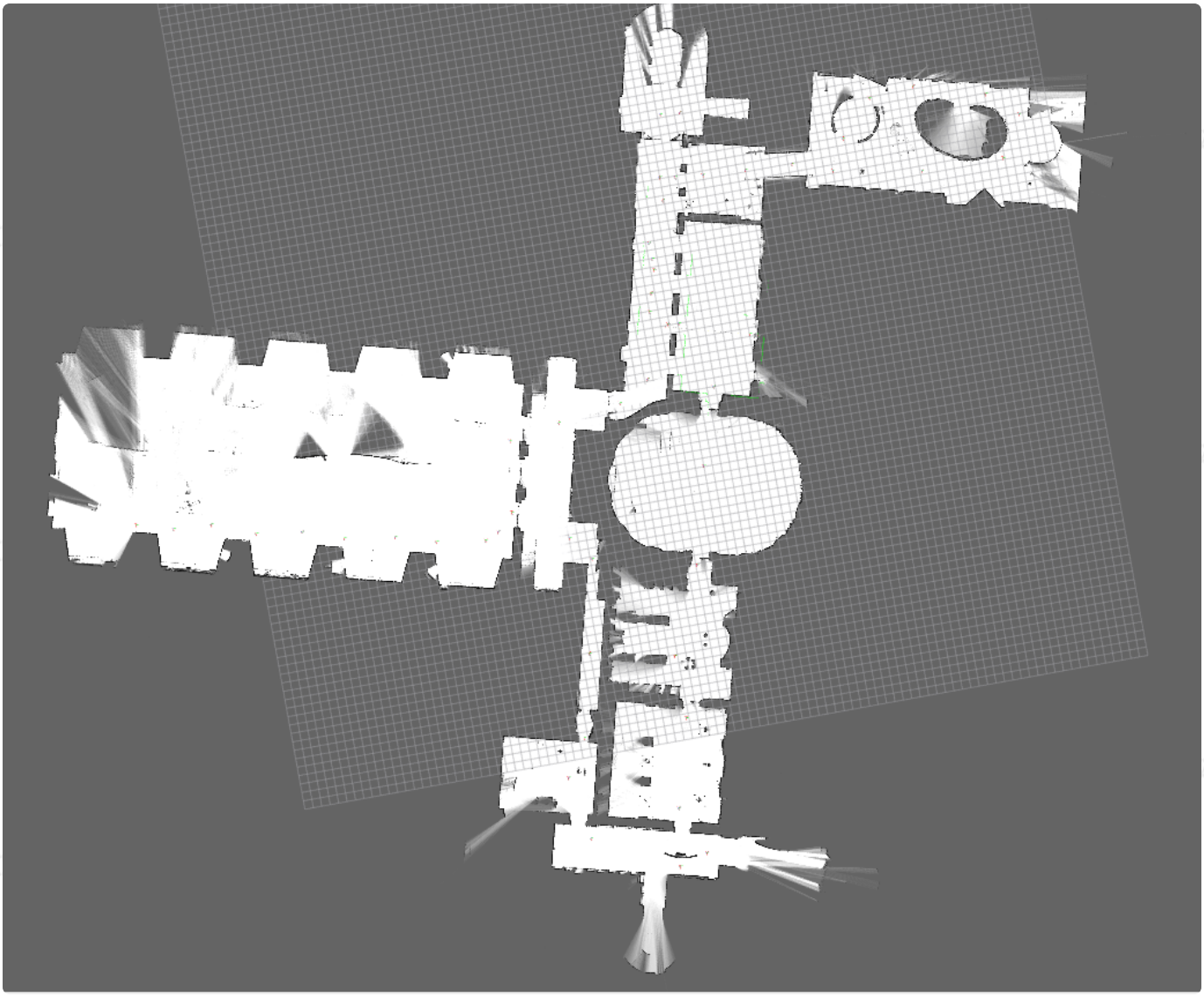
- 参数在cartographer的pose_graph.lua（与全局优化有关）文件中
- 该参数表示约束的最低分数，这个值越大，计算速度相对越快，约束数量相对越少，在sampling_ratio较小的情况下，很可能会造成约束过少而导致建图失败，这个值越小，则效果相反（当FastCorrelativeScanMatcher初步匹配的结果分数高于min_score时，才进入下一步的Ceres Scan Matcher处理）；

min_score=0.55(原始值)



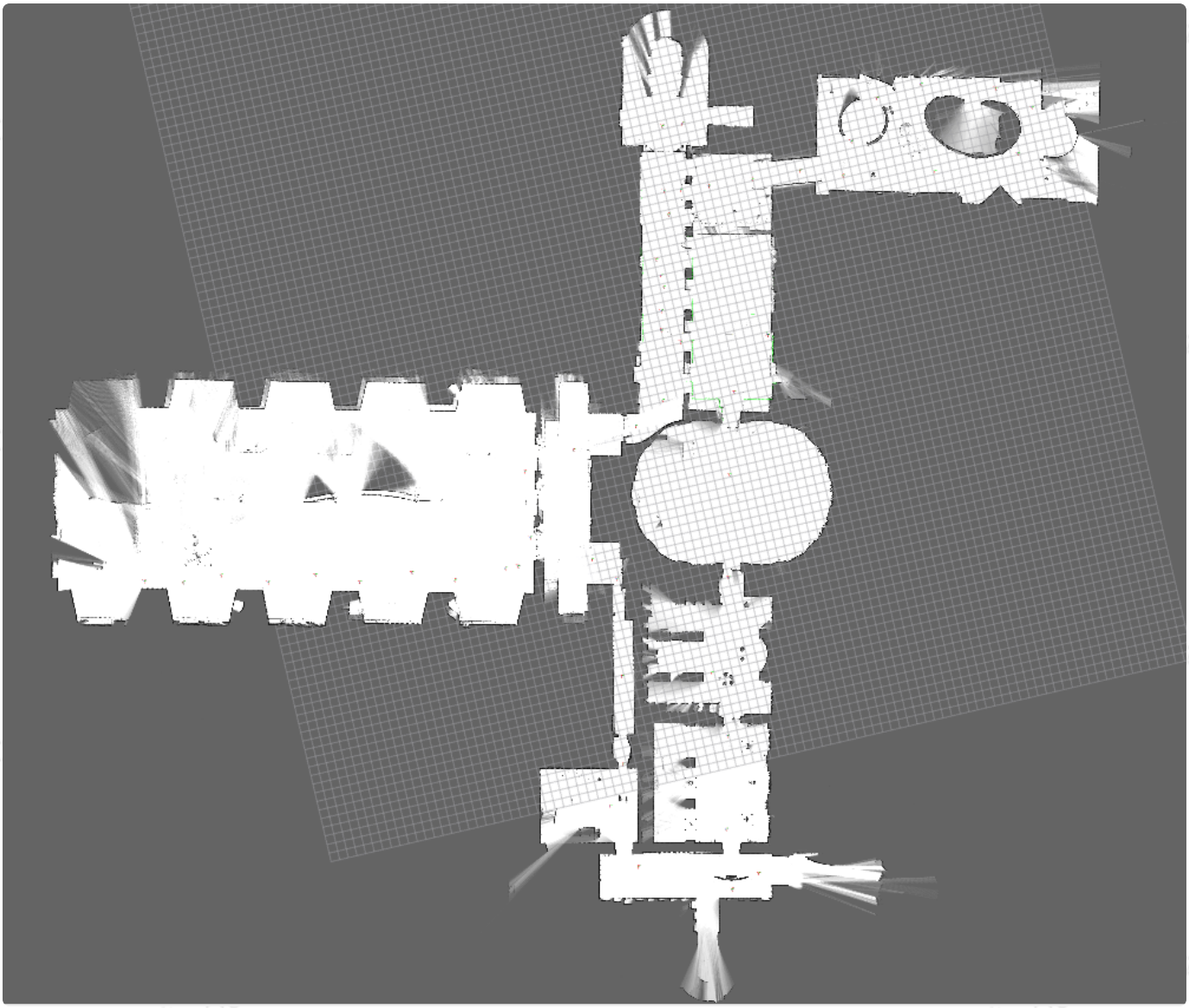
min_score=0.7

哈工大威海创新平台 - 718—刘渝婉(22971967)



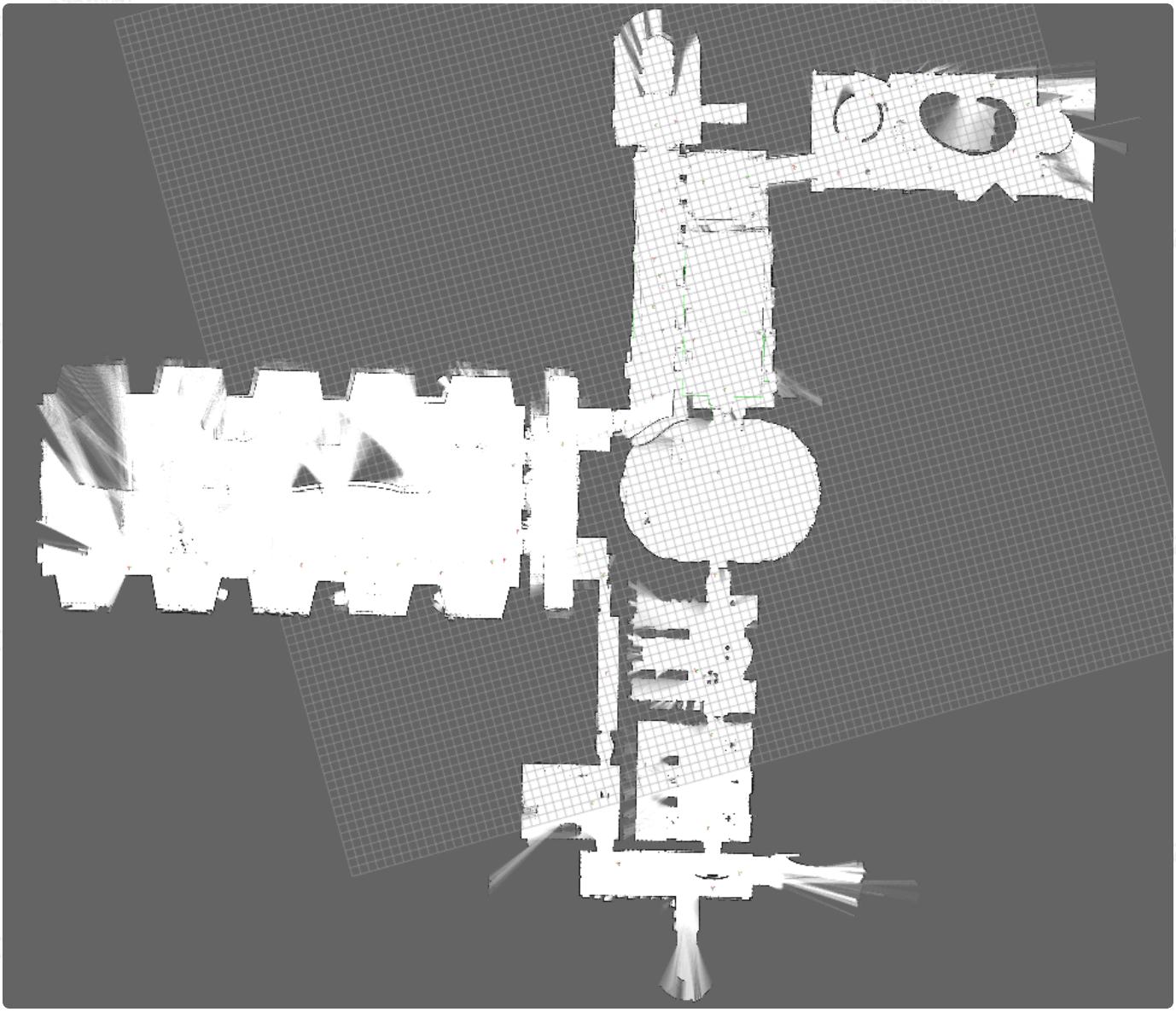
min_score=0.8

哈工大威海创新平台 - 718—刘渝婉(22971967)



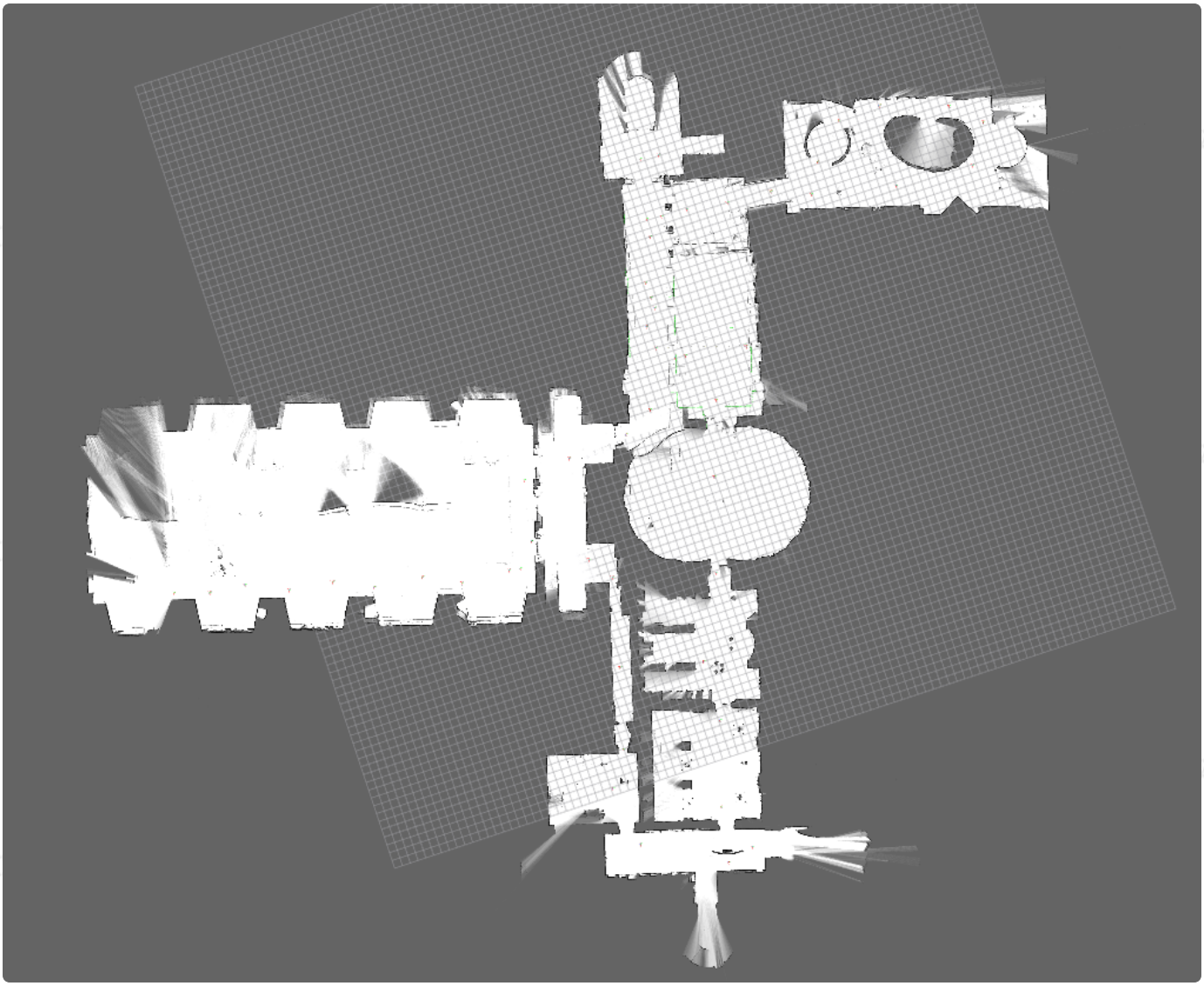
min_score=0.85

哈工大威海创新平台 - 718—刘渝婉(22971967)



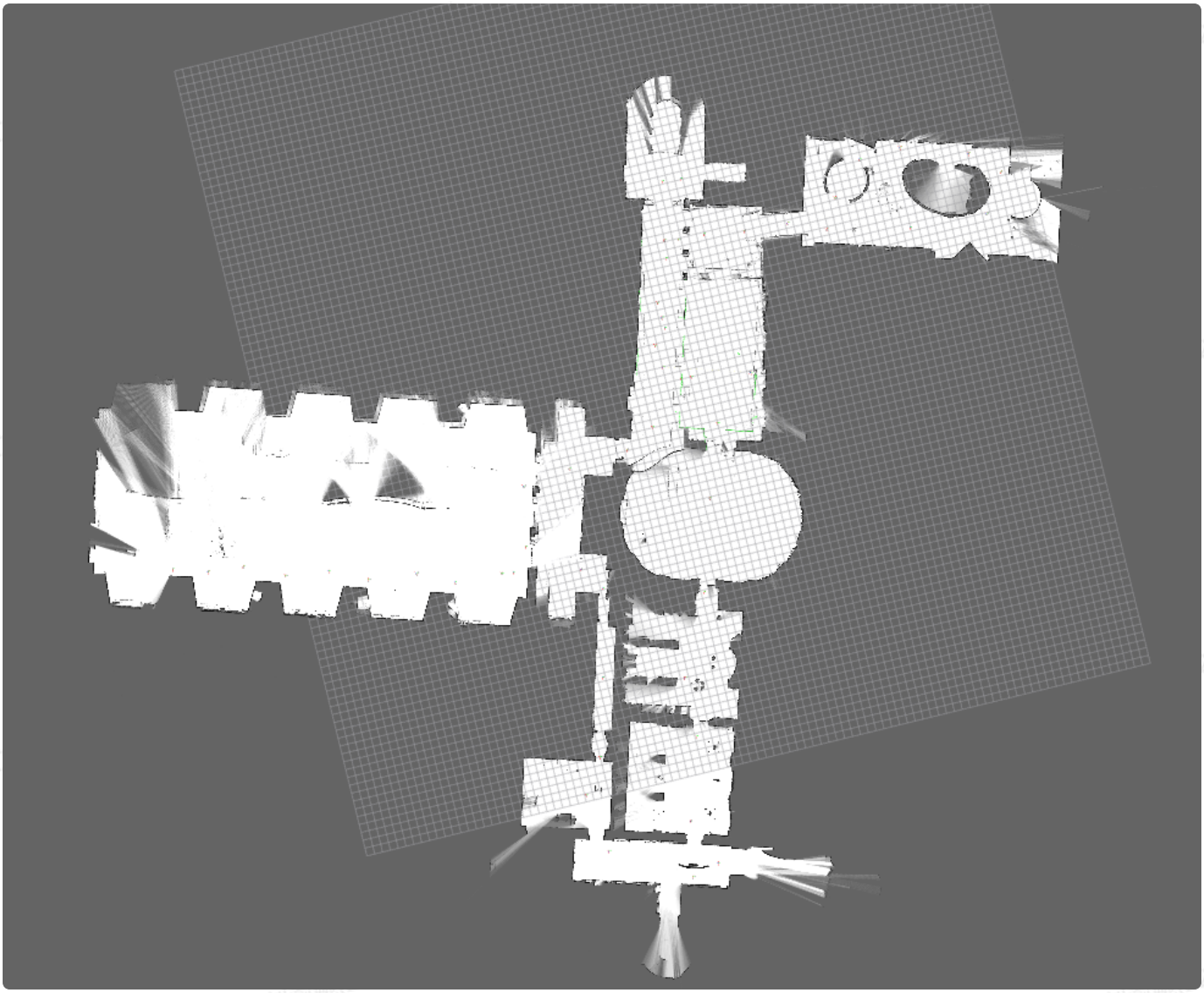
min_score=0.9

哈工大威海创新平台 - 718—刘渝婉(22971967)



minscore=1.5

哈工大威海创新平台 - 718—刘渝婉(22971967)



调参效果：

在建图效果没有太大变化的情况下，适当提高了min_score的值，相对提高了计算速度

筛选最佳参数值：

min_score=0.7

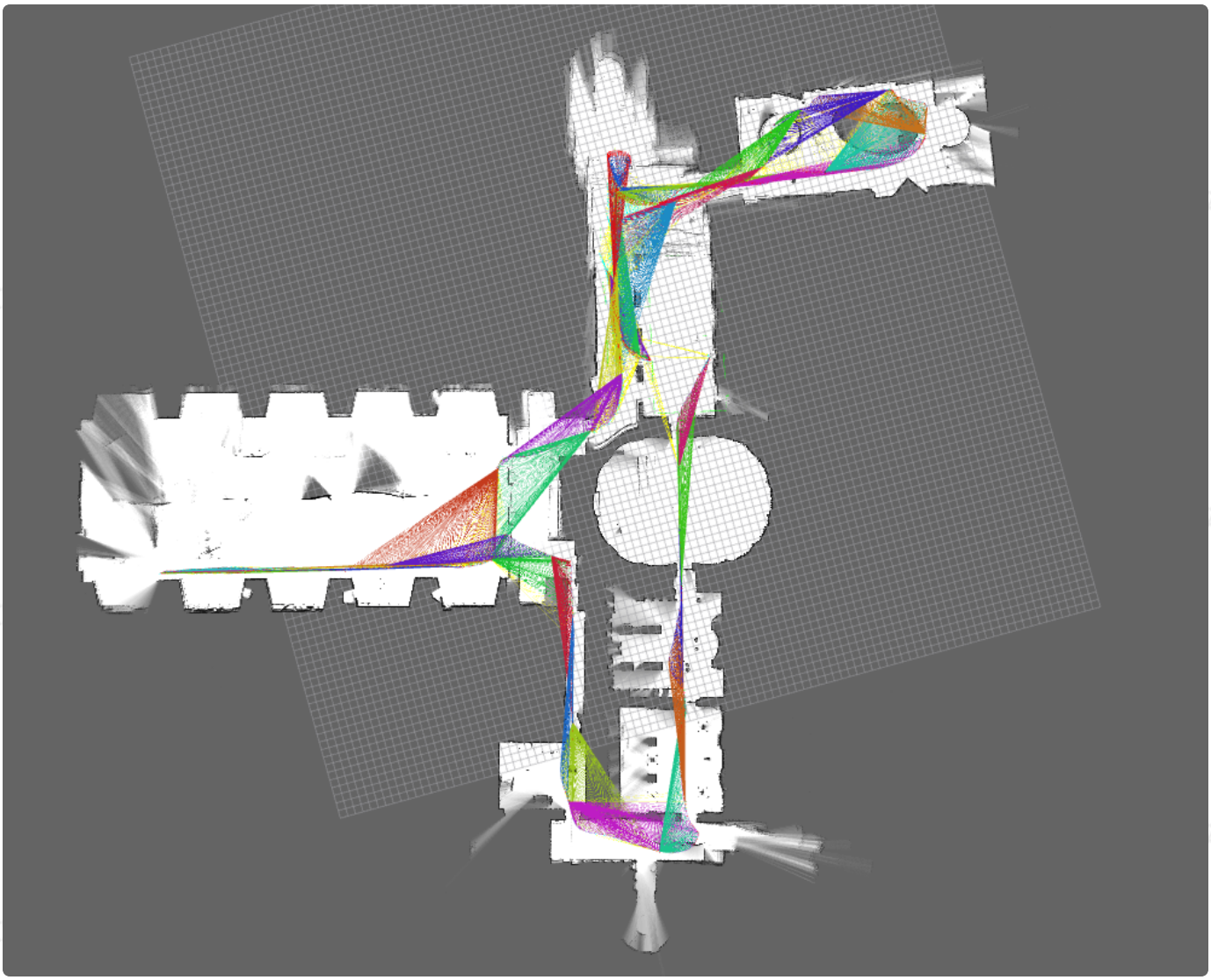
7.1.3num_subdivisions_per_laser_scan：

参数介绍：

- 在cartographer_ros/configuration_files/backpack_2d.lua中
- 该参数表示每扫描几帧作为一帧

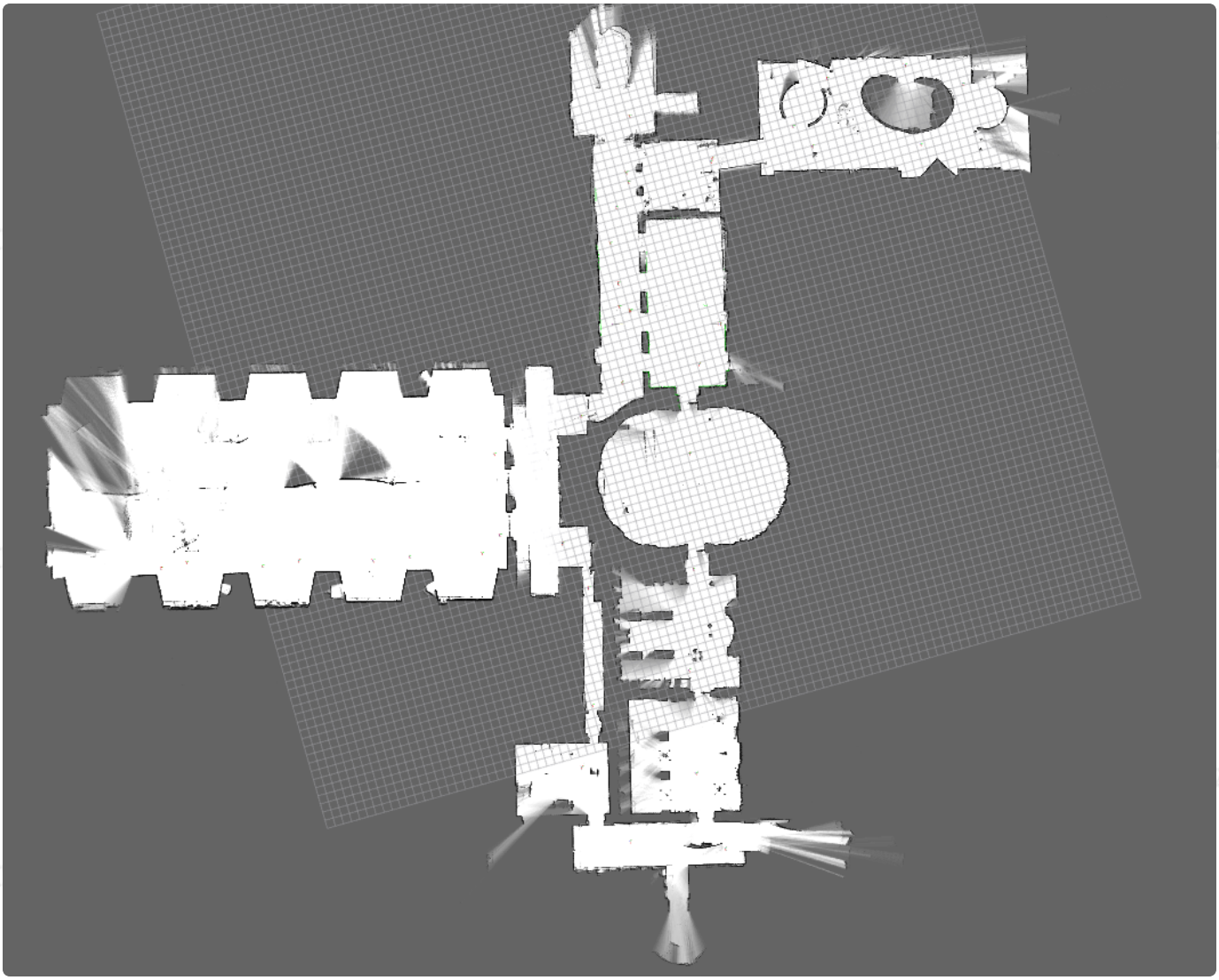
哈工大威海创新平台 - 718—刘渝婉(22971967)

num_subdivisions_per_laser_scan = **3**



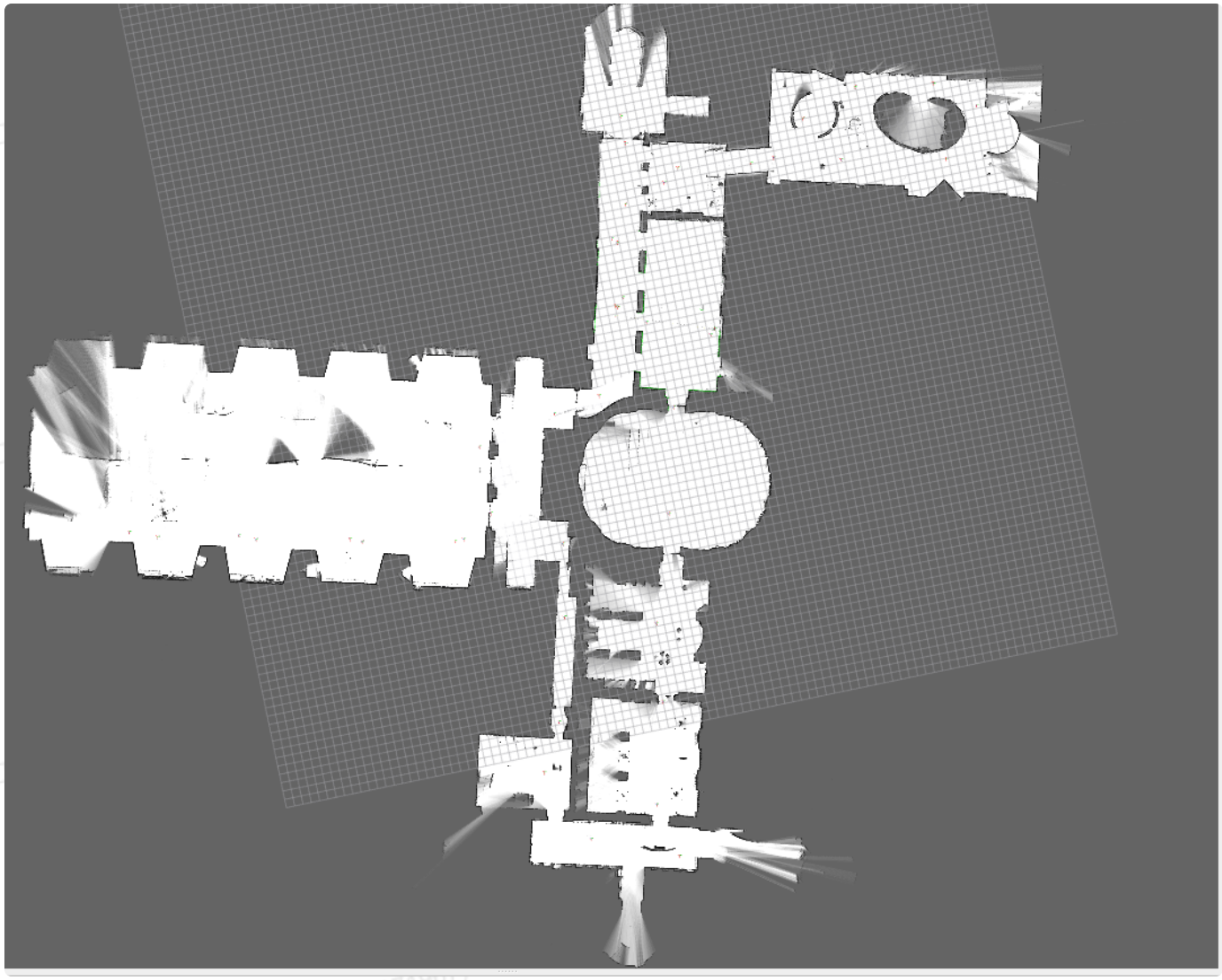
num_subdivisions_per_laser_scan = **4**

哈工大威海创新平台 - 718—刘渝婉(22971967)



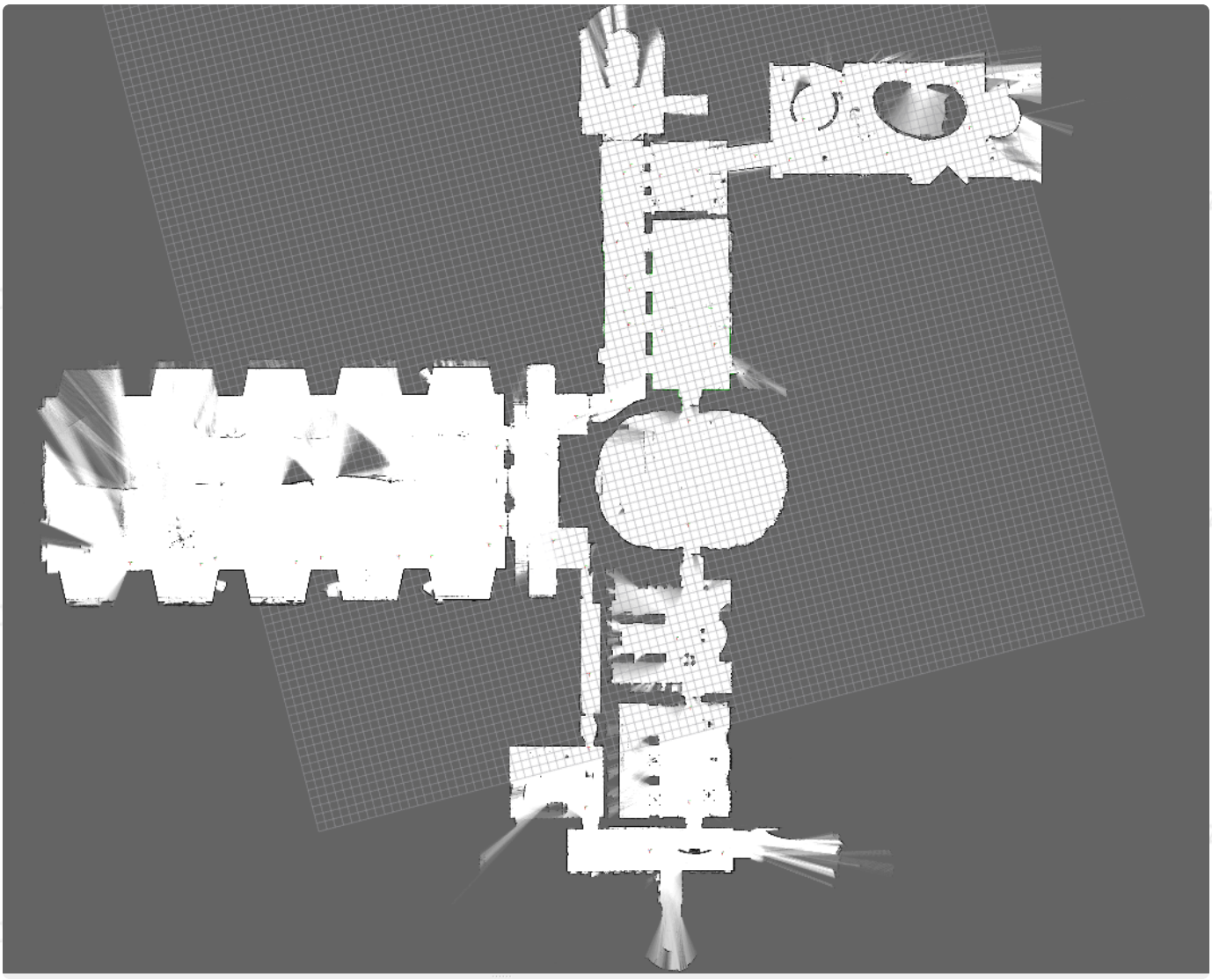
num_subdivisions_per_laser_scan = 5

哈工大威海创新平台 - 718—刘渝婉(22971967)



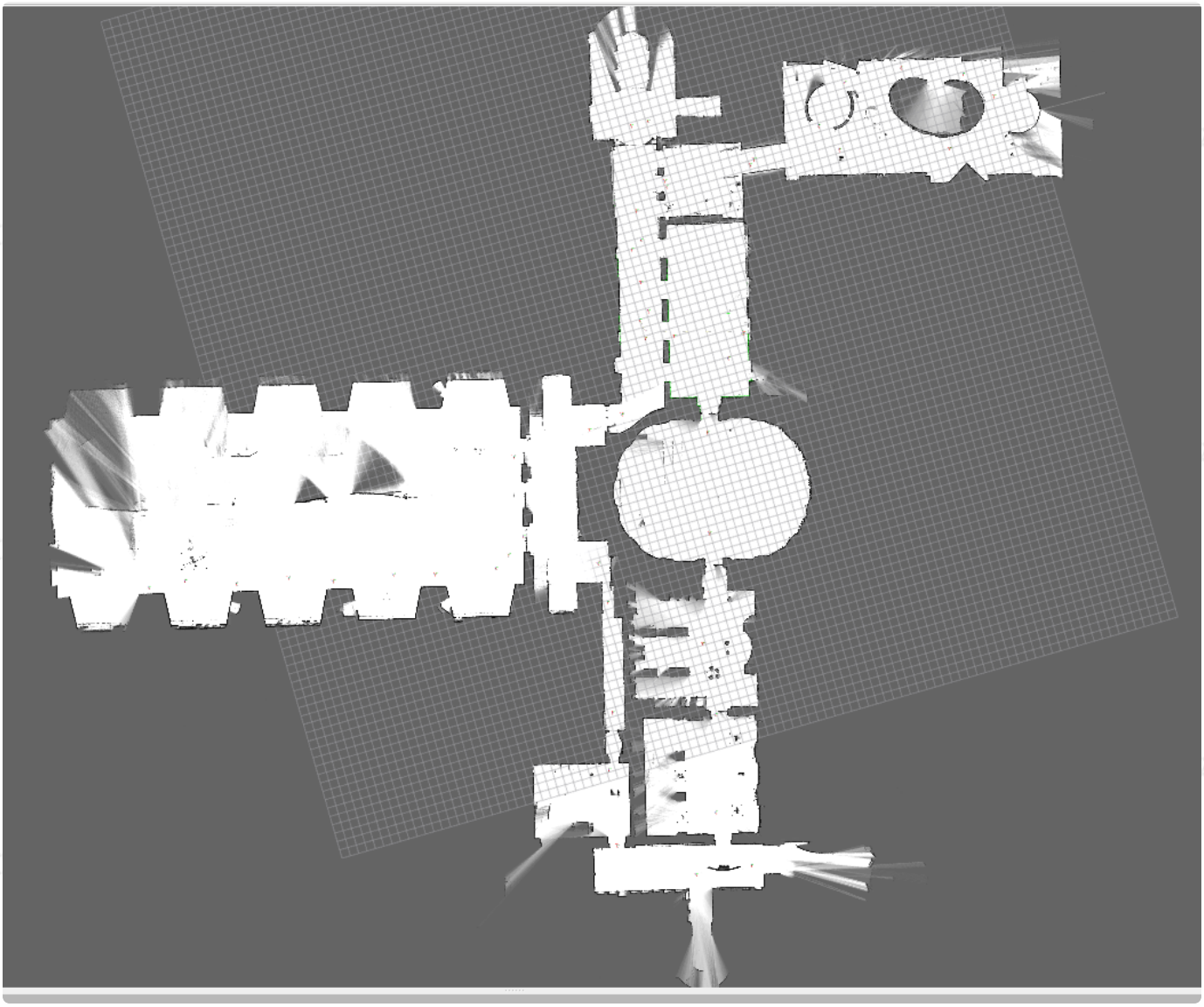
哈工大威海创新平台 - 718—刘渝婉(22971967)

num_subdivisions_per_laser_scan = **6**



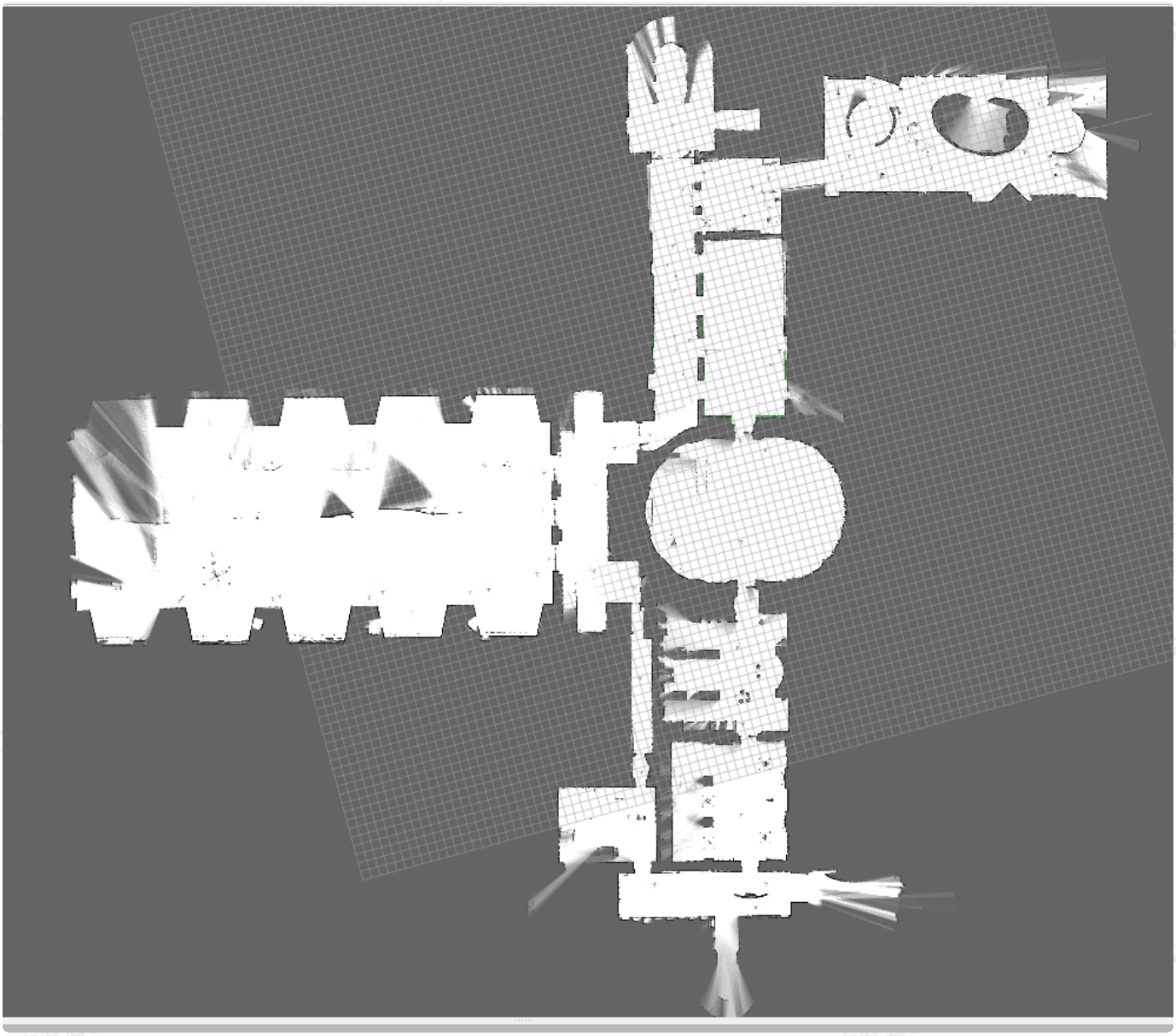
num_subdivisions_per_laser_scan = **7**

哈工大威海创新平台 - 718—刘渝婉(22971967)



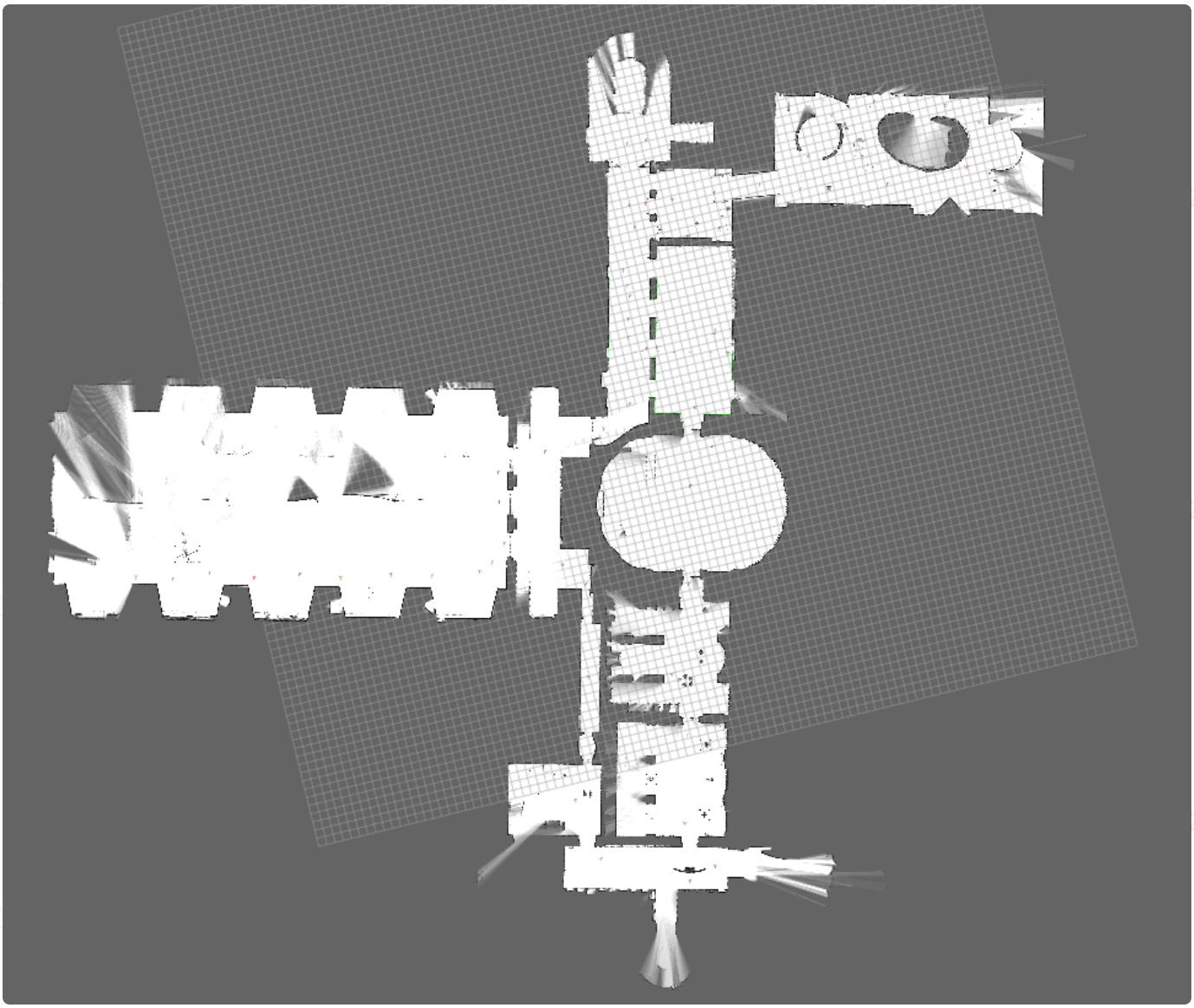
num_subdivisions_per_laser_scan = **9**

哈工大威海创新平台 - 718—刘渝婉(22971967)



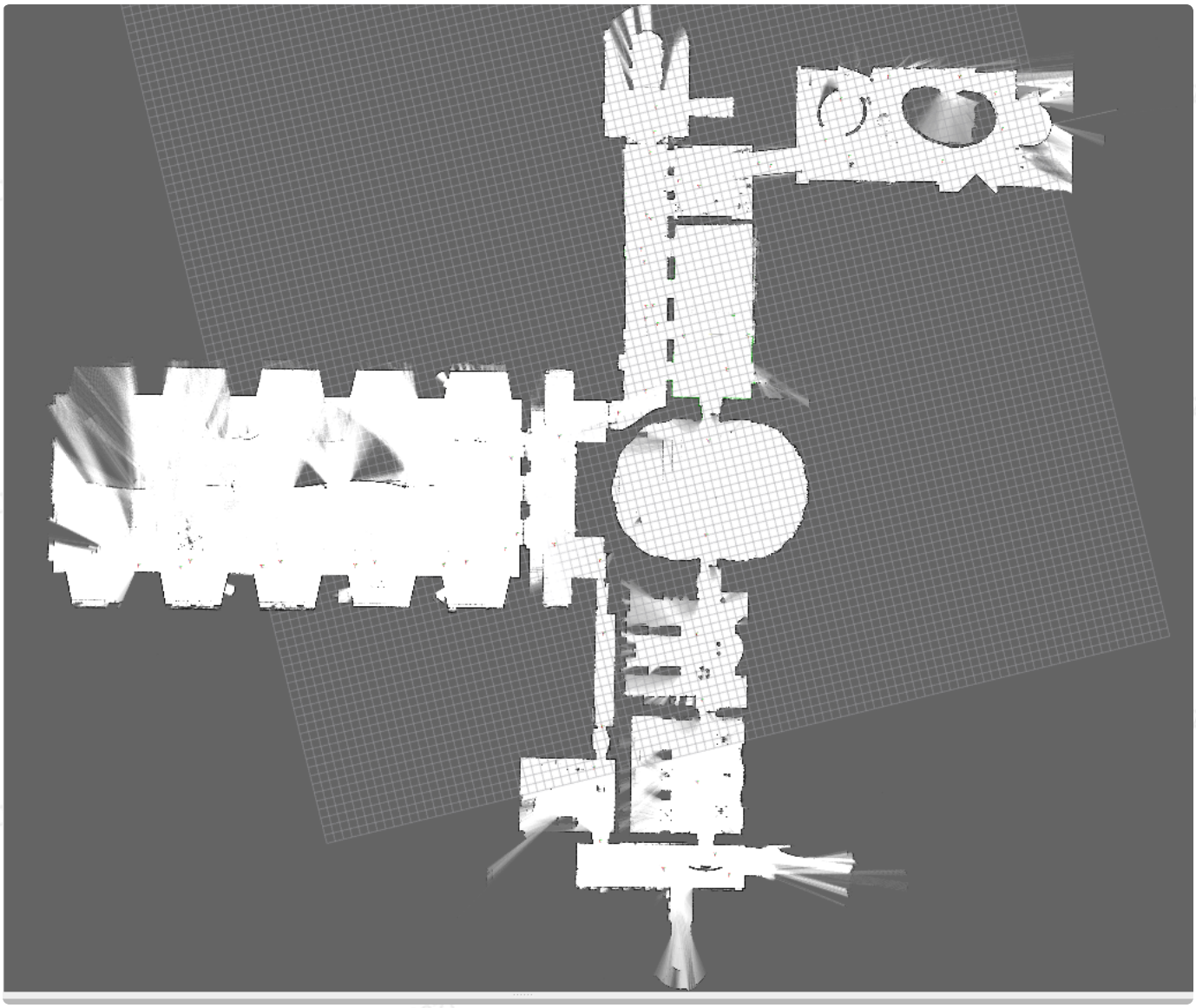
哈工大威海创新平台 - 718—刘渝婉(22971967)

num_subdivisions_per_laser_scan = 10（原始值）



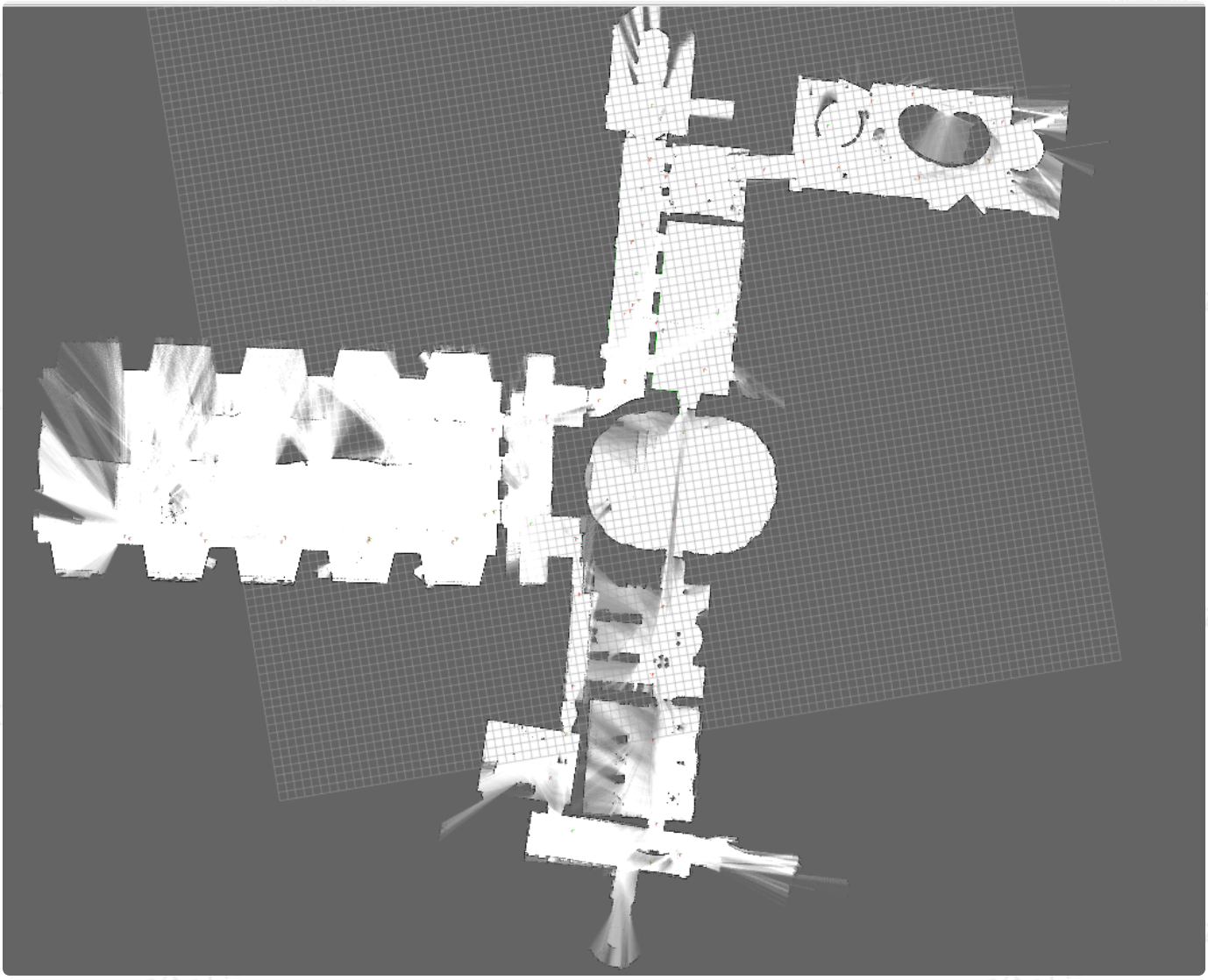
哈工大威海创新平台 - 718—刘渝婉(22971967)

num_subdivisions_per_laser_scan = 12



哈工大威海创新平台 - 718—刘渝婉(22971967)

num_subdivisions_per_laser_scan = 20,



调参结论：

原代码中该参数的值10附近的值都有较好的建图效果，该参数数值过小或过大会出现建出来的地图发生漂移、地图不完整的现象。

筛选最佳参数值：

num_subdivisions_per_laser_scan = 10（原始值）

7.1.4num_accumulated_range_data:

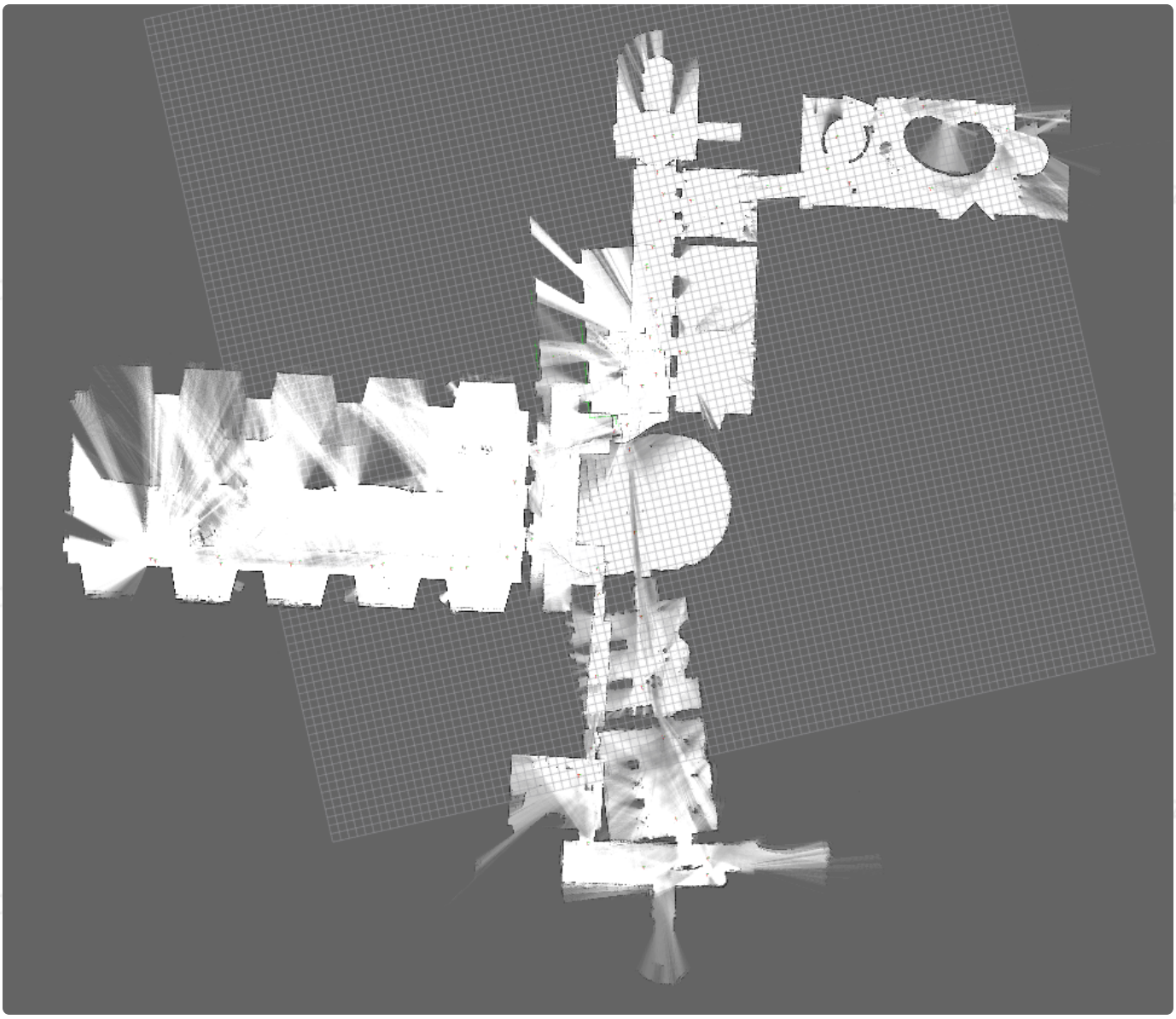
参数介绍：

该参数表示每几帧取一帧，调低该参数可以防止雷达帧率太高耗费计算资源，减少计算量

备注：

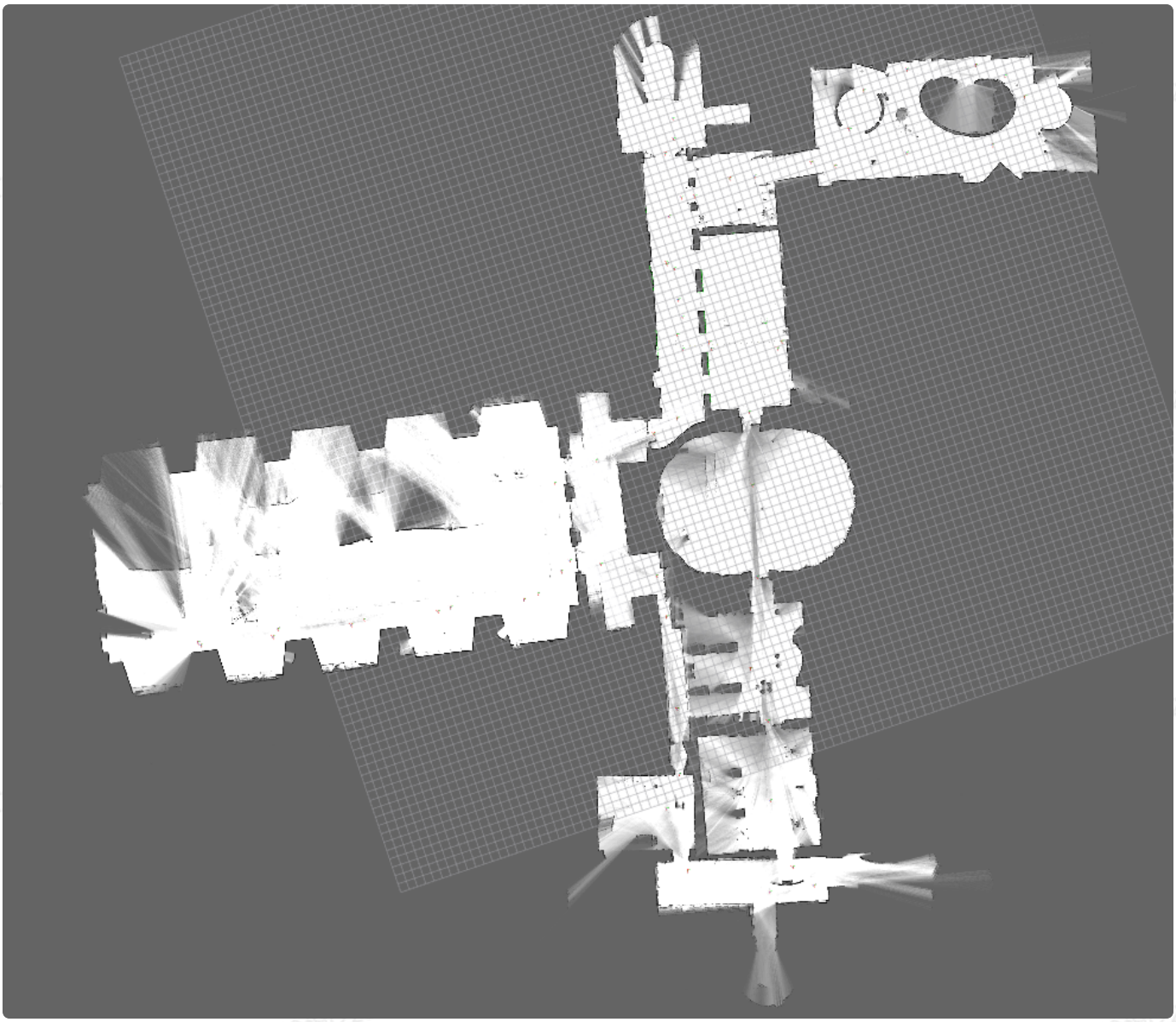
在num_subdivisions_per_laser_scan = 12、其他参数为原始值的情况下调的

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **4**



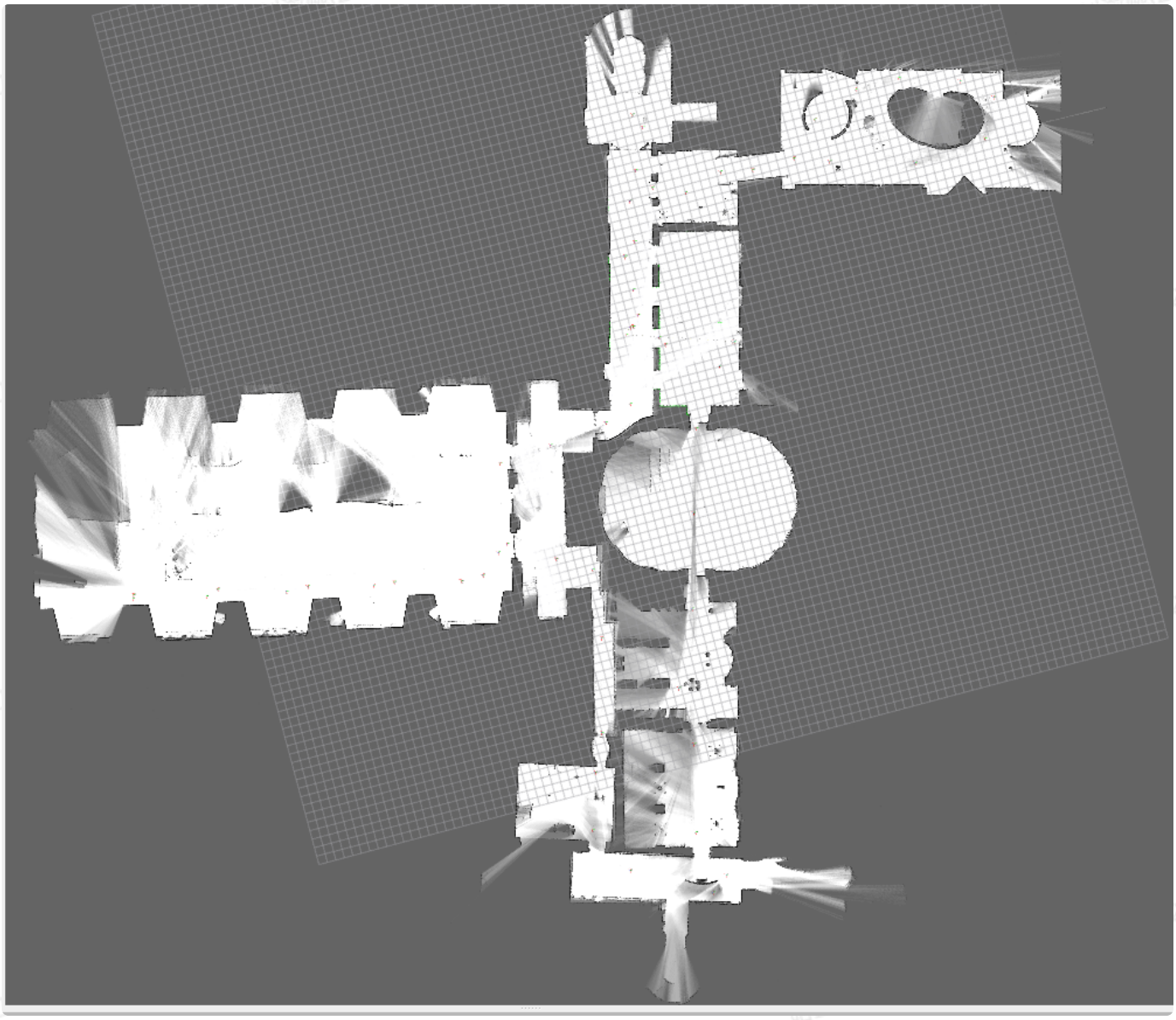
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **5**



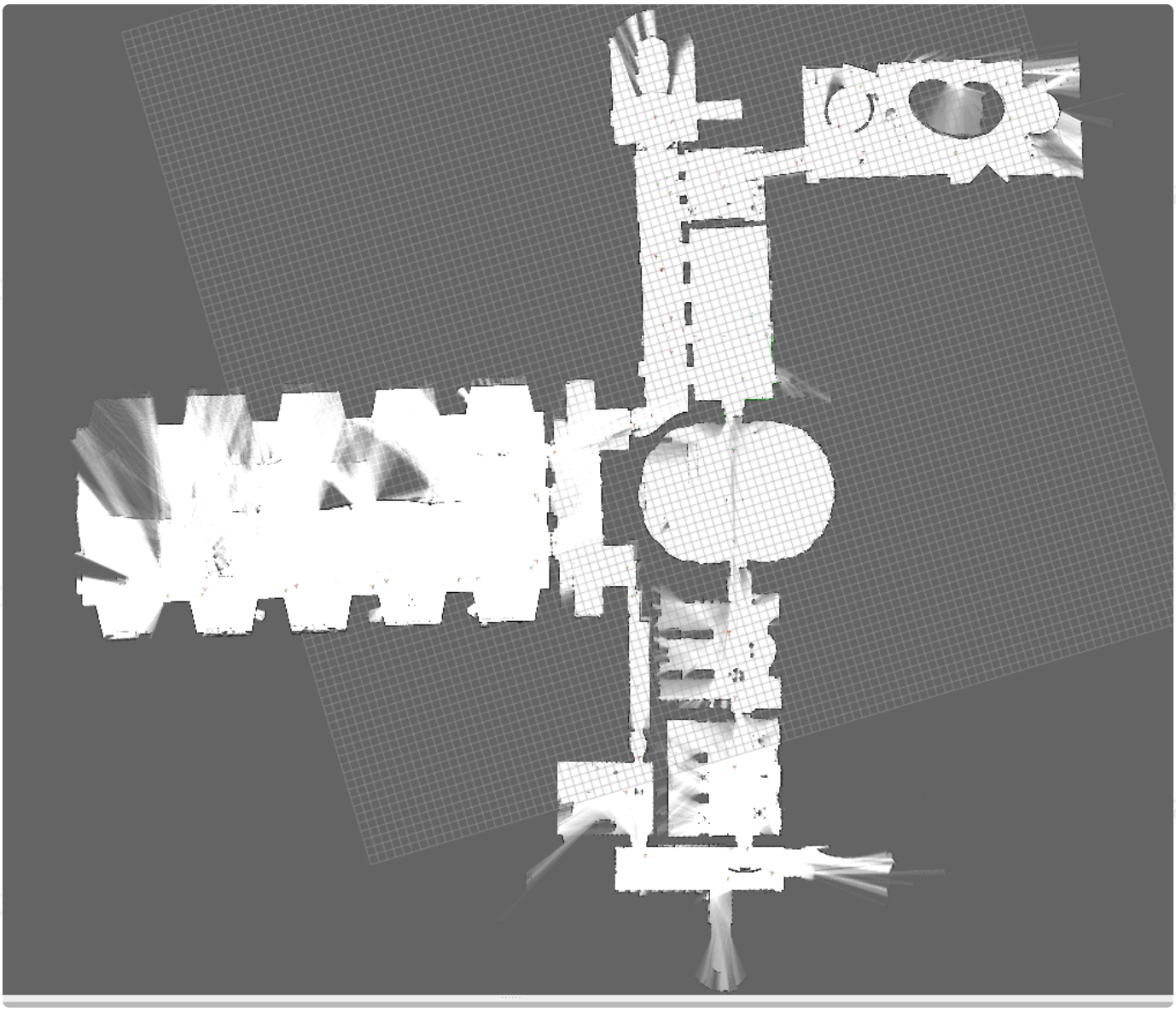
哈工大威海创新平台 - 718—刘瀚婉(22971967)

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **6**



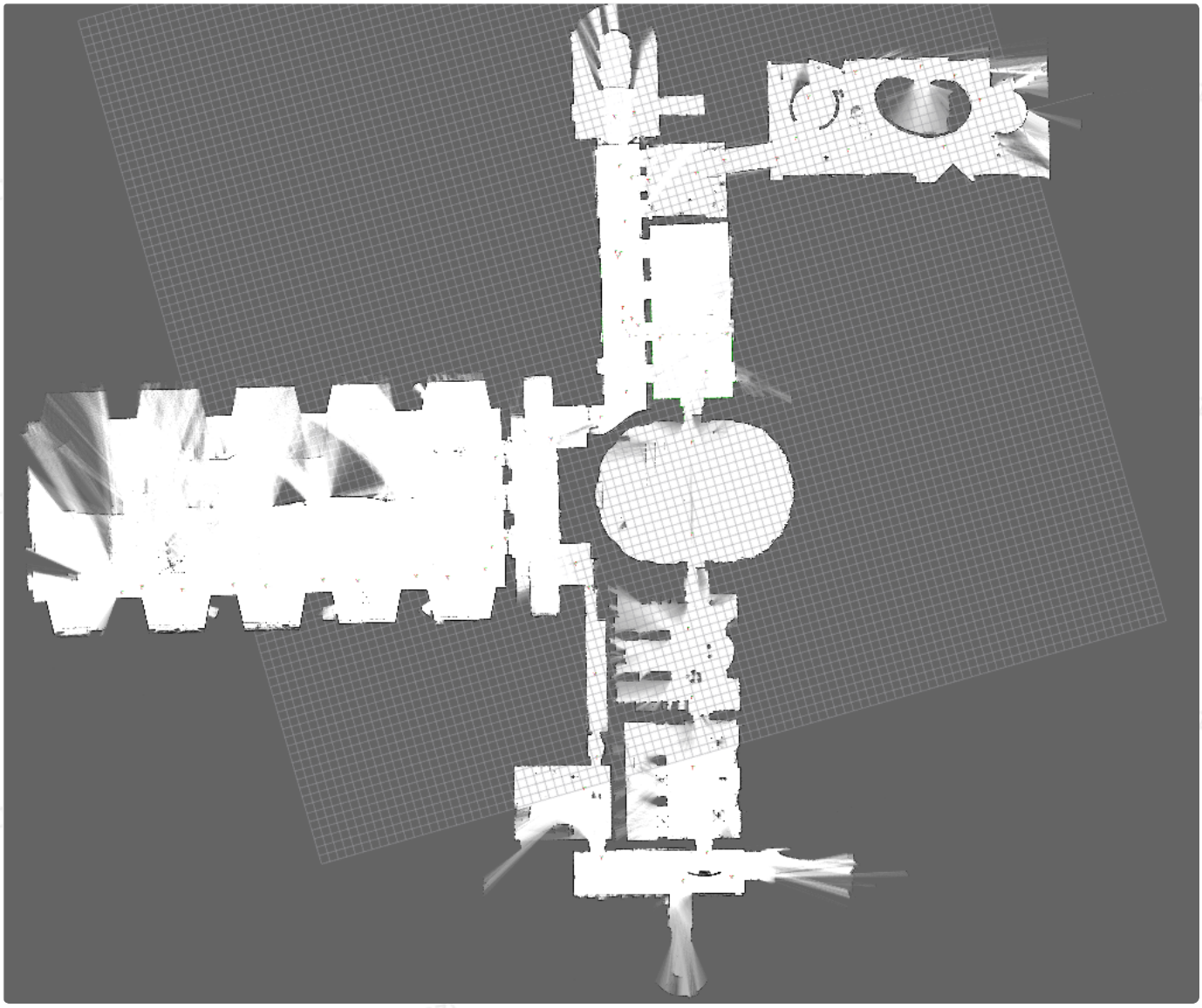
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = 7



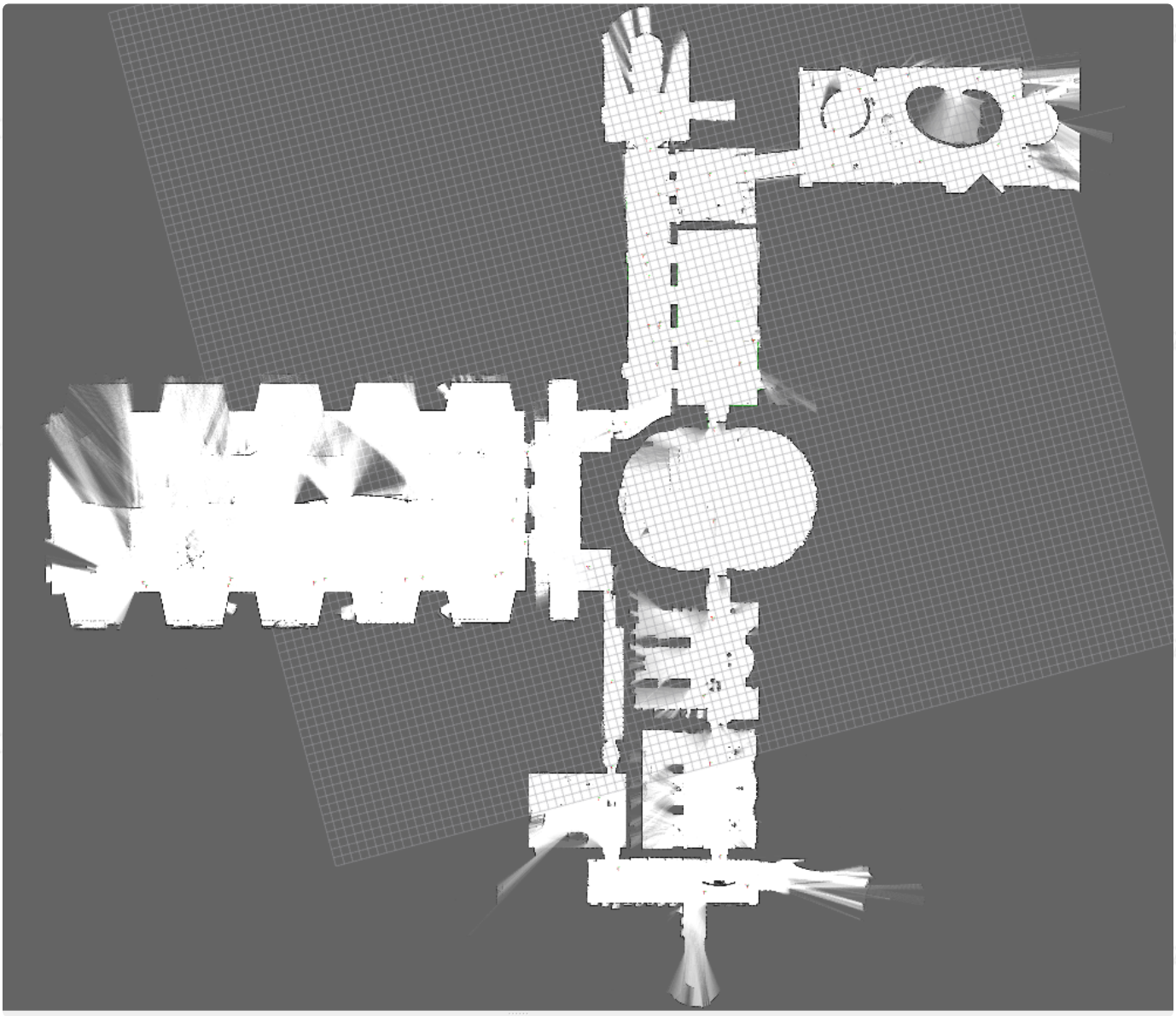
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **8**



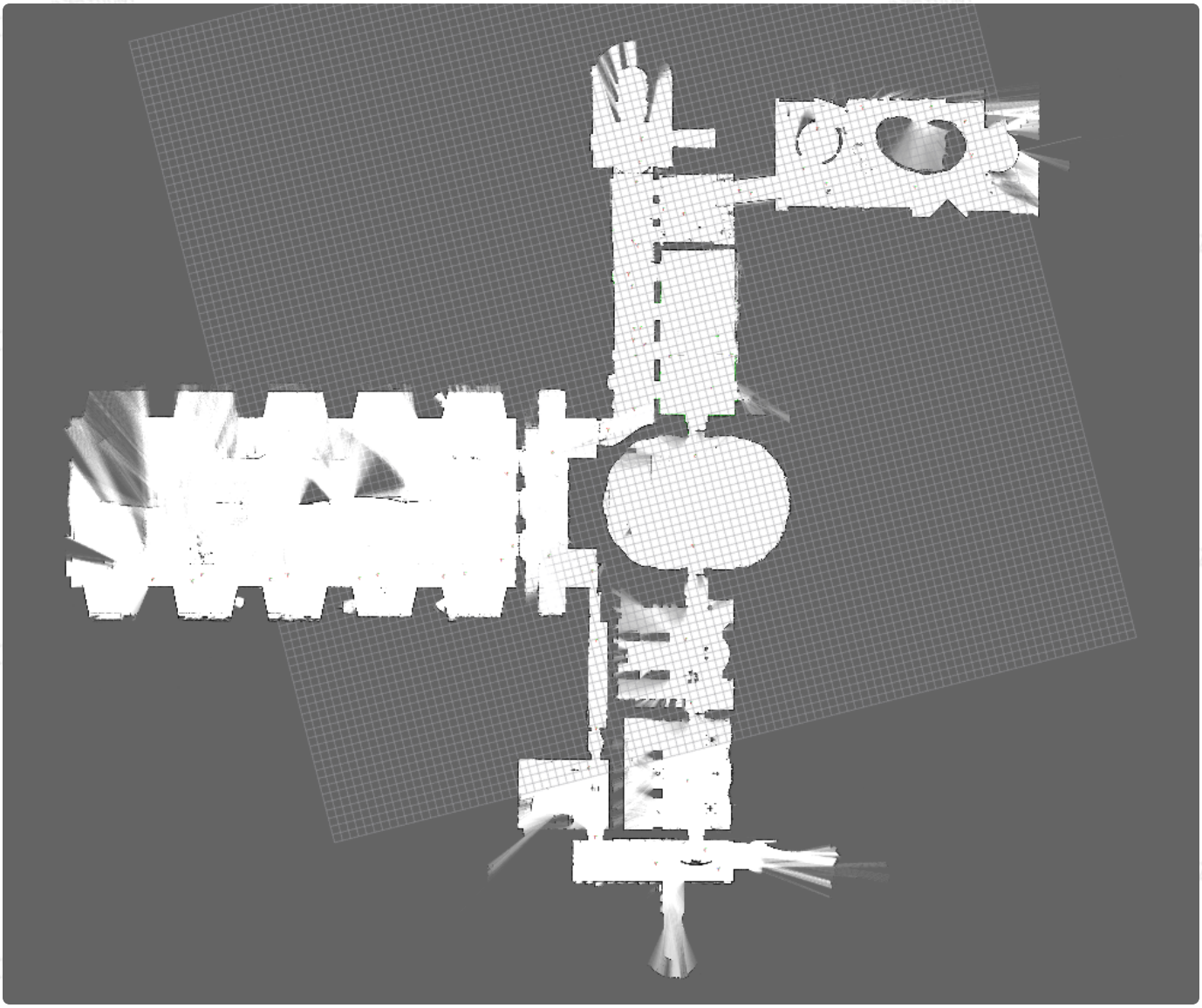
TRAJECTORY_BUILDER_2D.num_accumulated_range_data =**9**

哈工大威海创新平台 - 718—刘渝婉(22971967)



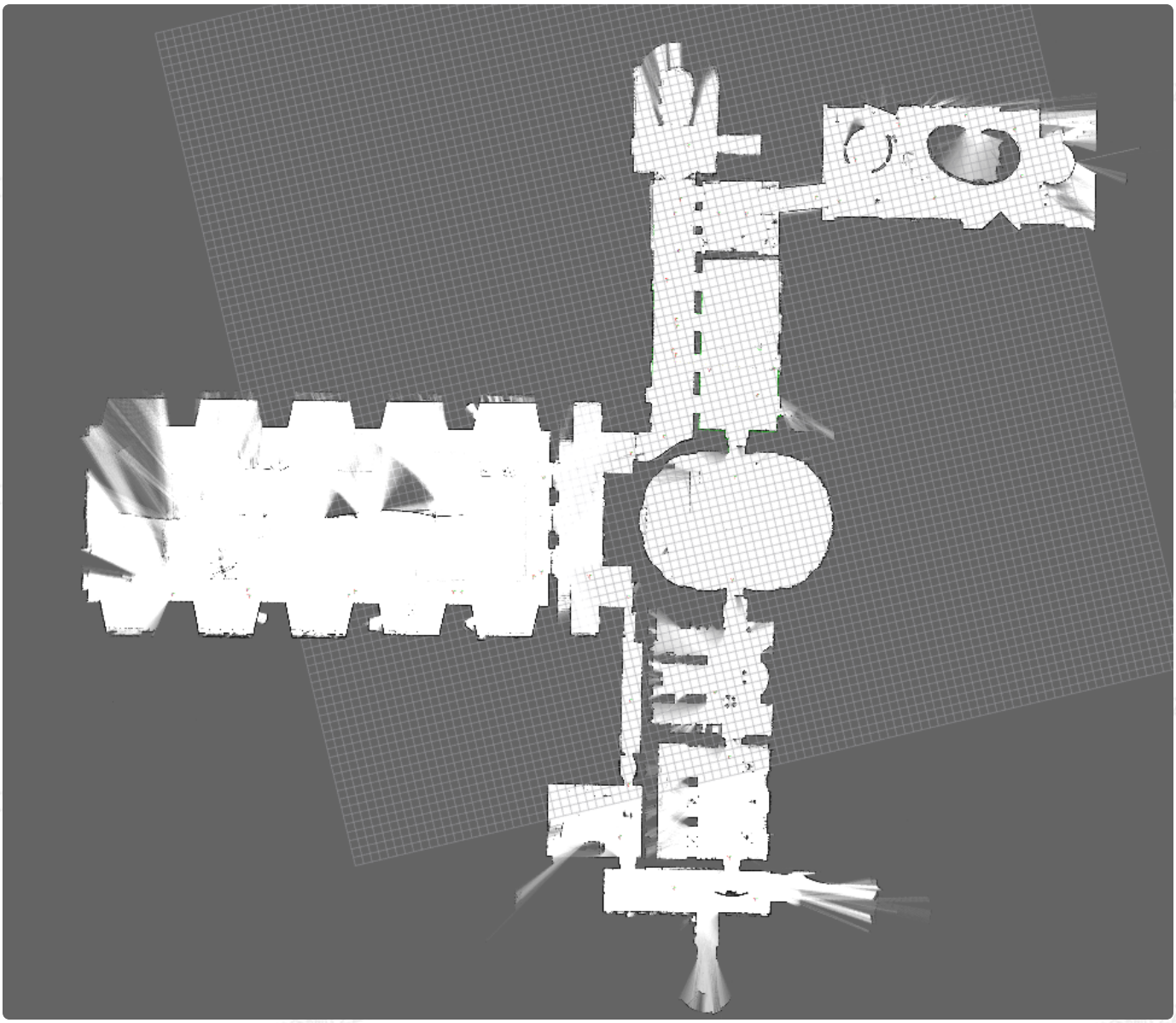
TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **10**（原始值）

哈工大威海创新平台 - 718—刘渝婉(22971967)



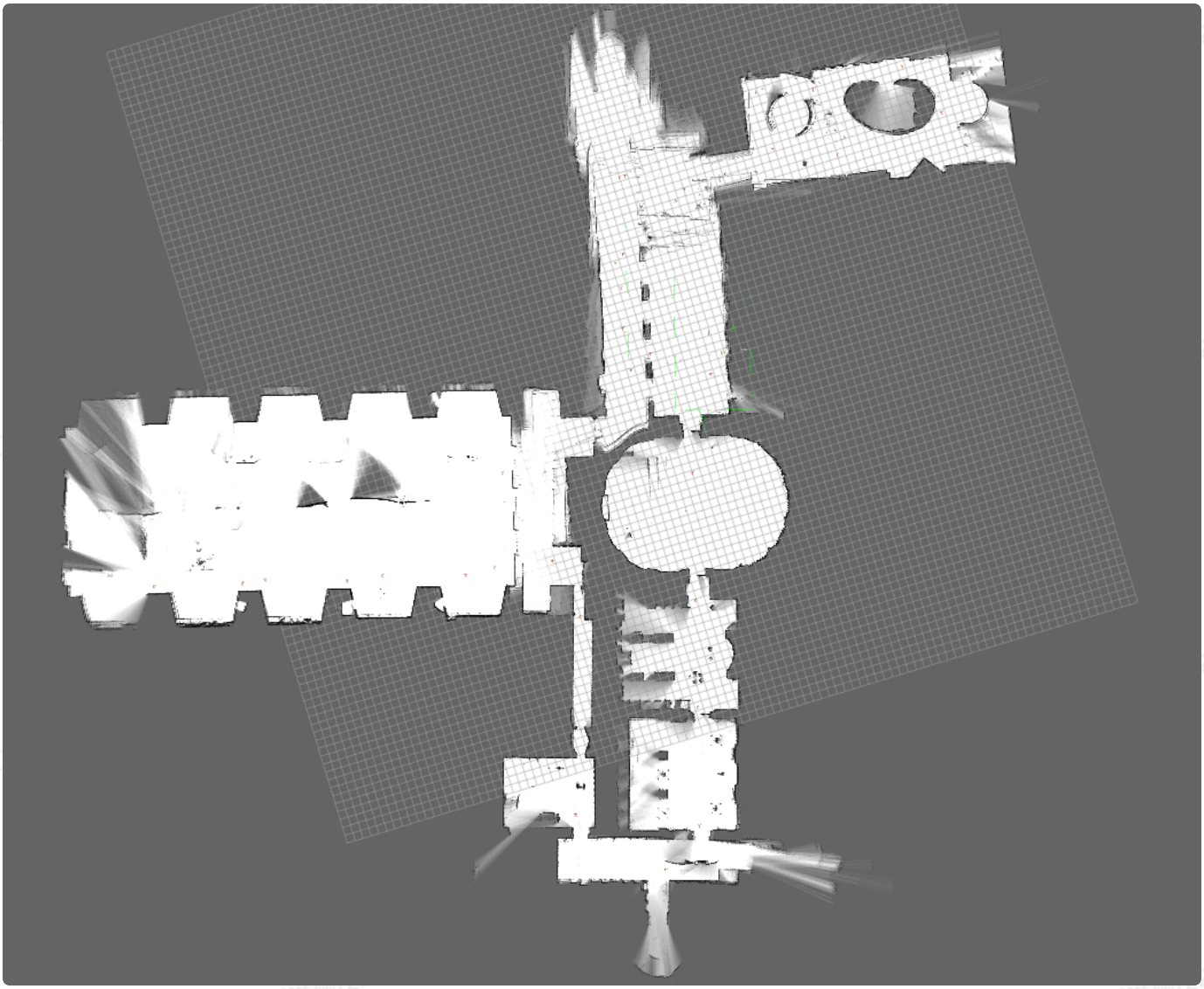
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **20**



TRAJECTORY_BUILDER_2D.num_accumulated_range_data = **40**

哈工大威海创新平台 - 718—刘瀚婉(22971967)



调参效果：

该参数过大或过小均会出现地图漂移的情况，调参的时候在保证建图效果无太大变化的情况下，适当调小了该参数数值，达到防止雷达帧率太高耗费计算资源，减少计算量的效果

筛选最佳参数值：

TRAJECTORY_BUILDER_2D.num_accumulated_range_data =9

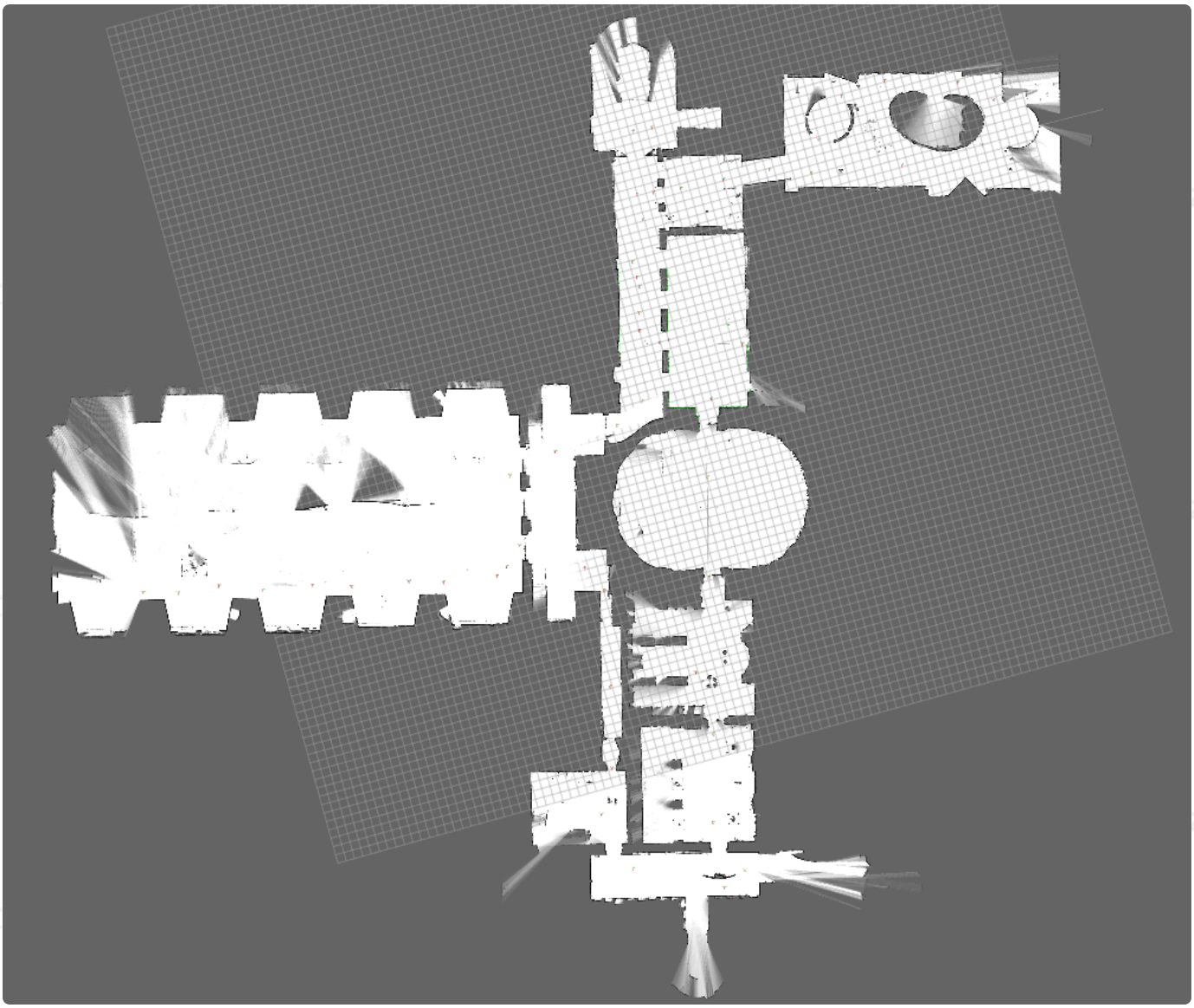
7.1.5missing_data_ray_length：

参数介绍：

- 在trajectory_builder_2d.lua中
- 该参数表示当传感器数据超出有效范围最大值时，按此值来处理。

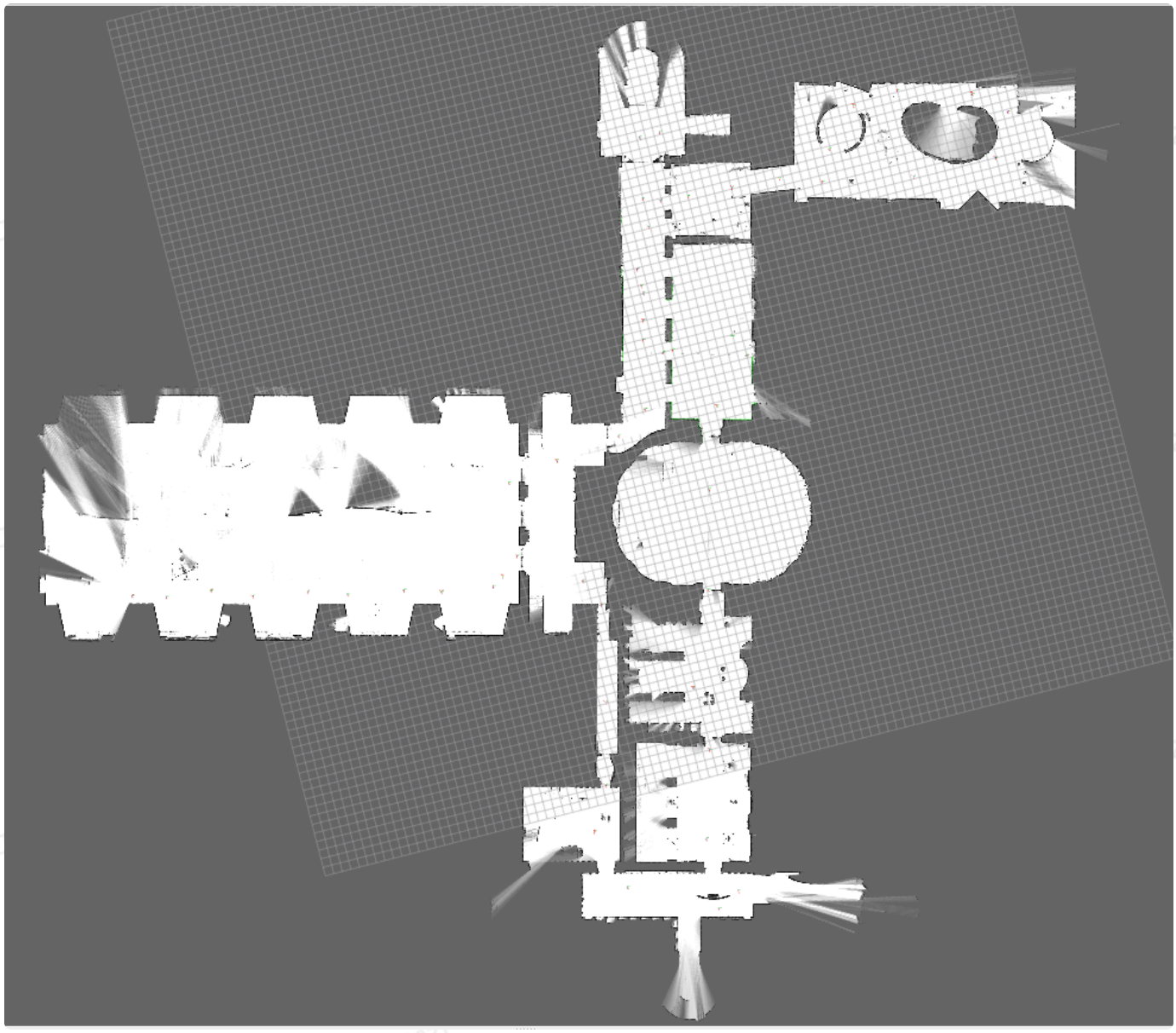
哈工大威海创新平台 - 718—刘渝婉(22971967)

missing_data_ray_length = 1.



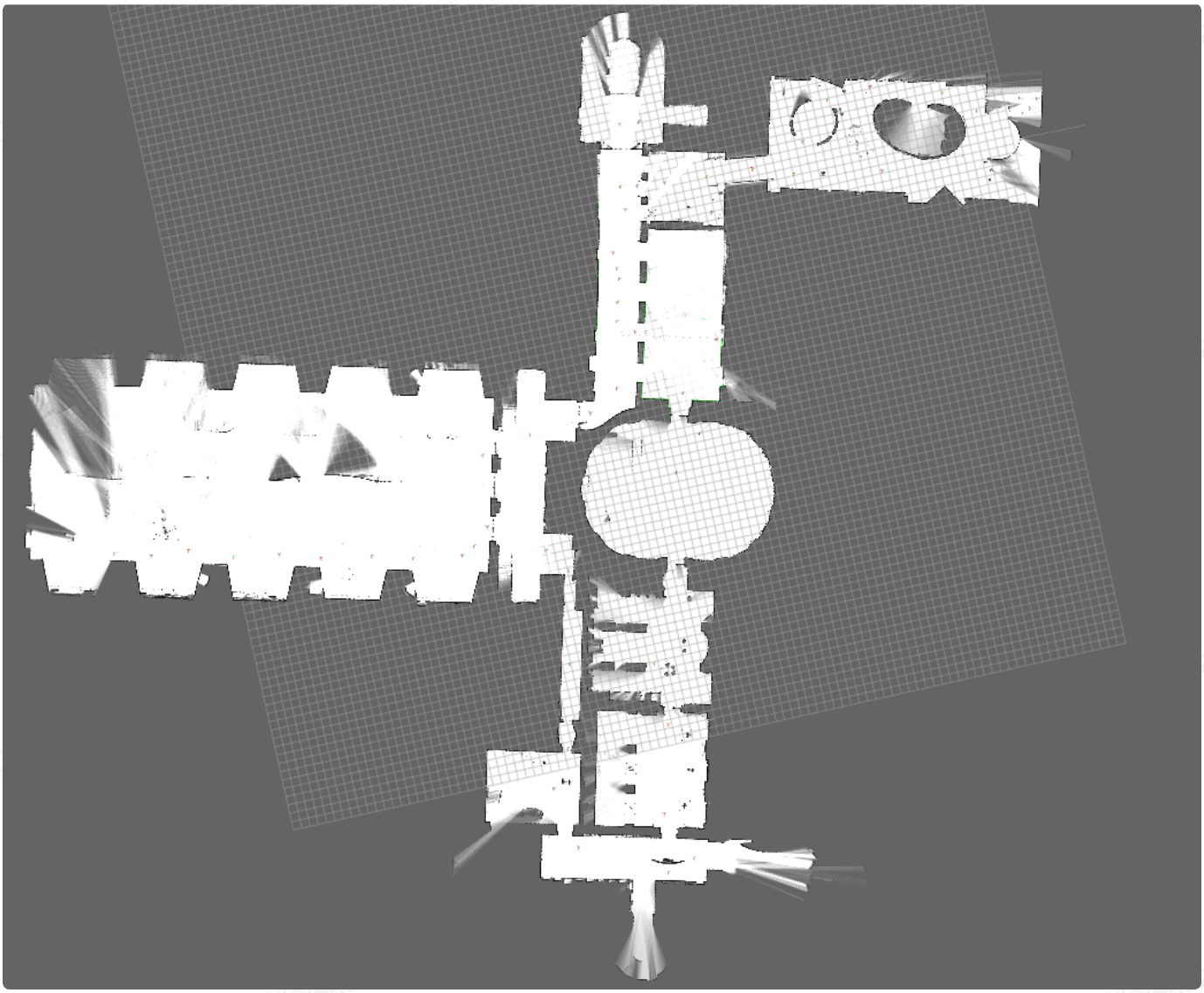
missing_data_ray_length = 3.

哈工大威海创新平台 - 718—刘渝婉(22971967)



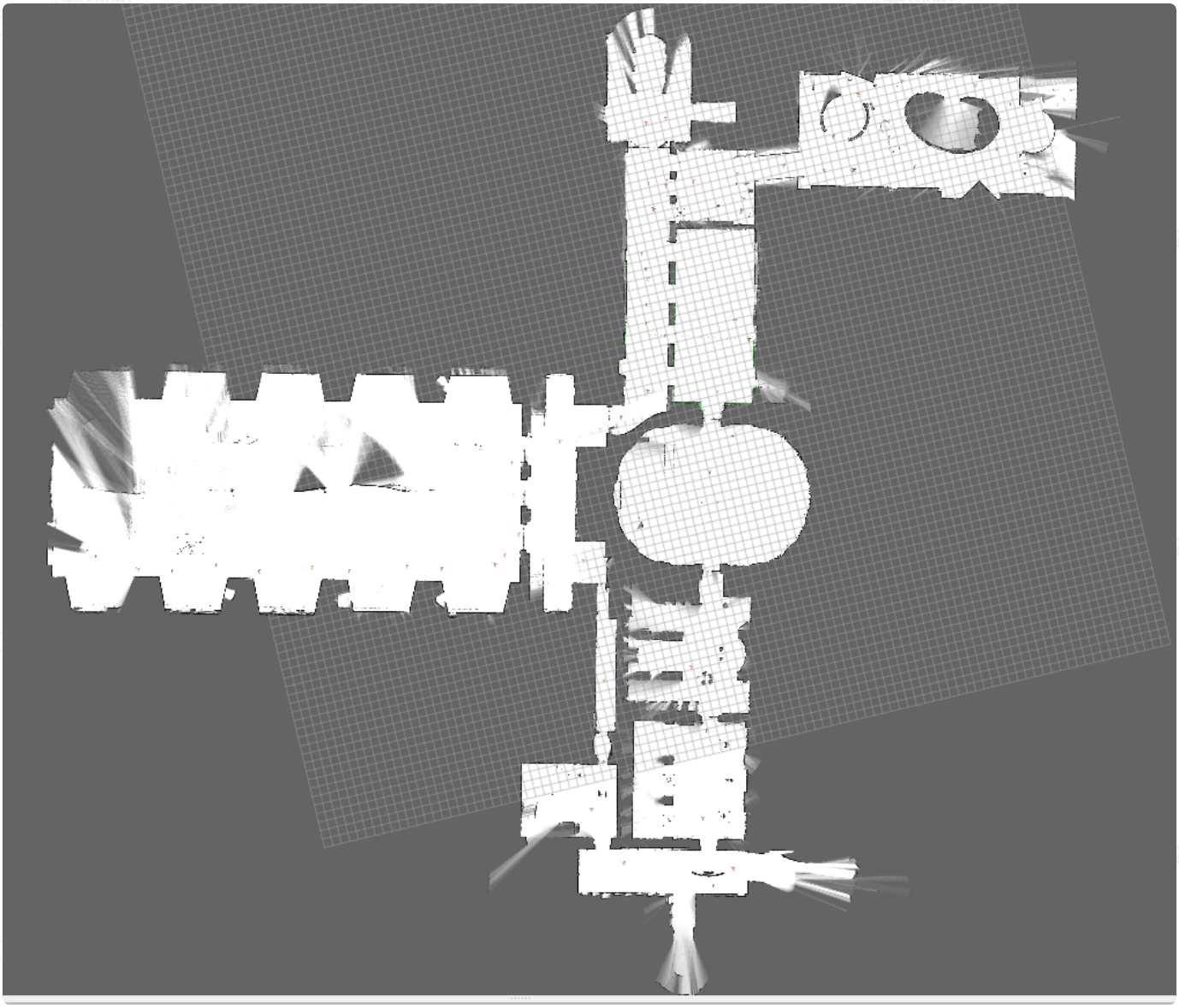
missing_data_ray_length =5.（原始值）

哈工大威海创新平台 - 718—刘渝婉(22971967)



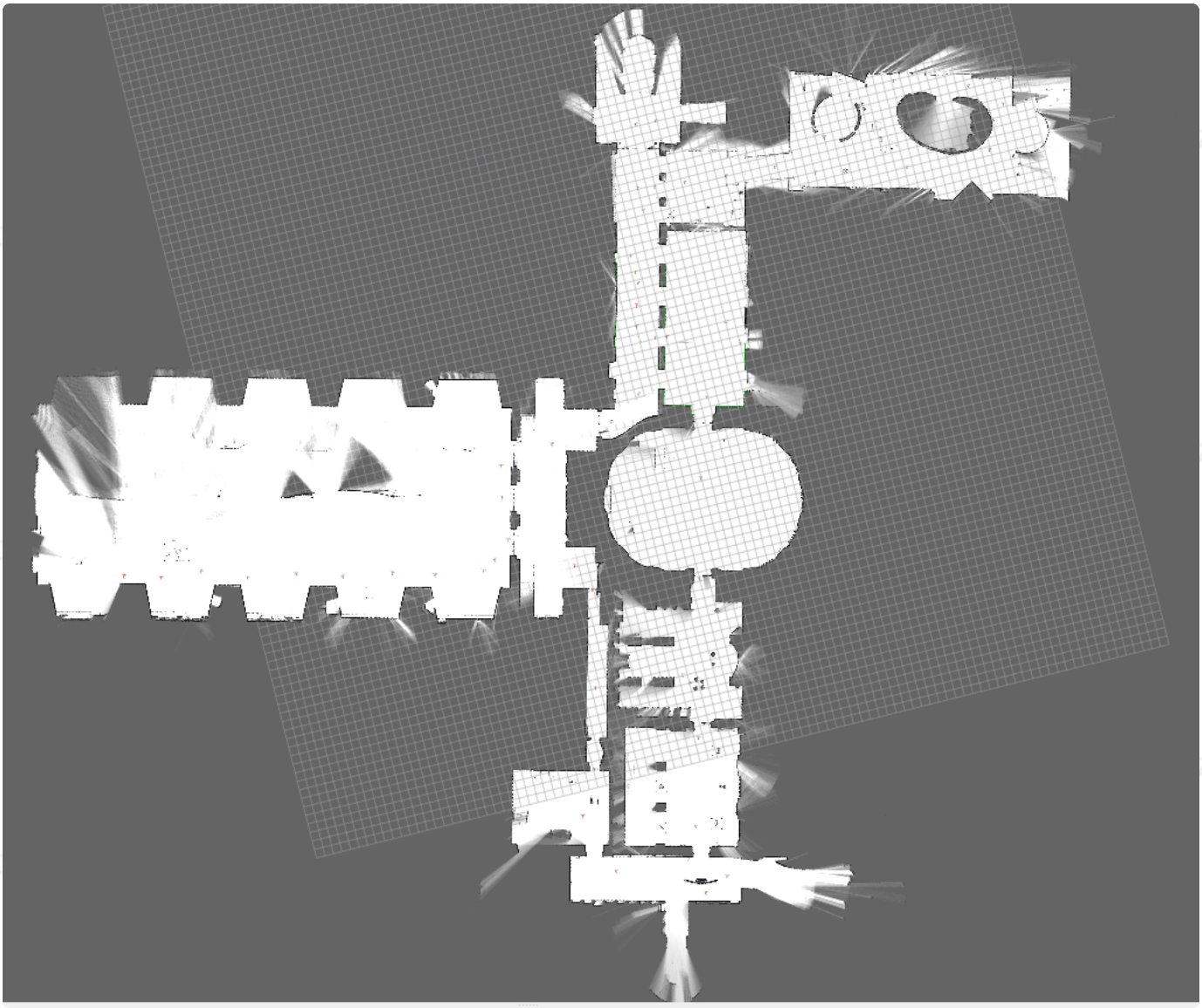
missing_data_ray_length = 7.

哈工大威海创新平台 - 718—刘渝婉(22971967)



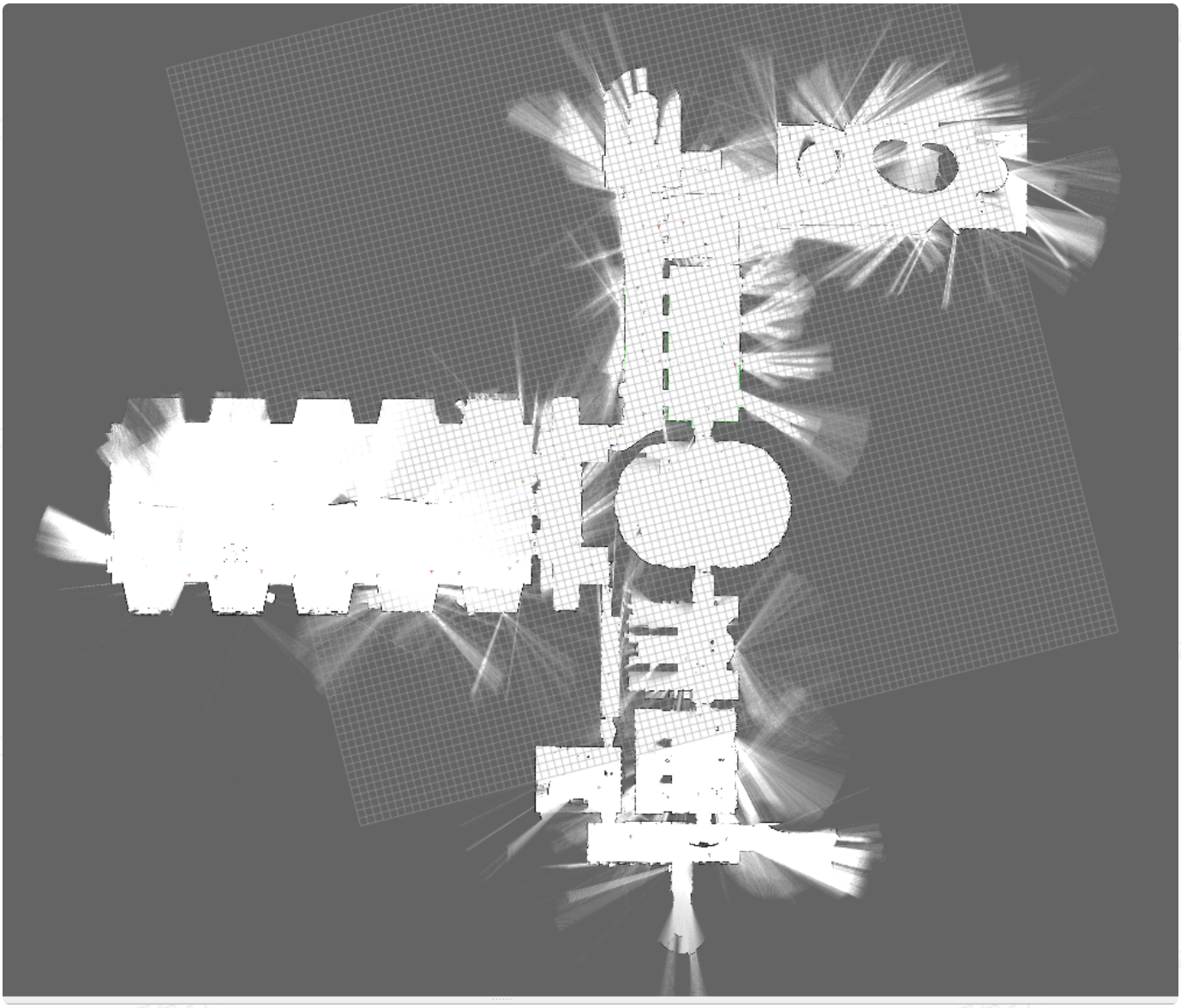
missing_data_ray_length = **10.**

哈工大威海创新平台 - 718—刘渝婉(22971967)



missing_data_ray_length = **20.**

哈工大威海创新平台 - 718—刘渝婉(22971967)



调参结论：

该参数在原代码中的该参数值附近时建图效果没什么变化，过大时出现地图部分区域辐射状的现象

筛选最佳参数值：

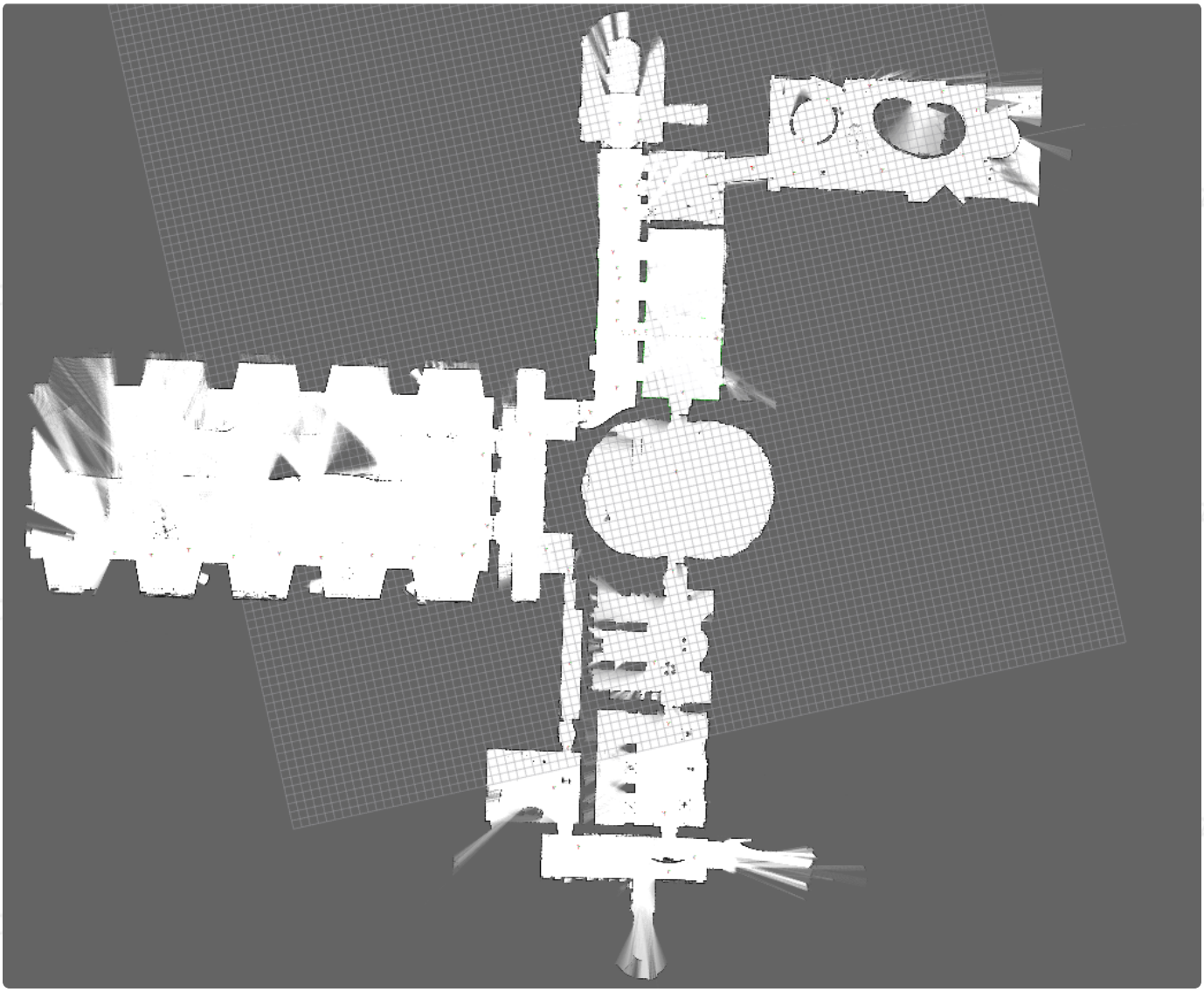
missing_data_ray_length =5.（原始值）

7.1.6voxel_filter_size：

参数介绍：

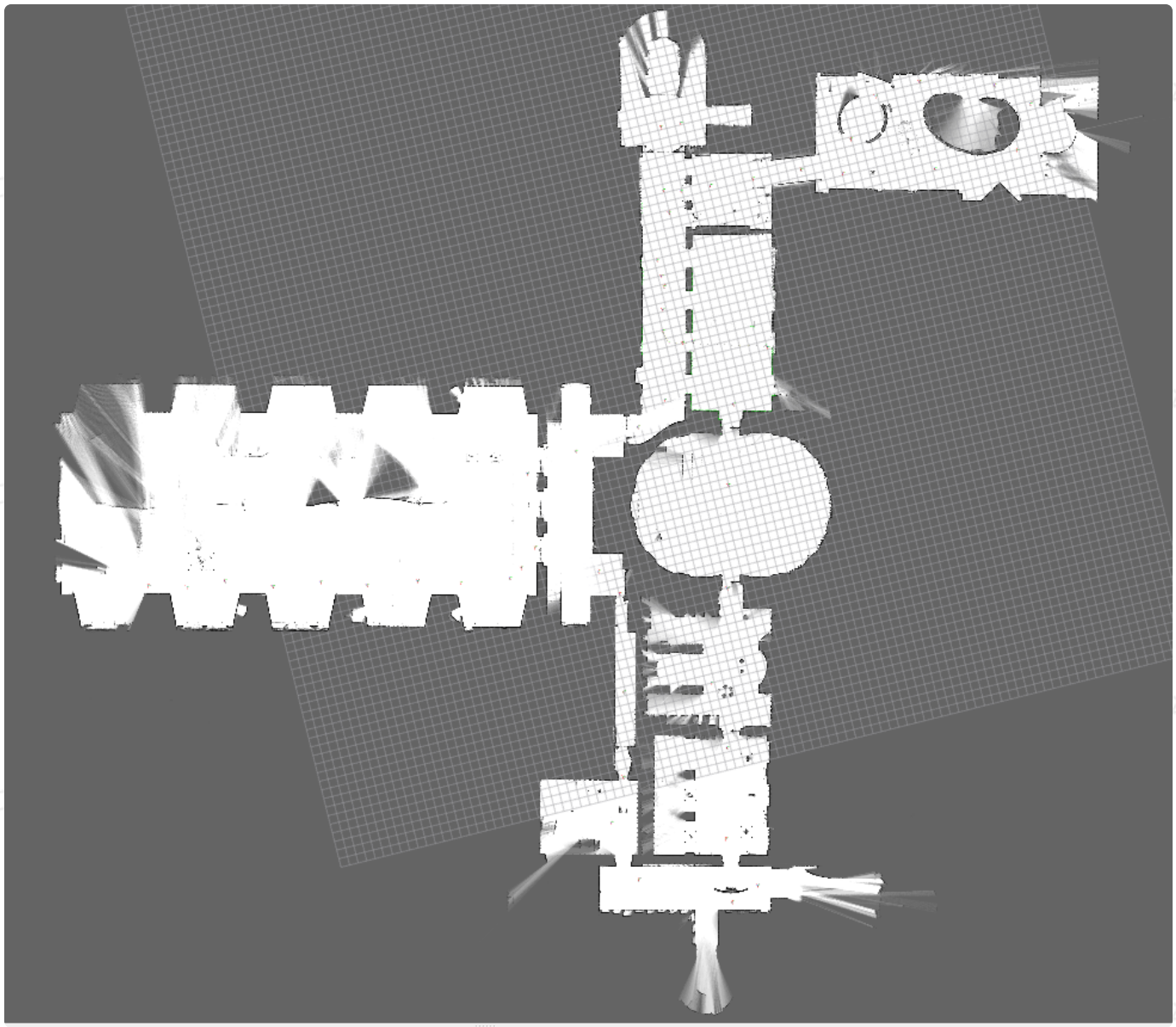
- 在trajectory_builder_2d.lua中
- 由于激光雷达的基于原点旋转扫描采样，距离近的障碍物采集数据密集， 距离远的障碍物采集点稀疏，为保证远近障碍物权重一致，需做处理。采用体素滤波器进行降采样机制，使得局部稠密的点变得稀疏。此值越小，得到的数据越稠密，但是计算量越高；此值越大，丢失数据越多，但计算更快。

voxel_filter_size = **0.025（原始值）**



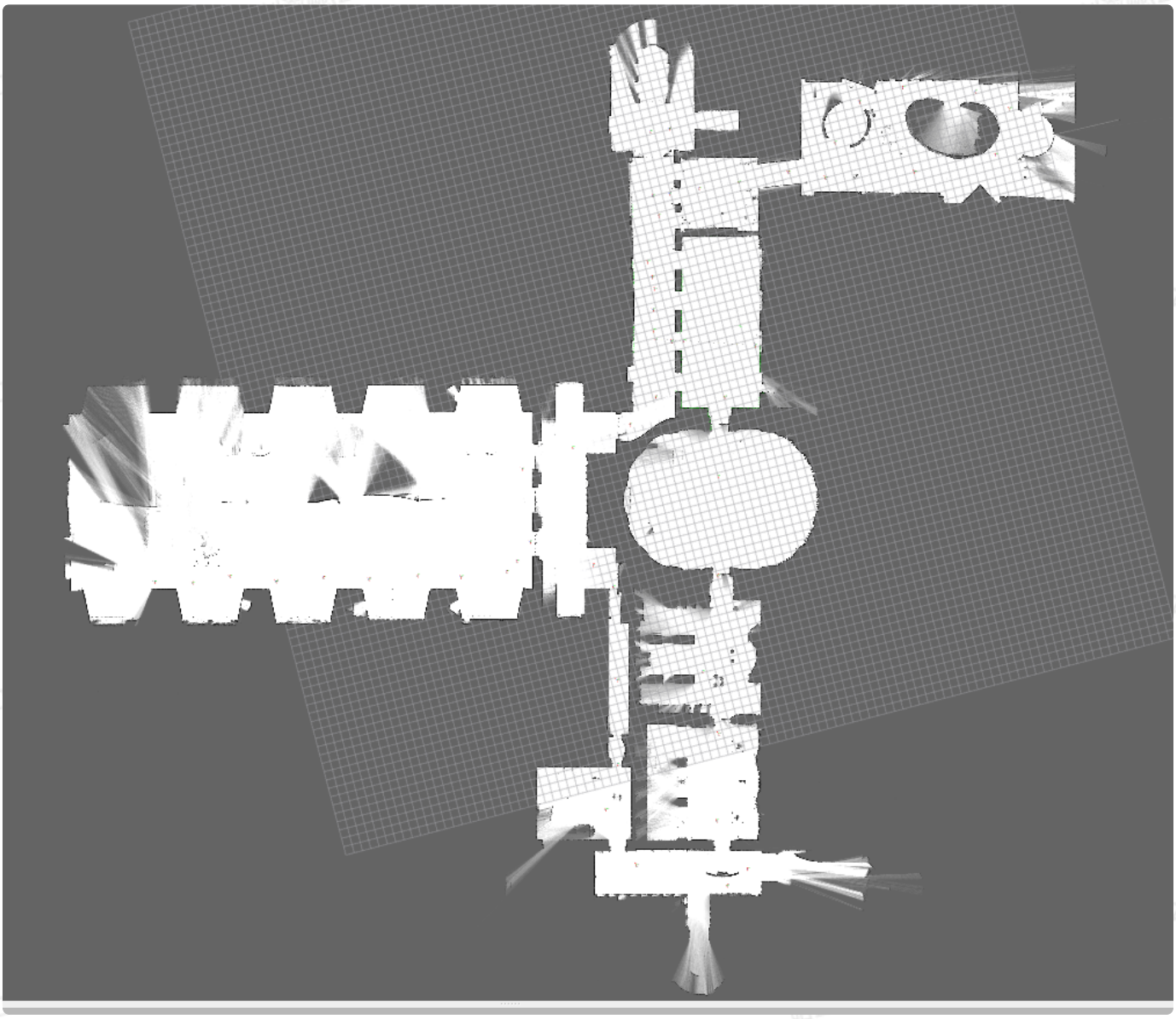
哈工大威海创新平台 - 718—刘渝婉(22971967)

voxel_filter_size = **0.05**



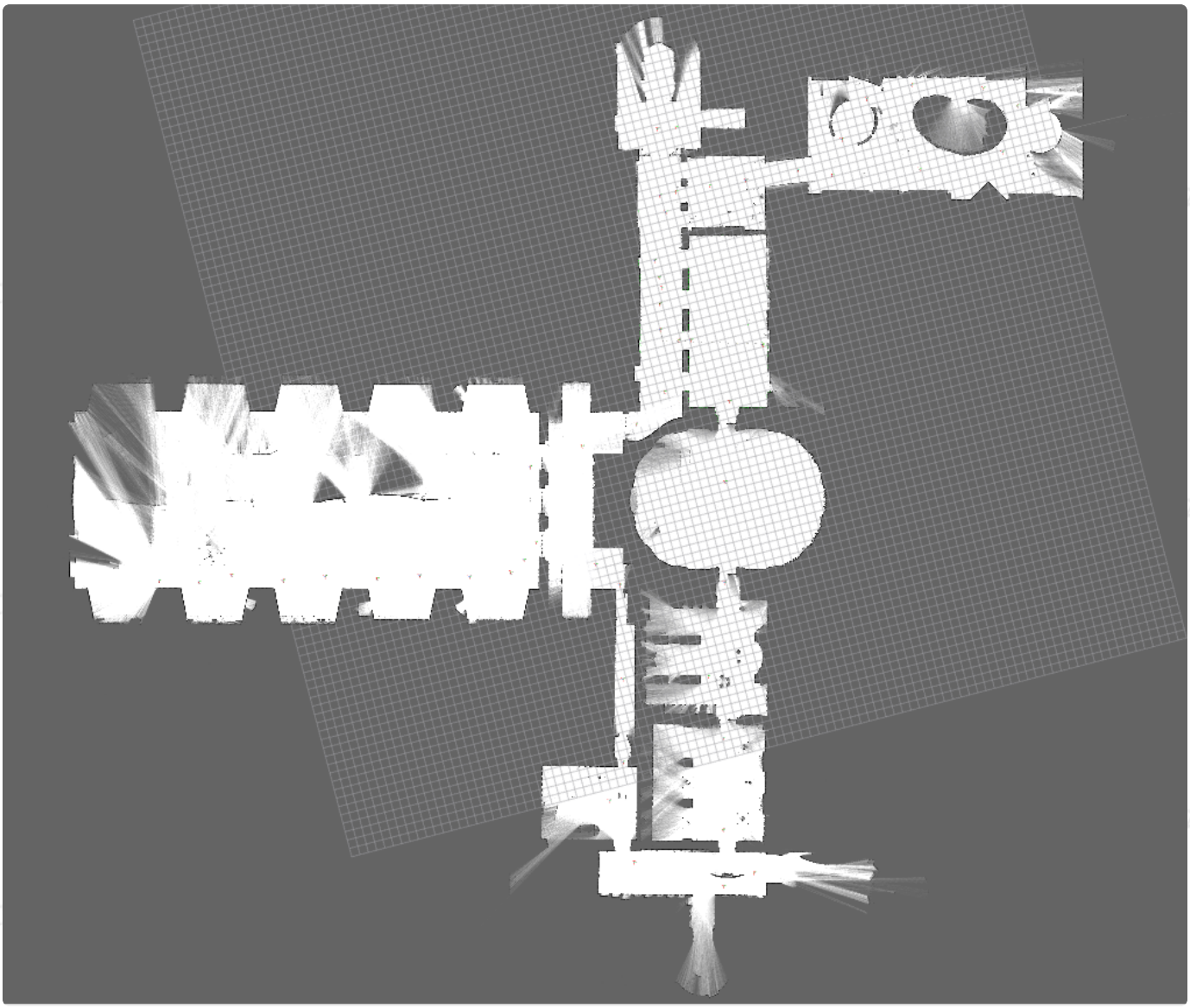
哈工大威海创新平台 - 718—刘渝婉(22971967)

voxel_filter_size = 0.1



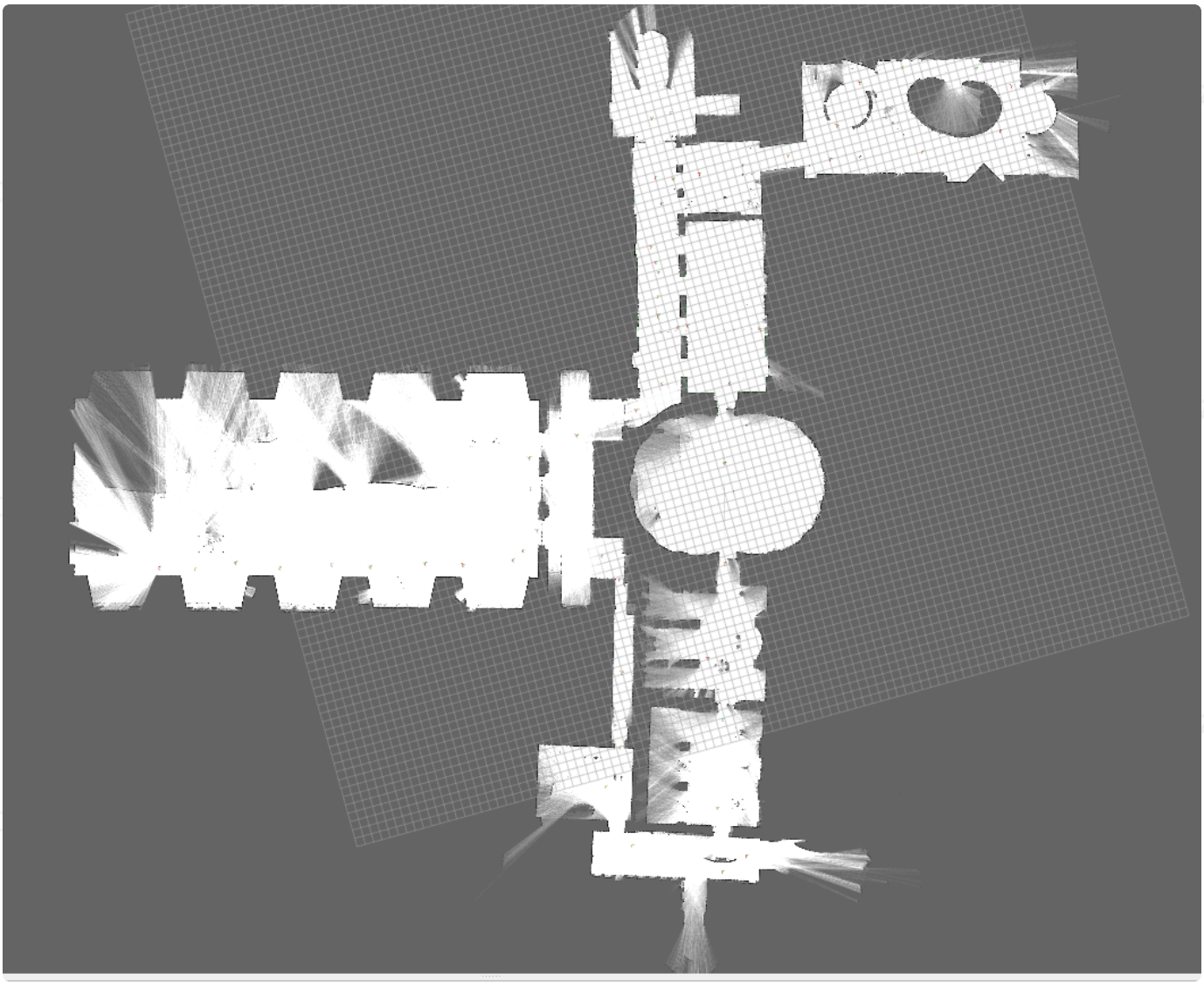
哈工大威海创新平台 - 718—刘渝婉(22971967)

voxel_filter_size = **0.2**



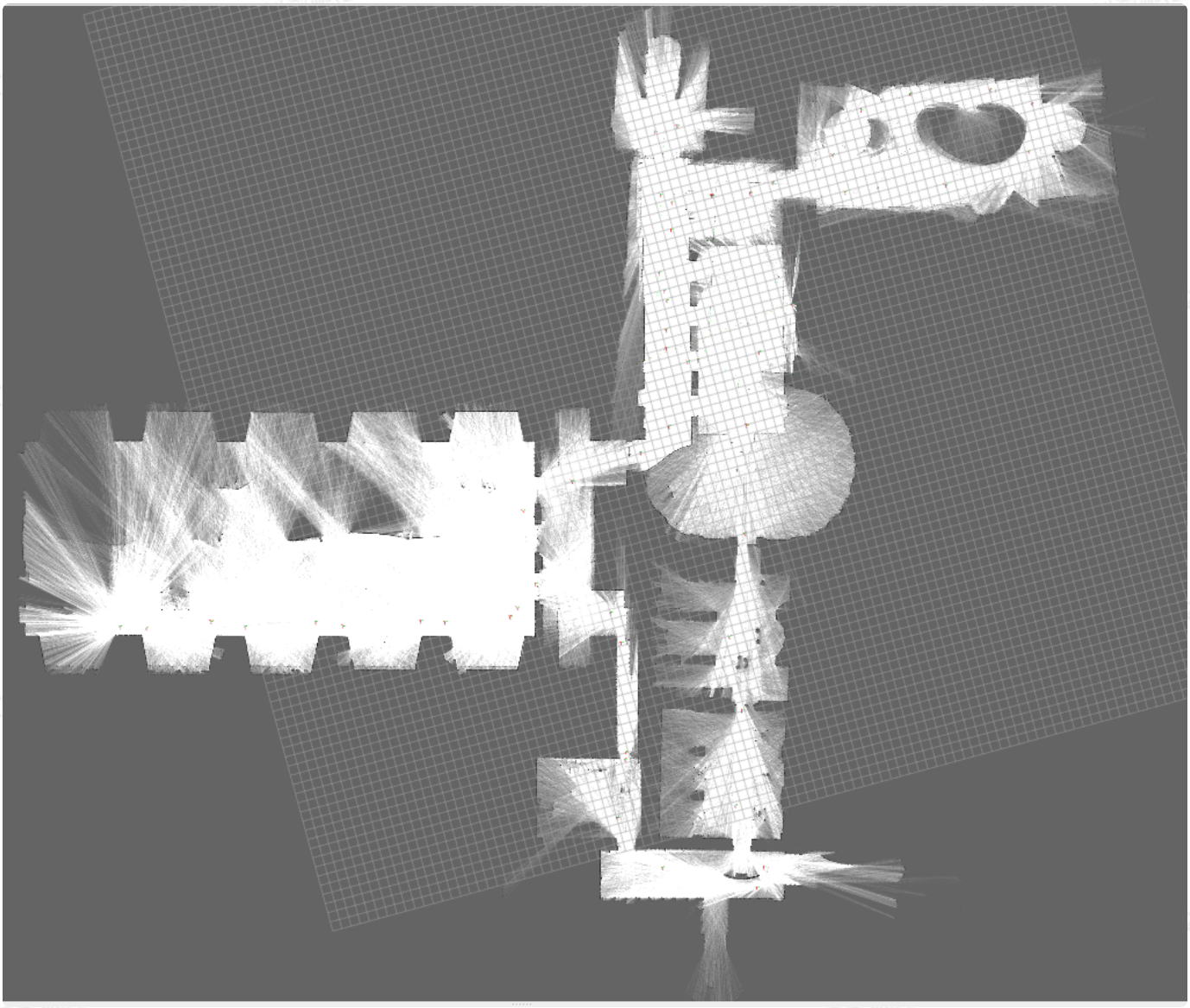
哈工大威海创新平台 - 718—刘渝婉(22971967)

voxel_filter_size = **0.3**



哈工大威海创新平台 - 718—刘渝婉(22971967)

voxel_filter_size = 0.9



调参效果：

该参数过大会出现地图虚化漂移的情况，在保证建图效果基本不变的情况下，适当提高了此参数的值，相对提高了计算速度。

筛选最佳参数值：

voxel_filter_size = 0.1

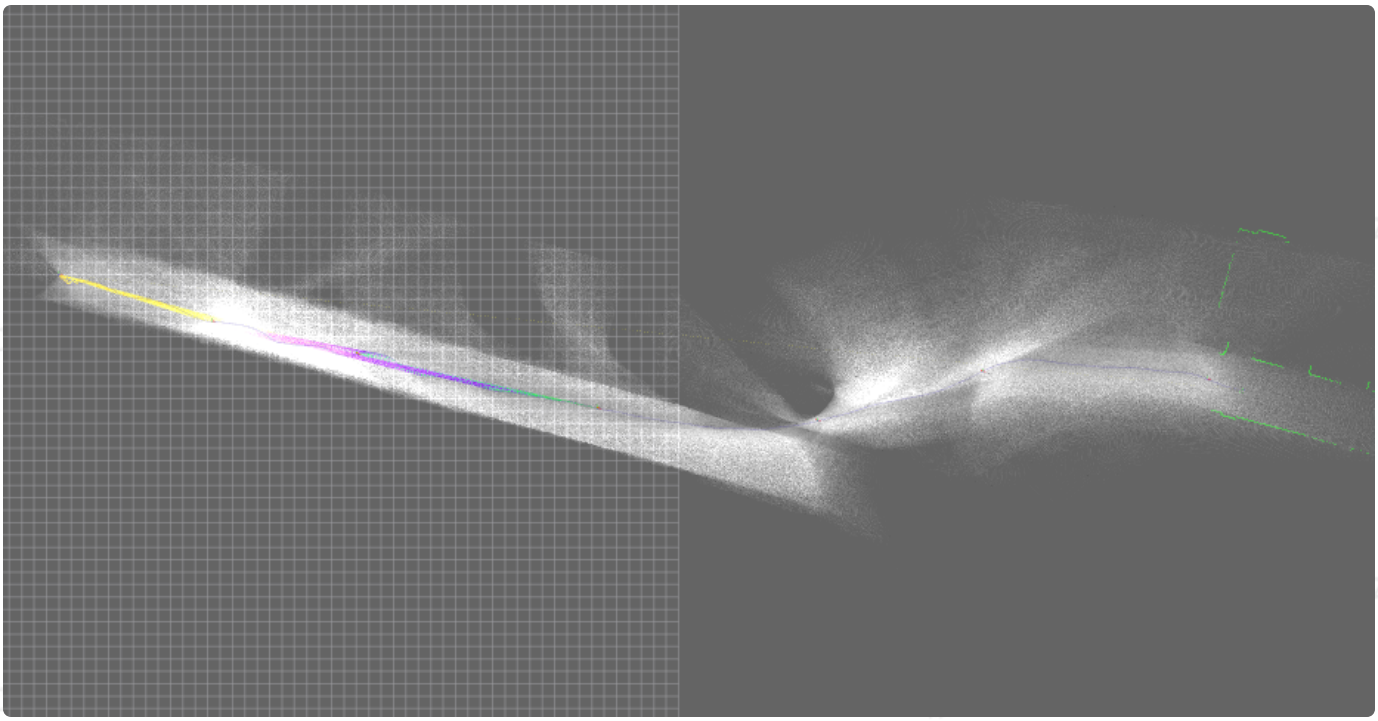
7.1.7resolution：

参数介绍：

- 在trajectory_builder_2d.lua中
- 该参数表示地图分辨率配置：2D地图仅有一个分辨率配置项

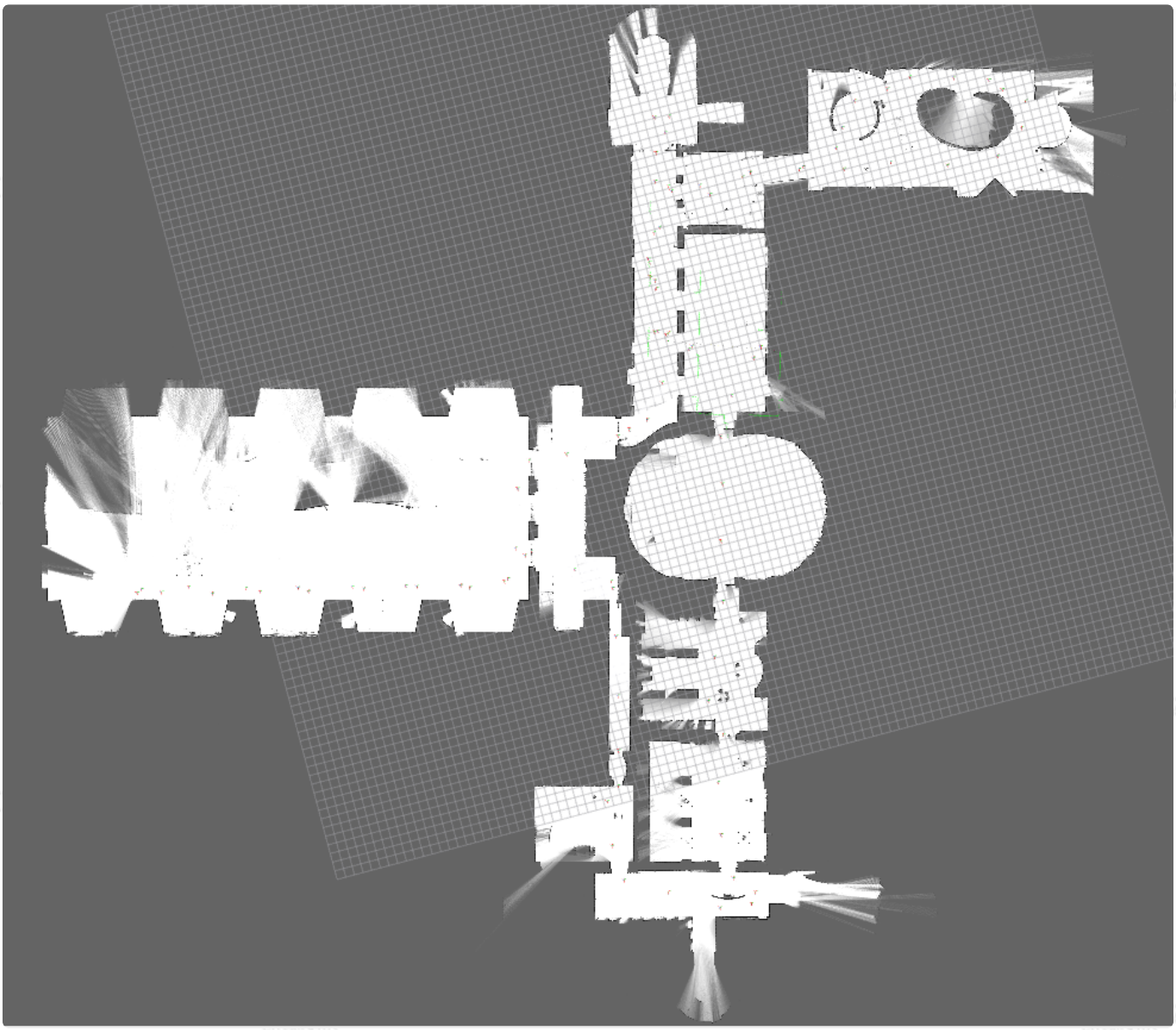
resolution = 0.005

说明：由于建图时间较长，故该图未建完，但是效果比较明显，分辨率过小，建的图非常模糊



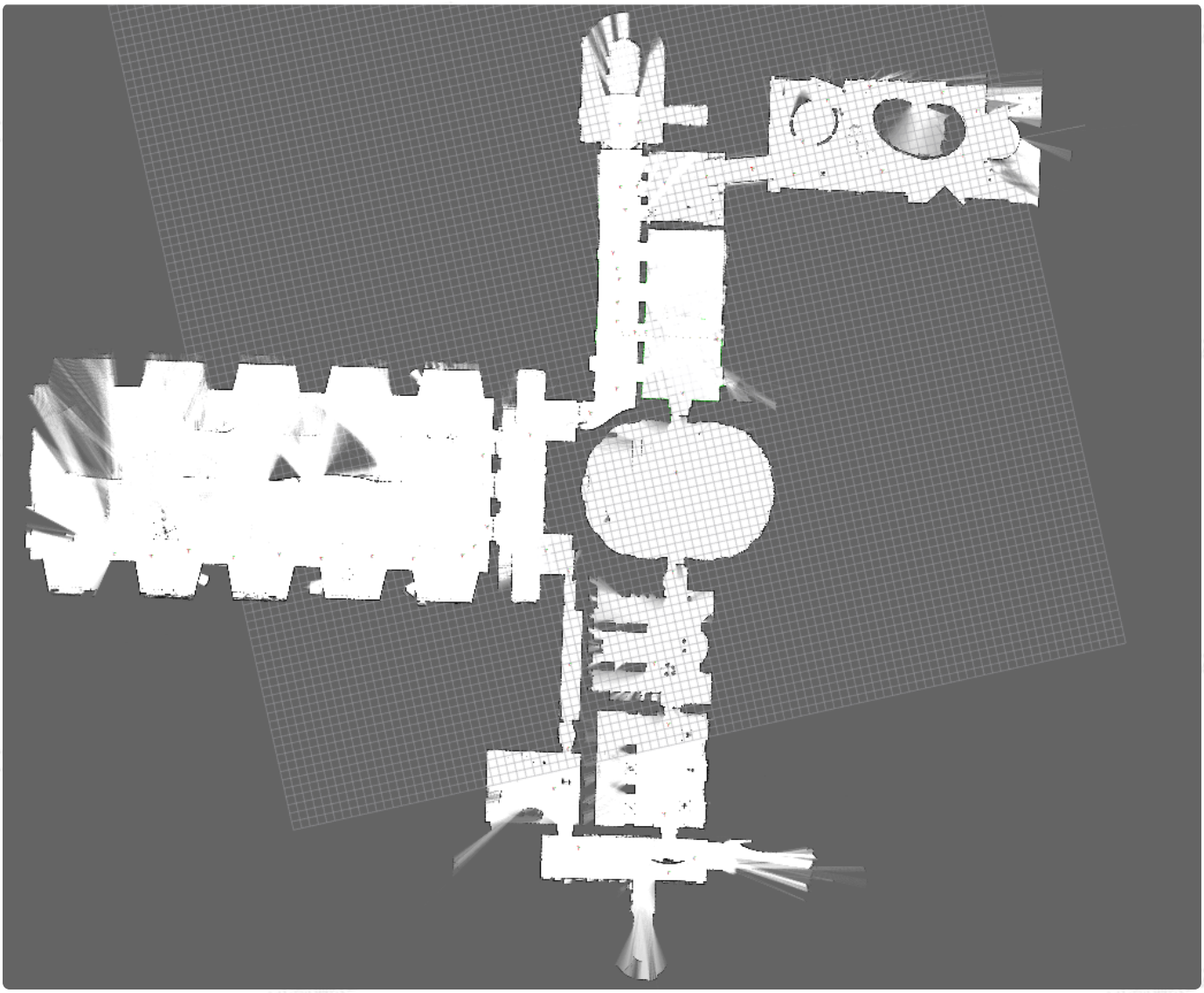
哈工大威海创新平台 - 718—刘渝婉(22971967)

resolution = **0.03**



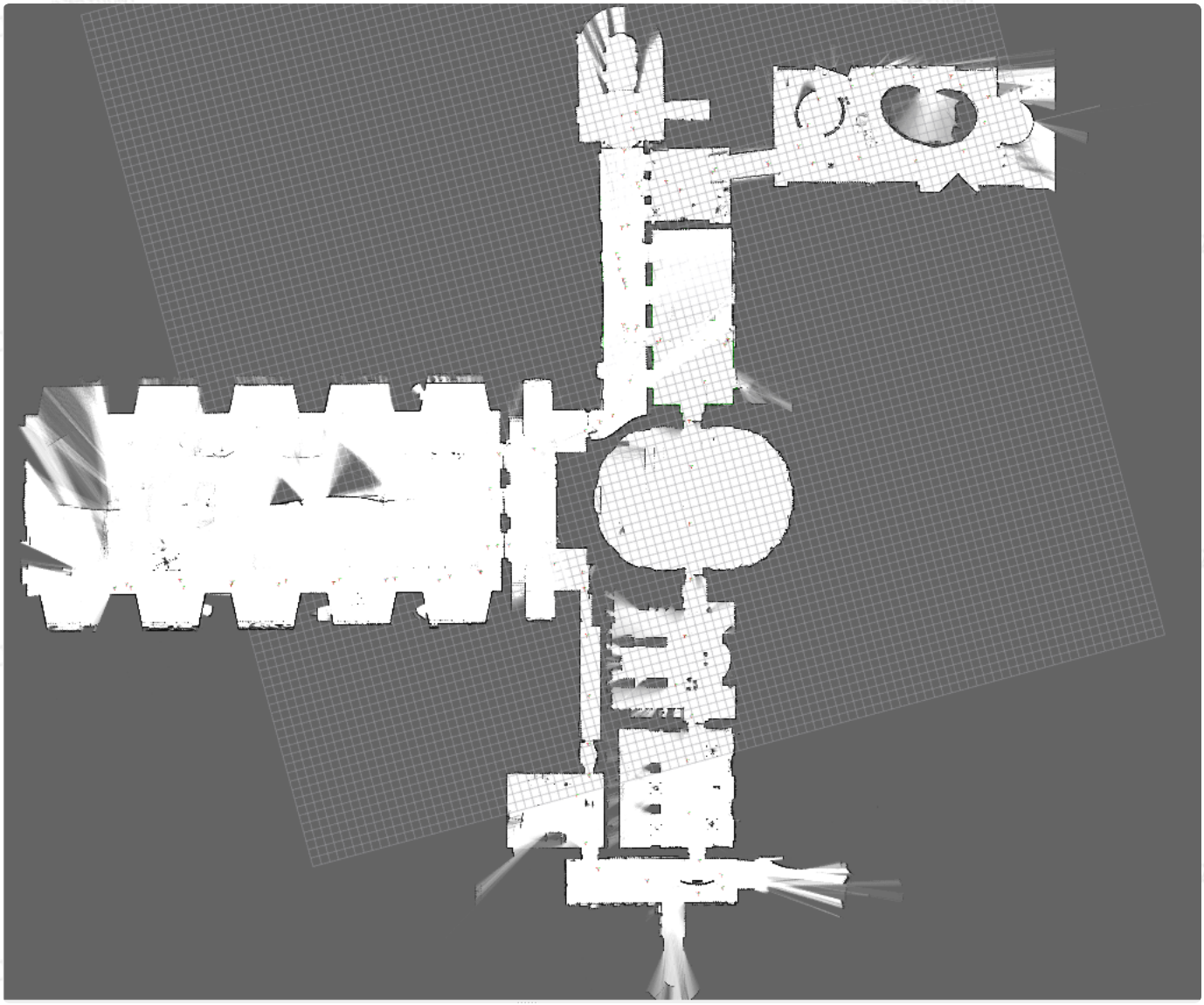
resolution = **0.05（原始值）**

哈工大威海创新平台 - 718—刘渝婉(22971967)



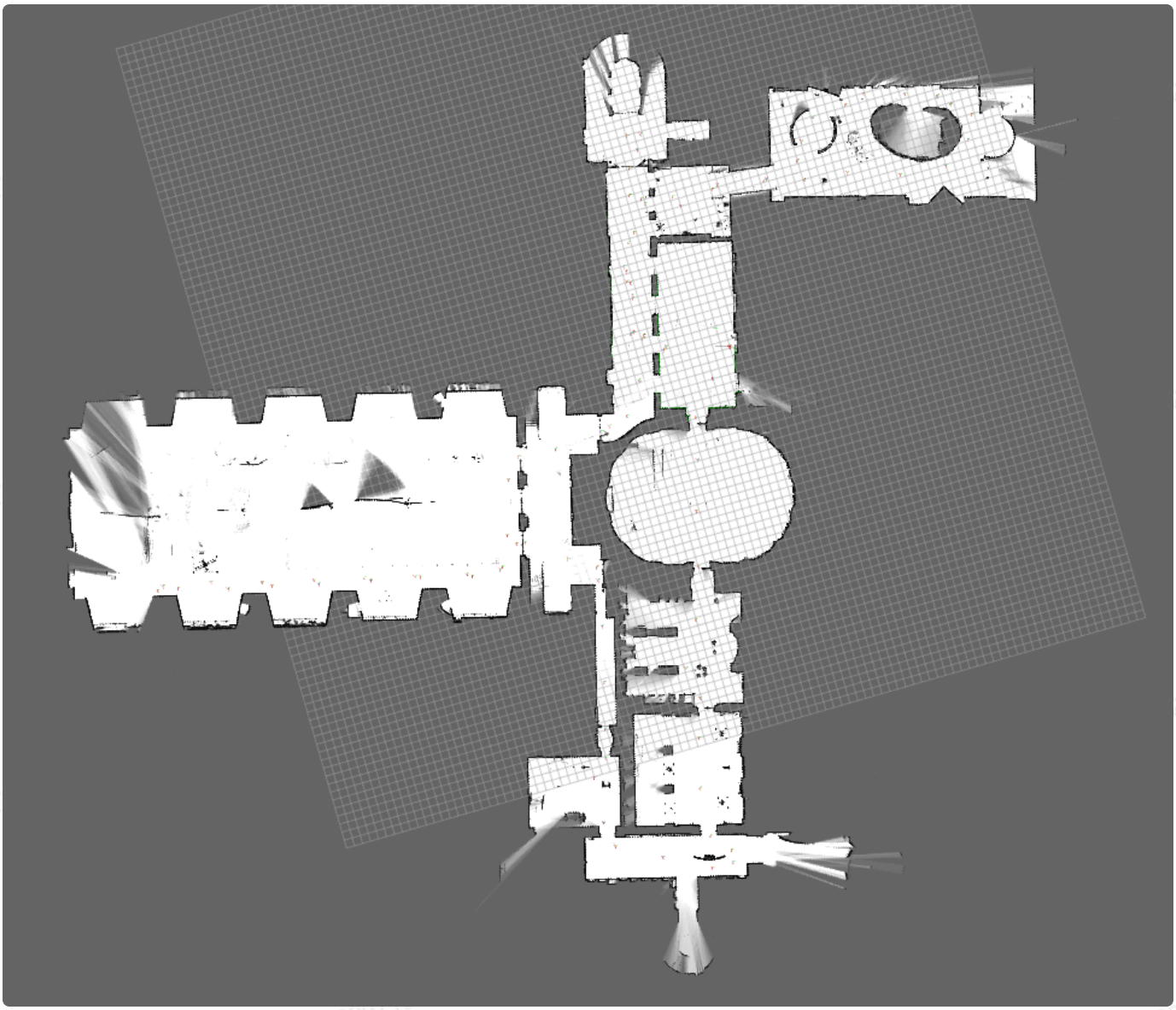
resolution = **0.07**

哈工大威海创新平台 - 718—刘渝婉(22971967)



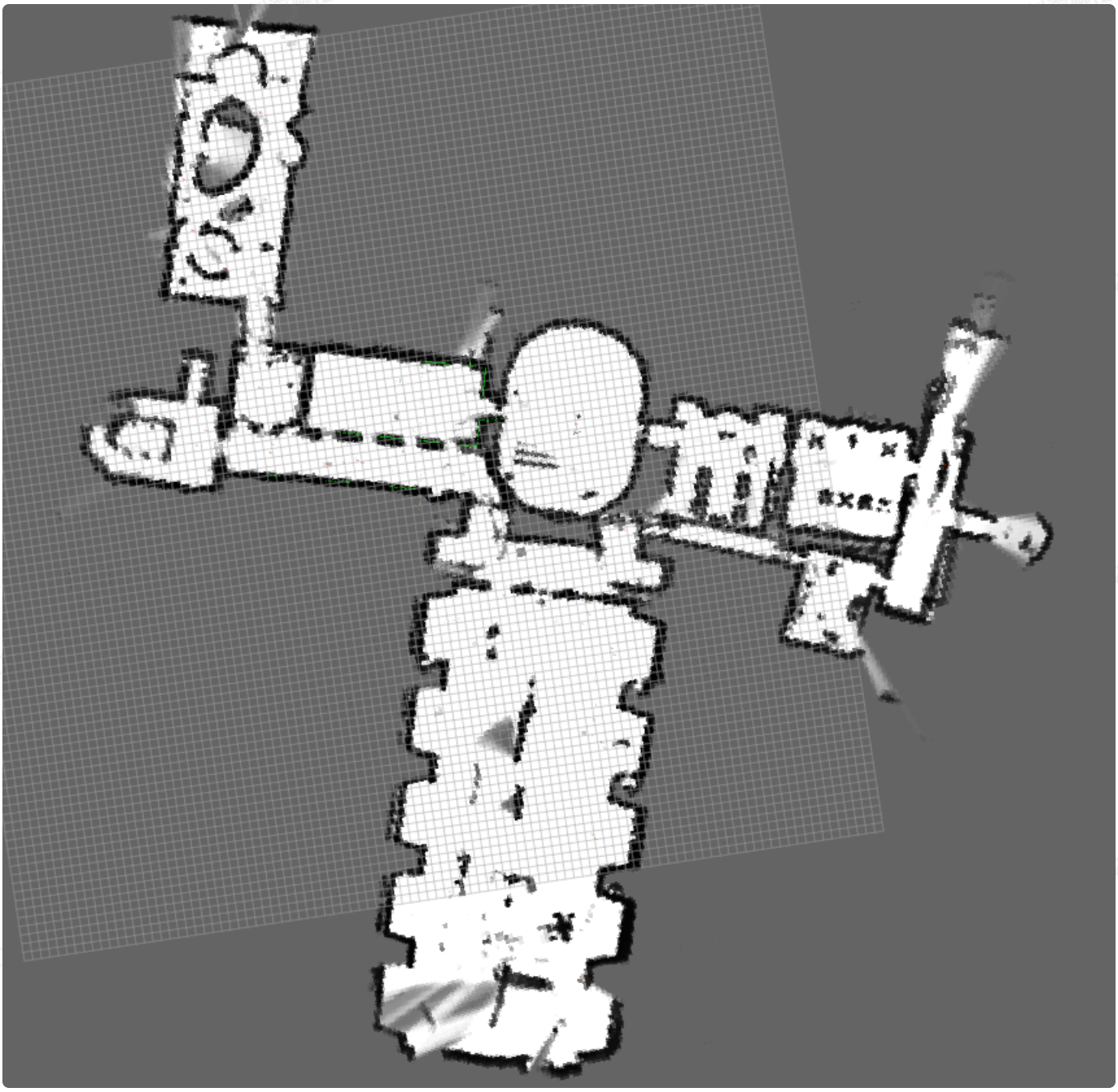
哈工大威海创新平台 - 718—刘渝婉(22971967)

resolution = 0.10



哈工大威海创新平台 - 718—刘渝婉(22971967)

resolution = 0.5



调参效果：

分辨率越小建图时间越长，越模糊；分辨率越大，建图时间越短，轮廓越清晰，效果有点像膨胀操作 。从上述尝试的参数中选择了一个地图轮廓比较清晰、未把噪点显示的过于明显的作为了调参结果。

筛选最佳参数值：

resolution = 0.10

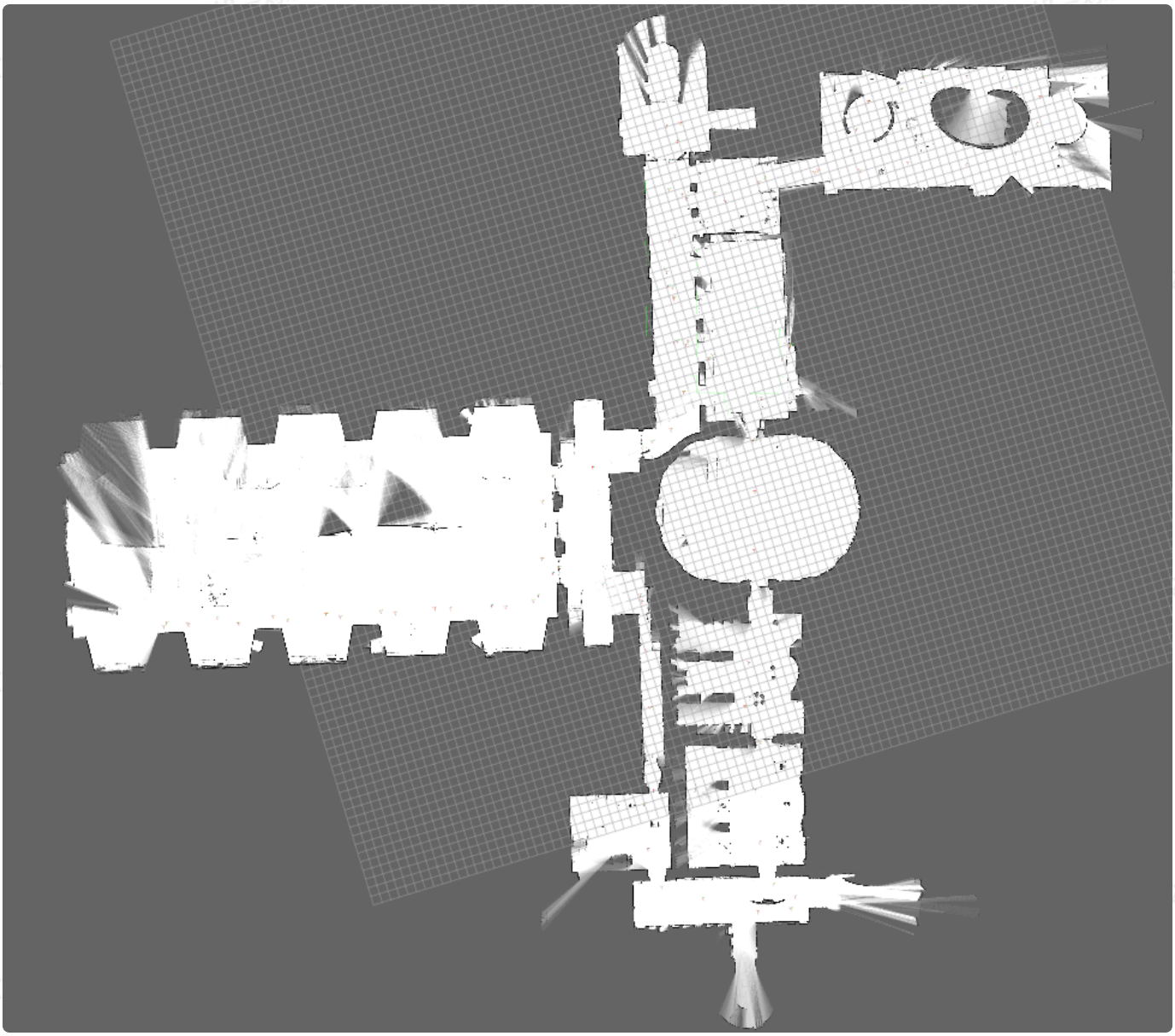
（8）（9）两项均是在后端优化关闭的情况下进行的

7.1.8TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight：

参数介绍：

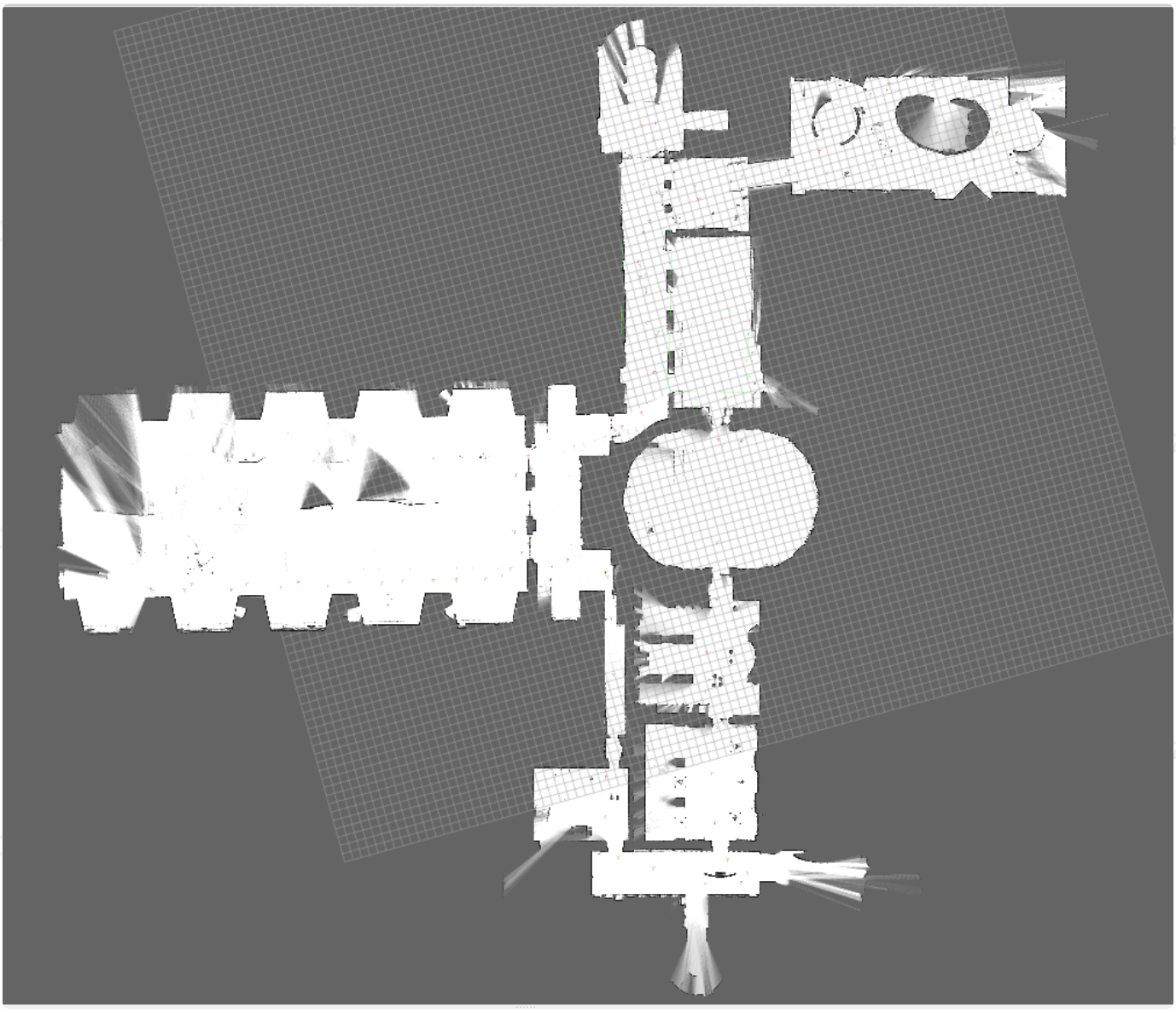
- 在trajectory_builder_2d.lua中
- 该参数值越高，从之前的结果中移除相关匹配就需要越高的代价，即扫描匹配器在另一个位置必须生成一个更高的匹配值才能被接受

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =1



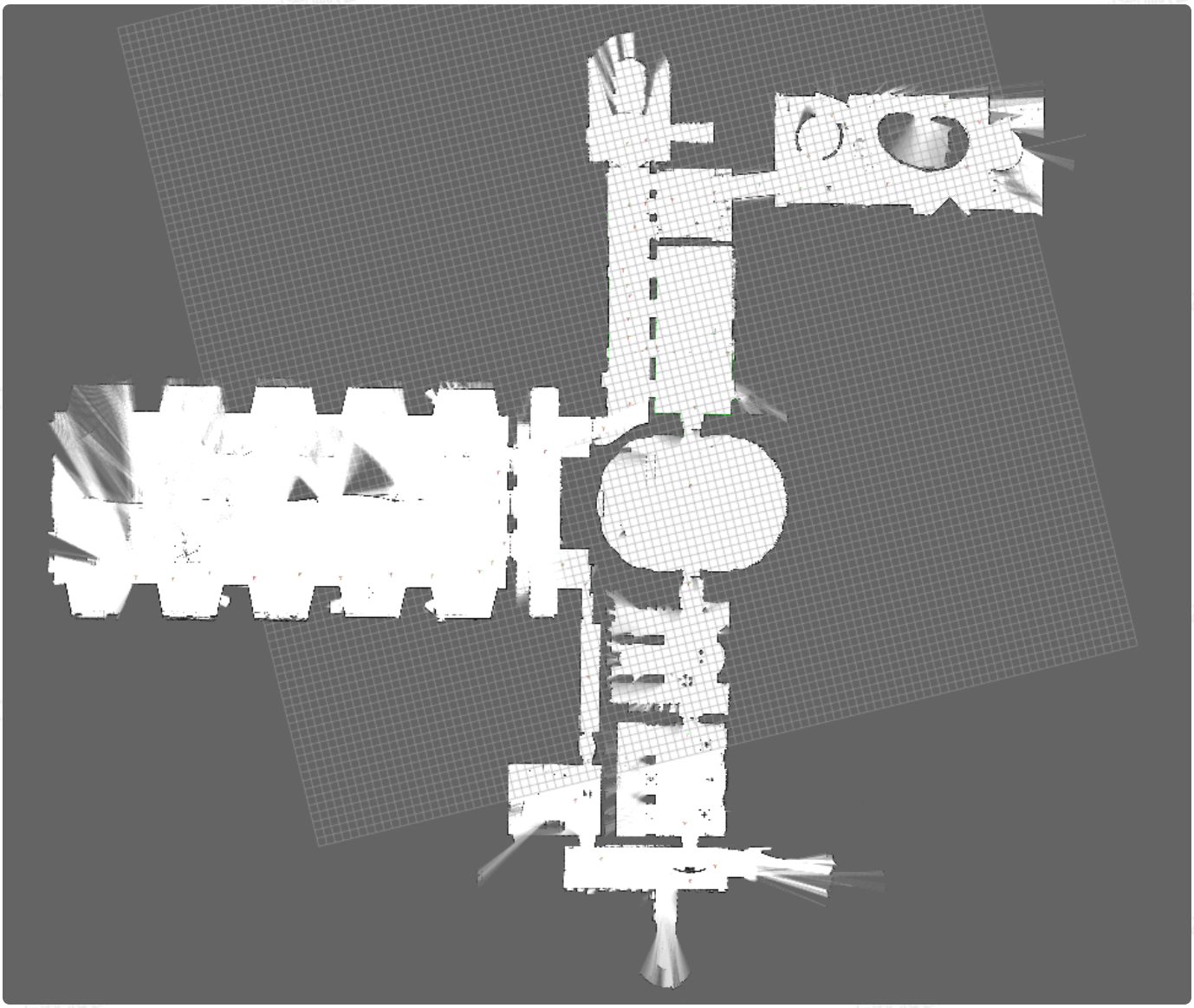
TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =5

哈工大威海创新平台 - 718—刘渝婉(22971967)



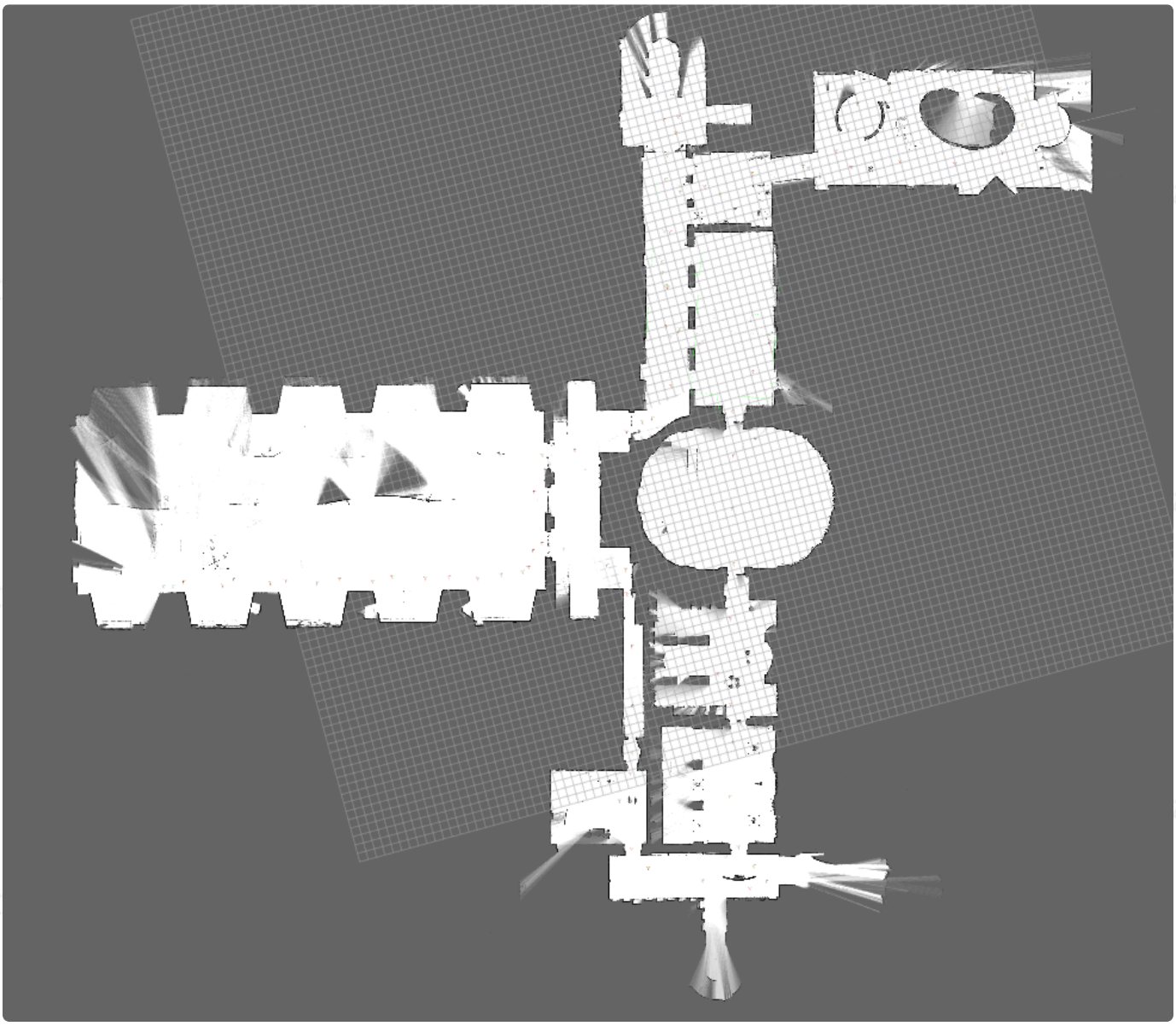
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =10（原始值）



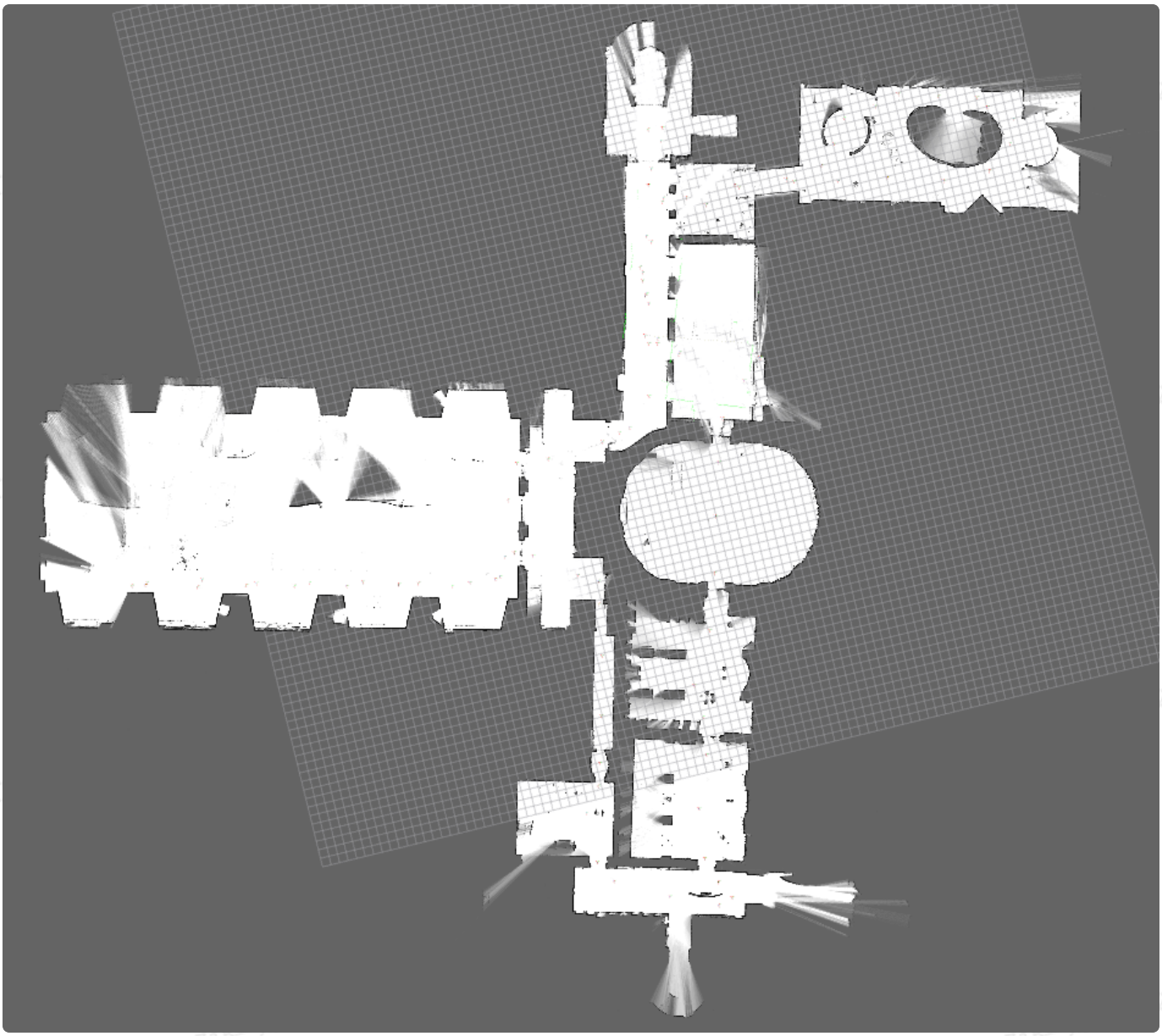
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =15



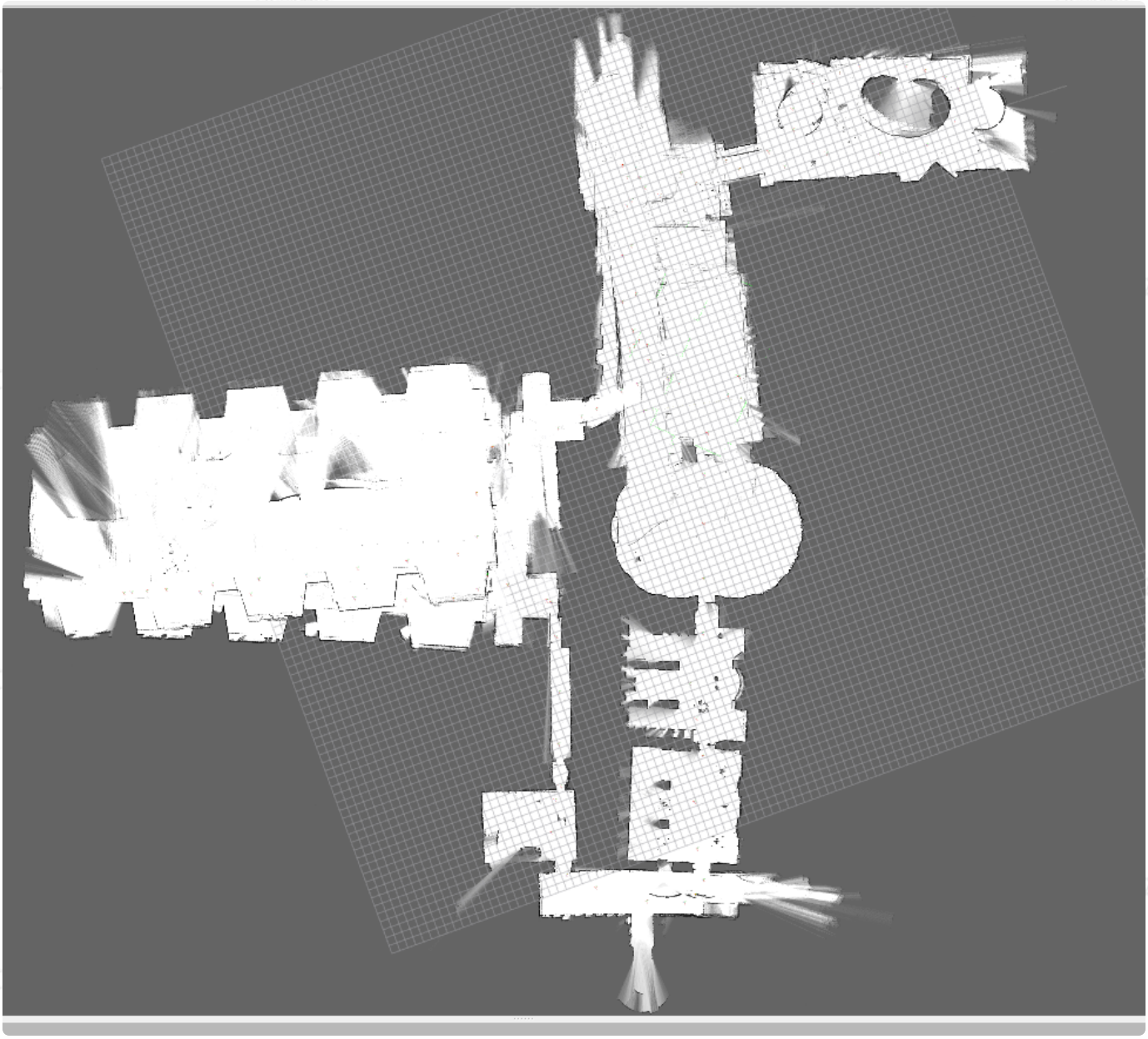
哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =**20**



哈工大威海创新平台 - 718—刘渝婉(22971967)

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =50



调参结论：

较小的值没什么太大的区别，但值过大会导致图漂移

筛选最佳参数值：

TRAJECTORY_BUILDER_2D.ceres_scan_matcher.translation_weight =10（原始值）

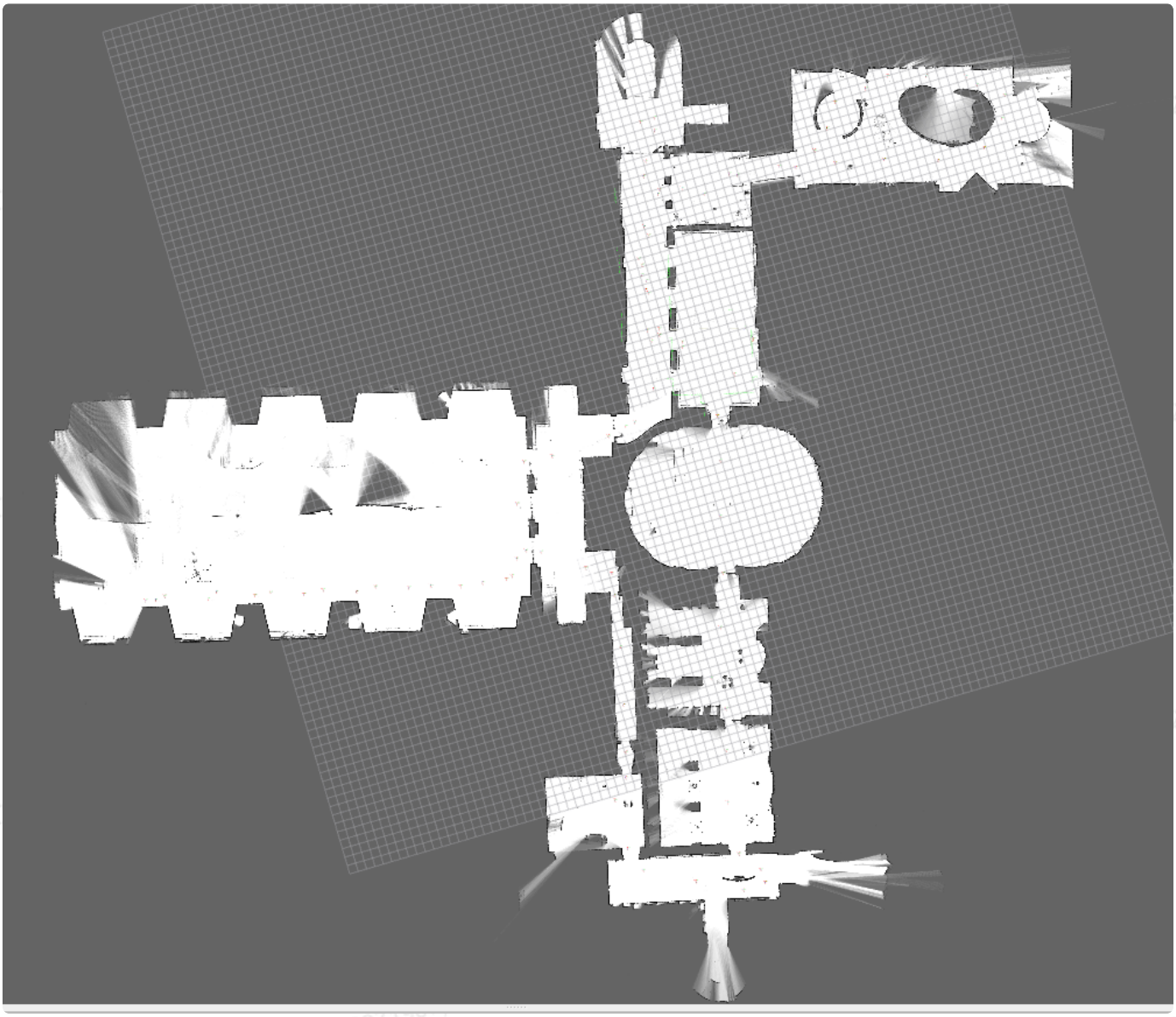
7.1.9TRAJECTORY_BUILDER_2D.ceres_scan_matcher.rotation_weight：

参数介绍：

- 在trajectory_builder_2d.lua中
- 和上面的参数一样，该参数值越高，从之前的结果中移除相关匹配就需要越高的代价，即扫描匹配

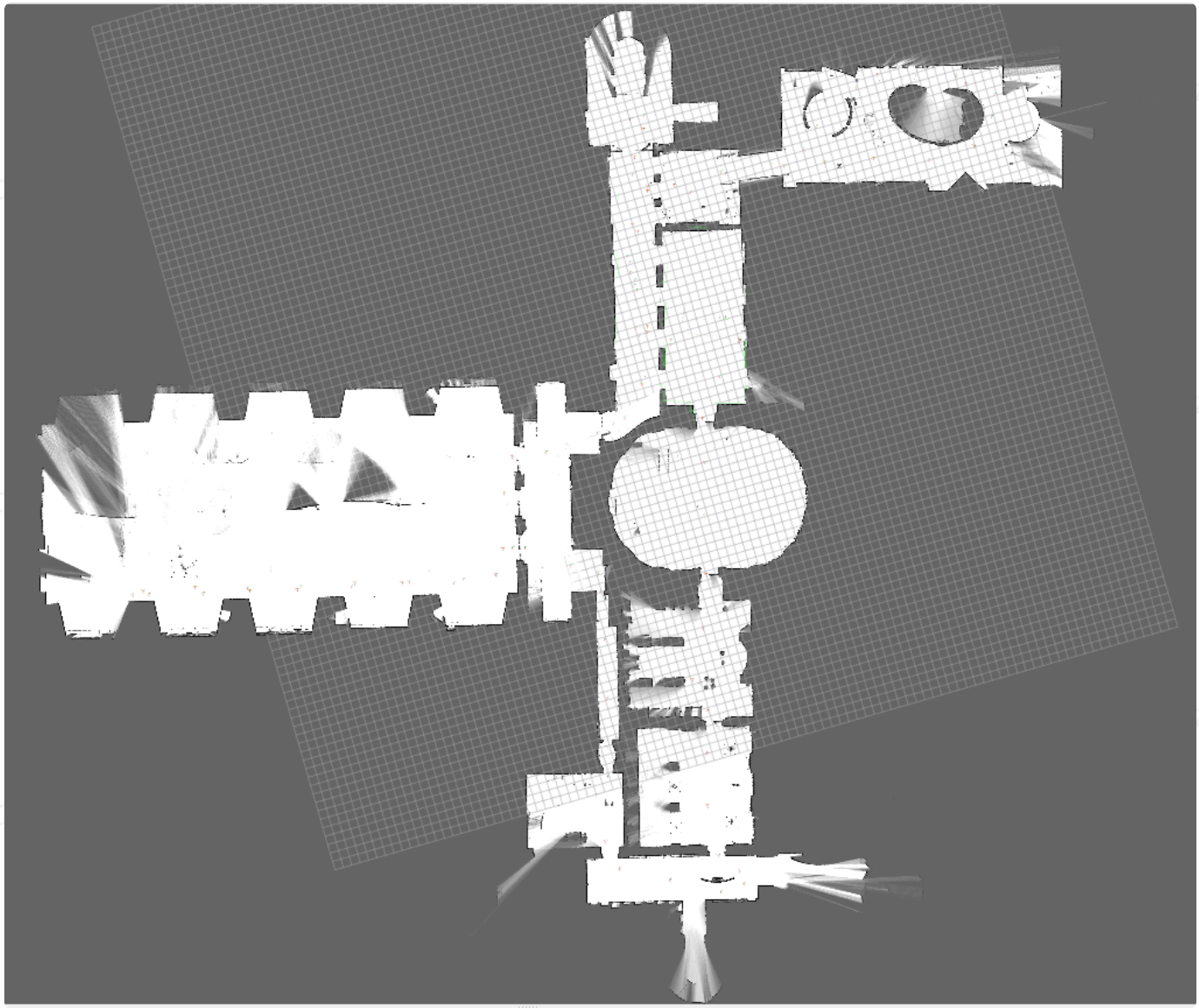
器在另一个位置必须生成一个更高的匹配值才能被接受

rotation_weight = 1



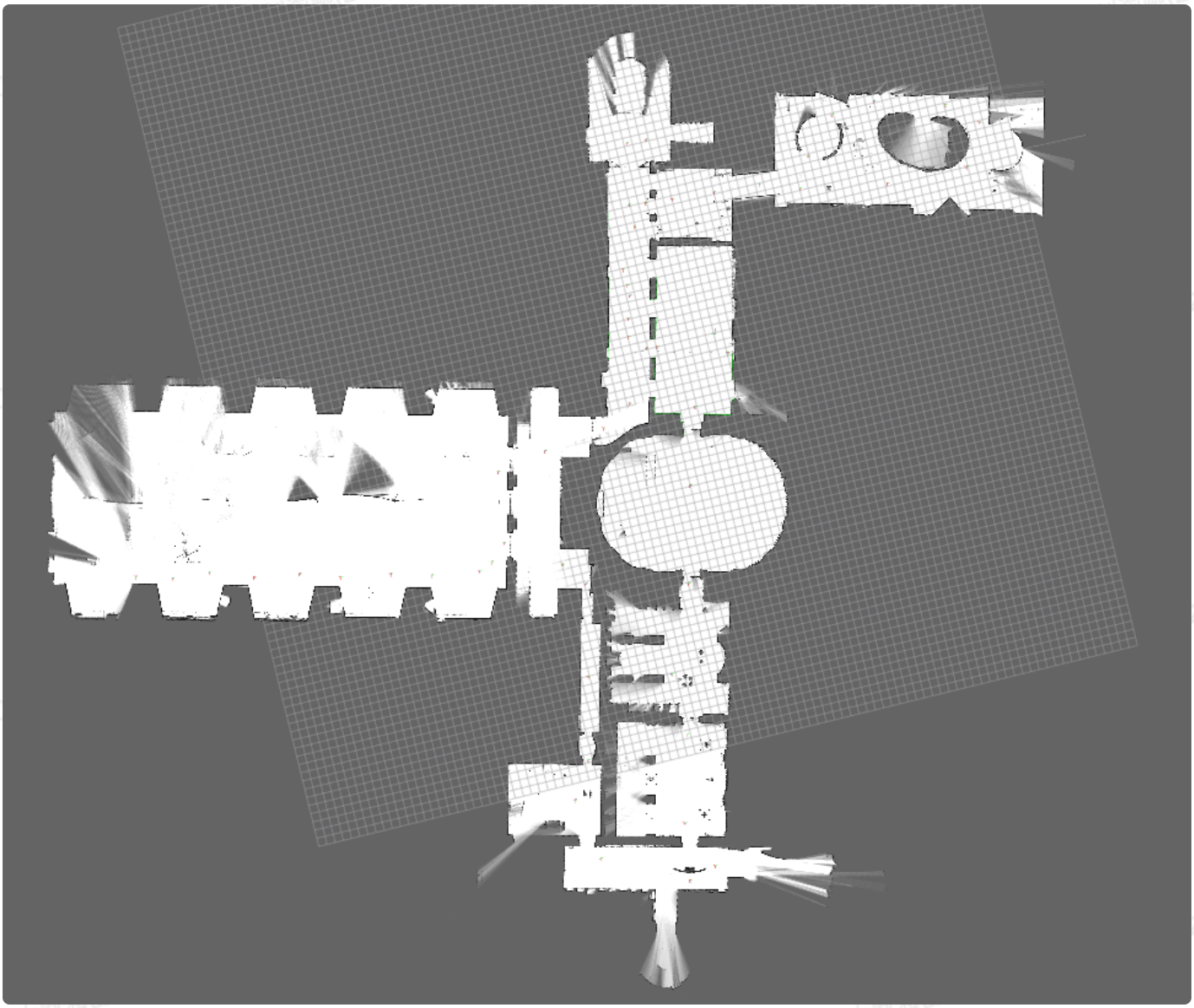
哈工大威海创新平台 - 718—刘渝婉(22971967)

rotation_weight = **10.**



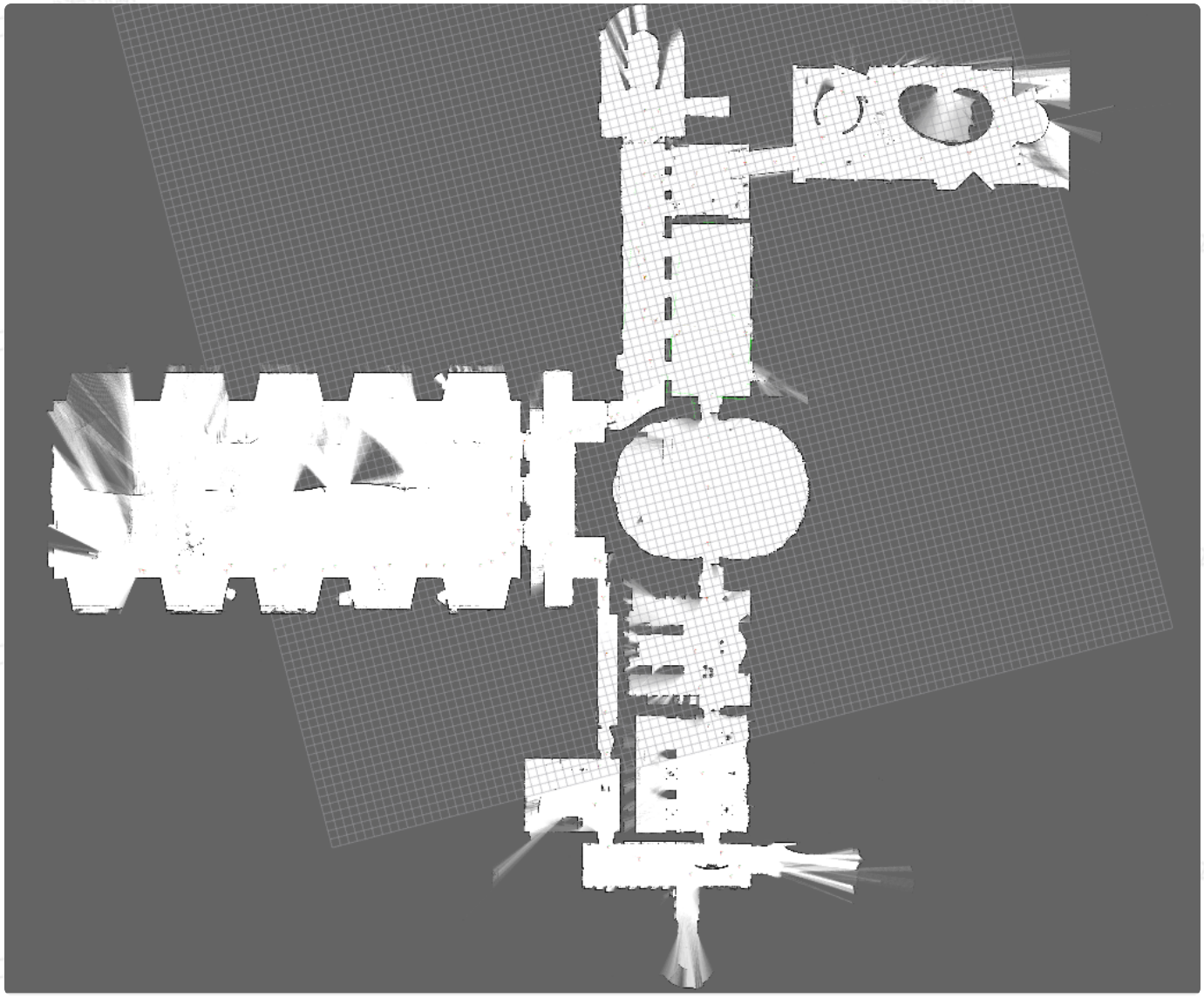
哈工大威海创新平台 - 718—刘渝婉(22971967)

rotation_weight = **40（原始值）**



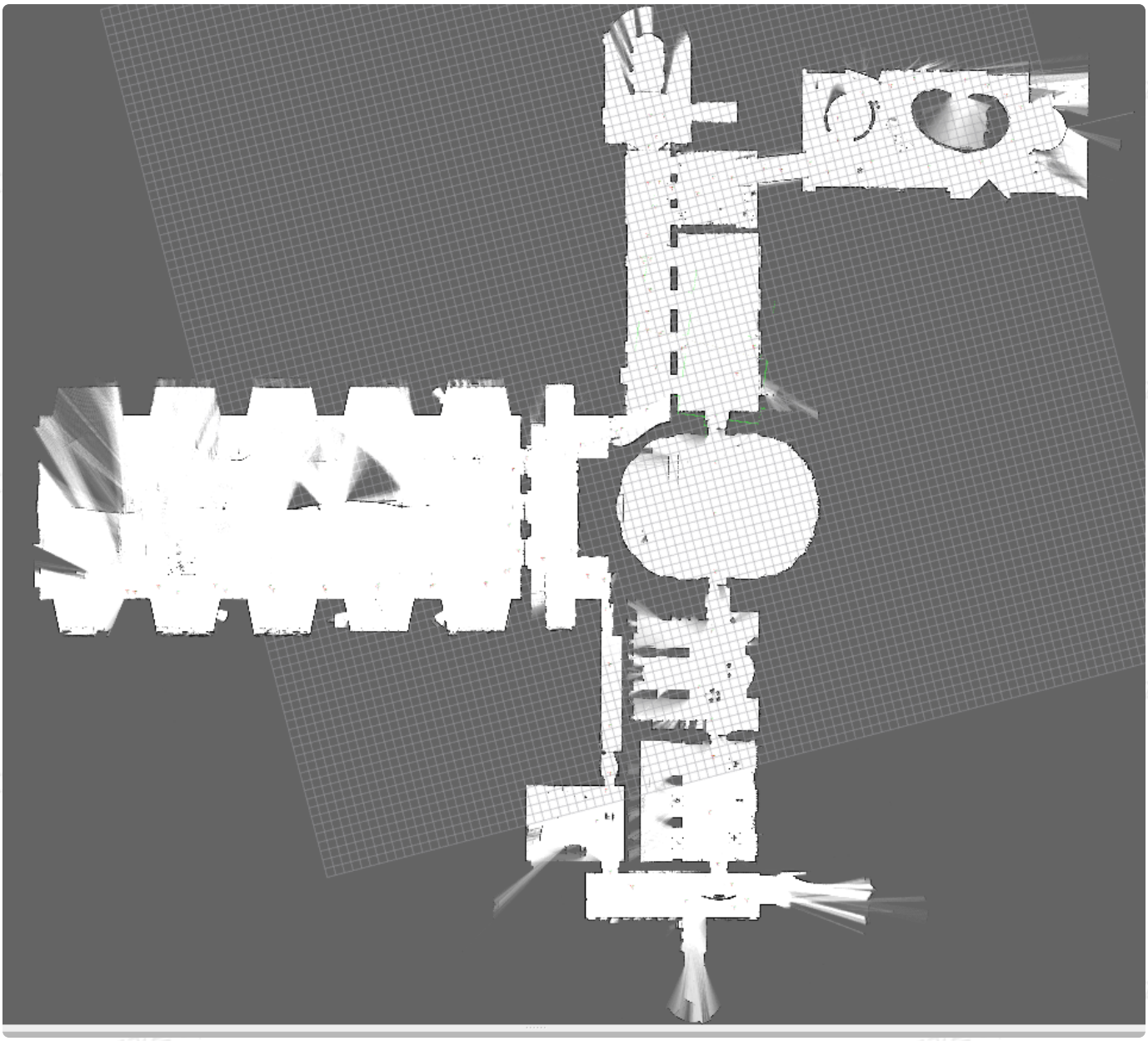
rotation_weight = **100**

哈工大威海创新平台 - 718—刘渝婉(22971967)



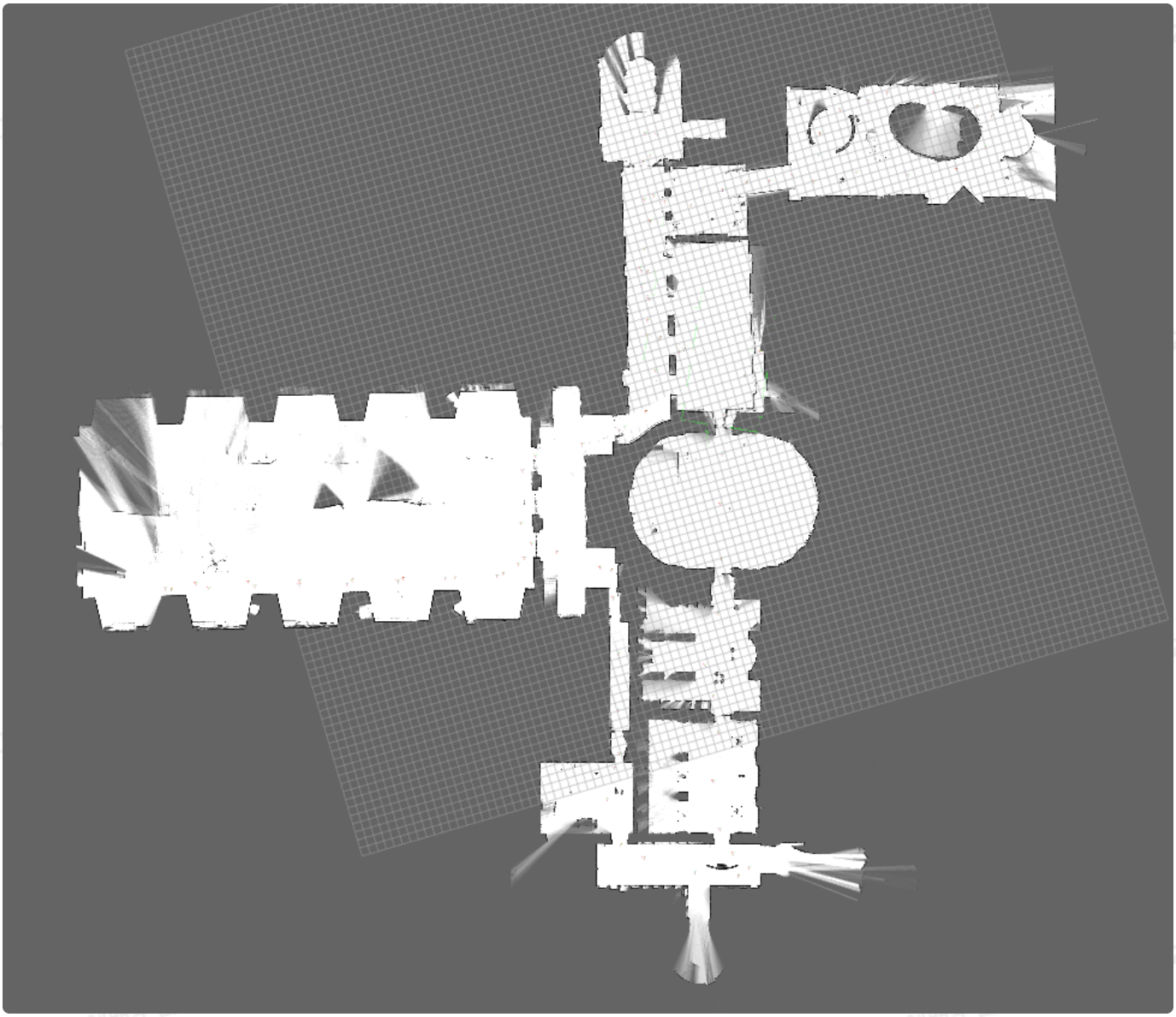
哈工大威海创新平台 - 718—刘渝婉(22971967)

rotation_weight = **200**



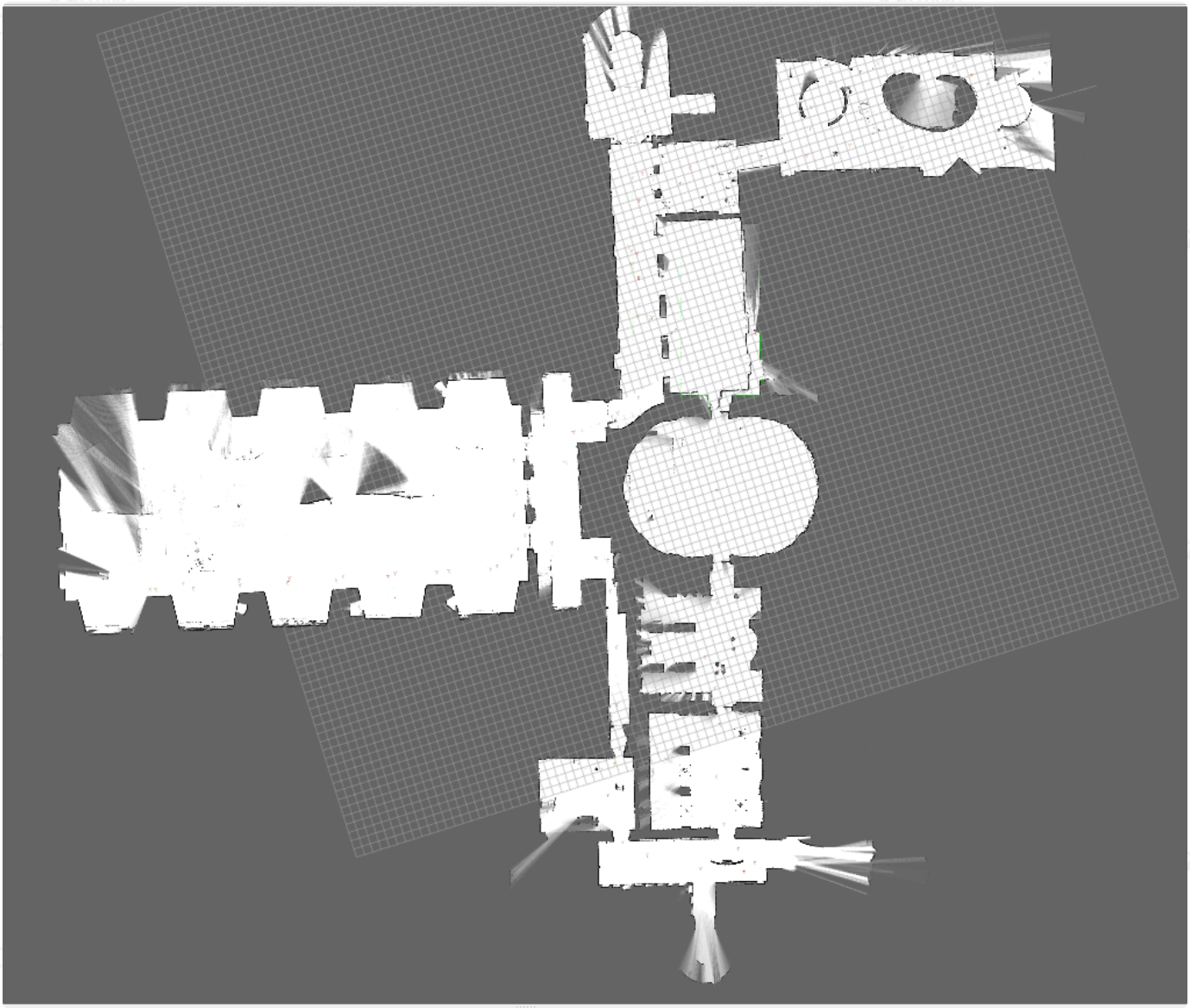
rotation_weight = **400**

哈工大威海创新平台 - 718—刘渝婉(22971967)



rotation_weight = **1000**

哈工大威海创新平台 - 718—刘渝婉(22971967)



调参结论：

较小的值没什么太大的区别，但值过大会导致图边缘出现异常

筛选最佳参数值：

rotation_weight = 40（原始值）

7.2代码部分优化：

说明：以下描述中的“激光帧”和“节点”同义

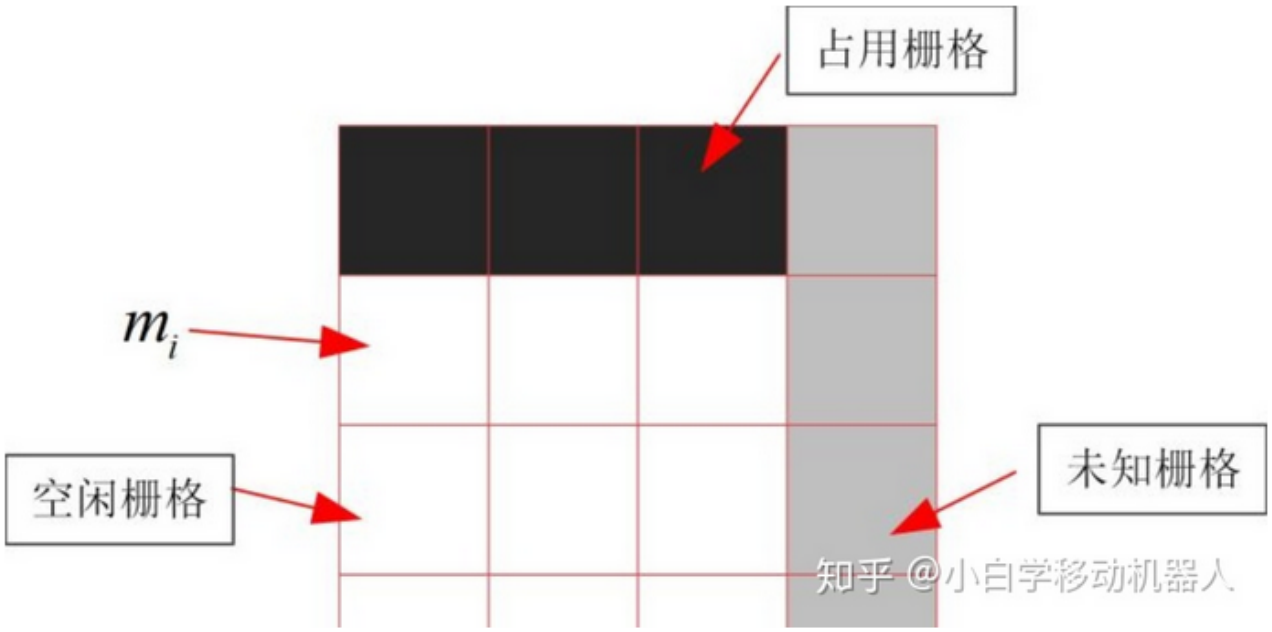
7.2.1分支定界min_score部分：

7.2.1.1思路来源：

占据栅格地图：

哈工大威海创新平台 - 718—刘渝婉(22971967)

本任务中涉及到的地图是占据栅格地图。在这种地图中，每个栅格都有自己的状态，称为Odd。将每一个栅格赋予一个概率P表示它被占据，比如P=0.8，则表示这个栅格有超过一半的概率（0.8）可能被占据。而为了计算方便等原因，我们对概率值取对数，得到一个量叫做Odd来表示这个栅格被占据的状态。Odd越大，表示这个栅格越可能已经被占据。



（上图来源于https://zhuanlan.zhihu.com/p/265221559）

分支定界中的min_score：

由于打分是将当前激光帧（节点）投影到预算图上（PrecomputationGridStack2D），使其落入某个区域内，依据这个区域中所有栅格中的最大栅格的Odd得到这个激光帧的打分。
根据原分支定界算法中会用设置的一个可人工赋值的参数阈值（称为min_score）来进行减枝操作，而这个赋值如果使用调参的话可能会出现调参效果不好，没法使min_score初始值就比较接近一个获得的激光帧对应栅格地图栅格Odd的值，减枝效果有限，我的想法是使用算法使最开始用于比较的min_score就是一个比较合适的值可以筛选掉大部分的枝，再对剩下少部分的枝进行深度搜索，这样可以减少计算量，也可以减少人工调参的时间。

7.2.1.2实现方法：

主要思路：

我的方法是除了优化最开始第一个激光帧需要一个参数（我使用的这个参数是源代码使用的、我经过我的调参过程验证的比较好的0.55）外，考虑到上一个激光帧与当前激光帧离得比较近，两者的得分应该是比较接近的，（我自己的理解是激光帧投影到地图中，因为这是在优化的部分，所以要用来优化的那一部分地图已经基本建好了，激光帧的点云图对应的应该是障碍物的图，这样前后离得近的两帧都投影

到地图中对应的障碍物应该是接近的甚至是同一片区域，对应的栅格的Odd也应该差不多），剩下每一个激光帧在分支定界时所使用的用来筛枝的min_score都是上一个激光帧找到的最优解对应的得分。

解决问题1：

问题：

虽然前后两个激光帧离得比较接近，两者得分应该比较相似，但也不排除前一个激光帧的最优解得分大于后一个激光帧所有解得分的情况，在这种情况下，因为将前一个激光帧的最优解得分作为分支定界中筛枝的阈值，根据分支定界的函数源码理解，在进入for循环的第一步的if语句中，该激光帧的所有解都因为得分小于作为阈值的前一个激光帧最优解得分而全部被筛选掉，直接跳出for循环到了最后一步——返回最优解，而在进入for循环之前最优解的得分被初始化为了用于筛枝的阈值——即上一个激光帧的最优解得分。因此总而言之，如果上一帧的最优解得分大于这一帧所有解的得分，那么这一帧的最优解得分就直接被赋值为上一帧的最优解得分。

这显然是有问题的。首先，因为上一帧的最优解得分大于下一帧的所有解得分导致下一帧最优解得分直接被赋值成了上一帧最优解得分，这直接导致这一帧的最优解得分与它应该有的值产生了一定的误差，而在ConstraintBuilder2D::ComputeConstraint函数中确定了某一帧的最优解得分后是有对得分进行相关操作的，如果这个值并非真实应该有的值，会影响接下来的操作效果；其次，如果出现某一激光帧（下文称为源头激光帧）最优解得分比较大的情况，那么之后的一部分激光帧的所有解得分都会不停的出现小于上一个激光帧的最优解得分的情况，那么这一部分激光帧最终得到的最优解得分全都会被赋值为源头激光帧的最优解得分，源头激光帧和新的激光帧之间的距离就比较远了，难以保证它们的最优解得分会接近，就算新的激光帧有解的得分大于源头激光帧的最优解得分，这样虽然可以进行筛选操作并接着往下进行深度搜索也能得到这一激光帧应该有的最优解得分，但由于无法保证源头激光帧最优解得分与当前激光帧的最优解得分接近，可能源头激光帧最优解得分过小，很多枝都没有在深度搜索前排除掉，增加了计算量，这种情况下的筛选效果未必比一个确定的阈值比如源代码使用的、我经过我的调参过程验证的比较好的0.55好。

解决方法：

根据分支定界的函数源码，其最终返回的最优解的得分只有等于筛枝的阈值或值大于筛枝的阈值两种可能，因此我对分支定界得到的激光帧的最优解分了两种情况，第一种情况是如果得到的最优解得分大于用于筛枝的阈值，那么*score和*pose_estimate就正常赋值，进行的操作和源码相同；第二种情况是如果得到的最优解得分等于用于筛枝的阈值，那么说明本次分支定界使用的作为筛枝阈值的上一激光帧的最优解得分大于了这一激光帧所有解得分，导致这一激光帧的最优解得分直接等于了上一激光帧的最优解得分，在这种情况下，说明不宜用上一激光帧最优解得分作为筛枝的阈值，则换用源代码使用的、我经过我的调参过程验证的比较好的0.55这个值作为筛枝阈值再进行一次分支定界，如果得到的最优解得分等于0.55说明它的所有解得分都是小于或等于0.55的，这就说明这一激光帧本身在地图中出现的概率就比较小，那么这个激光帧就直接被抛弃不建立约束，如果最优解得分大于0.55那么继续进行*score和*pose_estimate的赋值等操作。

解决问题2：

问题：

各个激光帧的最优解得分在储存时不能全部存在同一个数组中。由于调用分支定界算法找最优解出现在新完成的子图和过去所有节点建立约束以及新获得的节点和过去的子图建立约束，在这两种情况衔接部分需要作出调整。根据阅读的回环检测部分的代码，先进行了新获得的节点和过去的子图建立约束再进行新完成的子图和过去所有节点建立约束，涉及代码如下：

C++ 复制代码

```
//新获得的节点和过去的子图建立约束
for (const auto& submap_id : finished_submap_ids)
  {
    ComputeConstraint(node_id, submap_id);
  }


  if (newly_finished_submap)
  {
    const SubmapId newly_finished_submap_id = submap_ids.front();
    // We have a new completed submap, so we look into adding constraints
for
    // old nodes.
//新完成的子图和过去所有节点建立约束
    for (const auto& node_id_data : optimization_problem_->node_data())
    {
      const NodeId& node_id = node_id_data.id;
      if (newly_finished_submap_node_ids.count(node_id) == 0)
      {
        ComputeConstraint(node_id, newly_finished_submap_id);
      }
    }
  }
```

如果只用一个数组存储所有的最优解，新获得的节点和过去的子图建立约束完成后，此时用于储存最优解的数组的最后一个元素为新的节点和过去某一个子图之间得到的最优解，而下一步新完成的子图和过去所有节点建立约束，根据代码是从容器optimization_problem_->node_data()中取出每个历史节点，查阅optimization_problem_2d.cc发现：

```C++
const MapById<NodeId, NodeSpec2D>& node_data() const override
```

node_data()是一个MapById类模板实例化的类的一个对象，它的功能之一类似于一个map容器，储存所有的节点信息，新完成的子图和过去所有节点建立约束时，应该是从第一个节点开始遍历，而这第一个节点应该是最早的节点，这样就和刚刚储存最优解的数组的最后一个元素节点离的距离比较远，就违背了以上一个距离近的节点的最优解分数为分支定界筛枝的阈值的初衷。

解决方法：
因此，我的实现方法是建立两个可变长数组分别储存新完成的子图和过去所有节点建立约束以及新获得的节点和过去的子图建立约束这两个过程中节点的最优解得分，两个数组的第一个元素都设定为了0.55作为一个初始值，两个整型变量用于表示相应元素在数组中的索引，在使用的时候直接从数组中调取上一个节点的最优解得分。
由于阅读源码发现在ConstraintBuilder2D::ComputeConstraint函数中有“CHECK_GT(score, min_score)”的语句来比较前一个值是否大于等于后一个值，即将得到的最优解得分和分支定界中用来筛枝的阈值min_score再次比较确保最优解得分确实大于阈值min_score。
（
说明：在MatchFullSubmap和Match函数内部调用的MatchWithSearchParameters中“*score = best_candidate.score”将最优解得分赋值给*score，所以这里的score就表示每个节点最优解的得分
）
为了得到分支定界中用于筛枝的阈值，我增设了一个函数参数，得到最优解后就会把使用的筛枝的阈值赋值给这个参数，这样就便于实现ConstraintBuilder2D::ComputeConstraint函数中对最优解得分和筛枝的阈值再次比较验证。

7.2.1.3修改步骤：

第一步

在MatchWithSearchParameters以及所有调用MatchWithSearchParameters的函数的定义及其头文件中的声明中增加一个参数bool ston，并在相应的地方传入参数ston的值：
在pose_graph_2d.h/pose_graph_2d.cc中给ComputeConstraint函数加一个参数bool ston

```C++
 void ComputeConstraint(const NodeId& node_id, const SubmapId&
submap_id,bool ston)
```

将pose_graph_2d.cc中调用ComputeConstraint函数的地方增加参数，true表示该函数用于新完成的子图和过去所有节点建立约束,false表示新获得的节点和过去的子图建立约束

C++ 复制代码

```
for (const auto& submap_id : finished_submap_ids) {
    ComputeConstraint(node_id, submap_id,false);
  }

for (const auto& node_id_data : optimization_problem_->node_data()) {
      const NodeId& node_id = node_id_data.id;
      if (newly_finished_submap_node_ids.count(node_id) == 0) {
        ComputeConstraint(node_id, newly_finished_submap_id,true);
      }
    }
```

在constraint_builder_2d.h/constraint_builder_2d.cc中给MaybeAddConstraint和MaybeAddGlobalConstraint加参数

C++ 复制代码

```
 void MaybeAddConstraint(const SubmapId& submap_id, const Submap2D*
submap,
                         const NodeId& node_id,
                         const TrajectoryNode::Data* const
constant_data,
                         const transform::Rigid2d&
initial_relative_pose,bool ston)


  void MaybeAddGlobalConstraint(
      const SubmapId& submap_id, const Submap2D* submap, const NodeId&
node_id,
      const TrajectoryNode::Data* const constant_data,bool ston)

```

在pose_graph_2d.cc中调用MaybeAddConstraint和MaybeAddGlobalConstraint的地方多传一个参数

```cpp
 if (maybe_add_local_constraint) {
    const transform::Rigid2d initial_relative_pose =
        optimization_problem_->submap_data()
            .at(submap_id)
            .global_pose.inverse() *
        optimization_problem_->node_data().at(node_id).global_pose_2d;
    constraint_builder_.MaybeAddConstraint(
        submap_id, submap, node_id, constant_data,
initial_relative_pose,ston);
  } else if (maybe_add_global_constraint) {
    constraint_builder_.MaybeAddGlobalConstraint(submap_id, submap,
node_id,
                                                 constant_data,ston);
  }

```

在constraint_builder_2d.cc/constraint_builder_2d.h中给ConstraintBuilder2D::ComputeConstraint加参数

```cpp
void ConstraintBuilder2D::ComputeConstraint(
    const SubmapId& submap_id, const Submap2D* const submap,
    const NodeId& node_id, bool match_full_submap,
    const TrajectoryNode::Data* const constant_data,
    const transform::Rigid2d& initial_relative_pose,
    const SubmapScanMatcher& submap_scan_matcher,
    std::unique_ptr<ConstraintBuilder2D::Constraint>* constraint,bool
ston)
```

在调用ComputeConstraint的MaybeAddConstraint和MaybeAddGlobalConstraint中传参数

```cpp
ComputeConstraint(submap_id, submap, node_id, false, /* match_full_submap */
                      constant_data, initial_relative_pose, *scan_matcher,
                      constraint,ston);

ComputeConstraint(submap_id, submap, node_id, true, /* match_full_submap */
                      constant_data, transform::Rigid2d::Identity(),
                      *scan_matcher, constraint,ston);
```

在fast_correlative_scan_matcher_2d.cc/fast_correlative_scan_matcher_2d.h的Match和MatchFullSubmap中加参数

```cpp
bool FastCorrelativeScanMatcher2D::Match(
    const transform::Rigid2d& initial_pose_estimate,
    const sensor::PointCloud& point_cloud, const float min_score, float* score,
    transform::Rigid2d* pose_estimate,bool ston)

bool FastCorrelativeScanMatcher2D::MatchFullSubmap(
    const sensor::PointCloud& point_cloud, float min_score, float* score,
    transform::Rigid2d* pose_estimate,bool ston)
```

在constraint_builder_2d.cc的调用Match和MatchFullSubmap的ConstraintBuilder2D::ComputeConstraint函数中给这两个函数传参数

```cpp

    if (submap_scan_matcher.fast_correlative_scan_matcher->MatchFullSubmap(
                constant_data->filtered_gravity_aligned_point_cloud,
                options_.global_localization_min_score(), &score, &pose_estimate,ston))

    if (submap_scan_matcher.fast_correlative_scan_matcher->Match(
                initial_pose, constant_data->filtered_gravity_aligned_point_cloud,
                options_.min_score(), &score, &pose_estimate,ston))
```

在fast_correlative_scan_matcher_2d.cc/fast_correlative_scan_matcher_2d.h给MatchWithSearchParameters加参数并在调用它的Match和MatchFullSubmap中给这个函数传参数

```cpp
bool FastCorrelativeScanMatcher2D::MatchWithSearchParameters(
    SearchParameters search_parameters,
    const transform::Rigid2d& initial_pose_estimate,
    const sensor::PointCloud& point_cloud, float min_score, float* score,
    transform::Rigid2d* pose_estimate,bool ston)
```

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```cpp
bool FastCorrelativeScanMatcher2D::Match(
    const transform::Rigid2d& initial_pose_estimate,
    const sensor::PointCloud& point_cloud, const float min_score, float* score,
    transform::Rigid2d* pose_estimate,bool ston) const {
  const SearchParameters search_parameters(options_.linear_search_window(),
                                           options_.angular_search_window(),
                                           point_cloud, limits_.resolution());
  return MatchWithSearchParameters(search_parameters, initial_pose_estimate,
                                   point_cloud, min_score, score,
                                   pose_estimate,ston);
}

bool FastCorrelativeScanMatcher2D::MatchFullSubmap(
    const sensor::PointCloud& point_cloud, float min_score, float* score,
    transform::Rigid2d* pose_estimate,bool ston) const {
  // Compute a search window around the center of the submap that includes it
  // fully.
  const SearchParameters search_parameters(
      1e6 * limits_.resolution(),  // Linear search window, 1e6 cells/direction.
      M_PI,  // Angular search window, 180 degrees in both directions.
      point_cloud, limits_.resolution());
  const transform::Rigid2d center = transform::Rigid2d::Translation(
      limits_.max() - 0.5 * limits_.resolution() *
          Eigen::Vector2d(limits_.cell_limits().num_y_cells,
                          limits_.cell_limits().num_x_cells));
  return MatchWithSearchParameters(search_parameters, center, point_cloud,
                                   min_score, score, pose_estimate,ston);
}
```

由于constraint_builder_2d_test.cc也调用了MaybeAddConstraint和MaybeAddGlobalConstraint，而这个文件应该是用于测试的，其中NodeId{0, 0}表示这应该不是正式的计算约束，因此我把这两个函数的ston参数都设为了true（不影响结果）

```C++
 for (int j = 0; j < 2; ++j) {
      constraint_builder_->MaybeAddConstraint(submap_id, &submap, NodeId{0, 0},
                                              &node_data,
 transform::Rigid2d::Identity(),true);
    }
    constraint_builder_->MaybeAddGlobalConstraint(submap_id, &submap,
                                                  NodeId{0, 0},
&node_data,true);
    constraint_builder_->NotifyEndOfNode();
    thread_pool_.WaitUntilIdle();
    EXPECT_EQ(constraint_builder_->GetNumFinishedNodes(),
++expected_nodes);
```

fast_correlative_scan_matcher_2d_test.cc中也调用了Match和MatchFullSubmap函数，由于这个文件也应该是用于测试的，因此我把这两个函数的ston参数都设为了true（不影响结果）

```C++
    EXPECT_TRUE(fast_correlative_scan_matcher.Match(
        transform::Rigid2d::Identity(), point_cloud, kMinScore, &score,
        &pose_estimate,true));

EXPECT_TRUE(fast_correlative_scan_matcher.MatchFullSubmap(
        point_cloud, kMinScore, &score, &pose_estimate,true));
```

第二步

加入参数my_score：

给fast_correlative_scan_matcher_2d.h/fast_correlative_scan_matcher_2d.cc中的Match、MatchFullSubmap、MatchWithSearchParameters增加一个参数float* my_score，此参数用来存储用来筛枝的阈值，方便MatchWithSearchParameters的上上一层函数ConstraintBuilder2D::ComputeConstraint使用CHECK_GT语句比较得到的最优解得分和筛枝阈值的大小

第三步

修改fast_correlative_scan_matcher_2d.h/fast_correlative_scan_matcher_2d.cc文件：

在分支定界所在文件添加2个全局可变长数组分别用于存储新完成的子图和过去所有节点建立约束以及新获得的节点和过去的子图建立约束这两个过程中节点的最优解得分，新建2个全局变量表示相应元素在数组中的索引。

C++ 复制代码

```cpp
//在头文件中加
#include <vector>
//创建vector，只把第一个元素设置为0.55给第一个要激光帧使用
std::vector<float> min_score_bag1{0.55};
//新建全局变量用于记录得分的标号
int num_score1=0;

std::vector<float> min_score_bag2{0.55};
int num_score2=0;
```

在调用了BranchAndBound函数的FastCorrelativeScanMatcher2D::MatchWithSearchParameters中修改

哈工大威海创新平台 - 718—刘渝婉(22971967)

Python | 复制代码

```
//定义两个浮点型变量用于存储两种情况下的上一个节点最优解的得分，初始值为0
  float min_score1=0;
  float min_score2=0;
//定义变量the_score表示在上一个节点最优解的得分不宜作为筛枝阈值时换用的阈值
  float the_score=0.55;

//如果ston为true表示此时正进行新完成的子图和过去所有节点建立约束
if(ston){
    //对min_score2进行赋值，赋值为bag2对应索引的元素值（即上一个节点最优解得分）
      min_score2=min_score_bag2[num_score2];
    //以min_score2为筛枝阈值进行分支定界，搜索出来的最优解赋给best_candidate
      Candidate2D best_candidate = BranchAndBound(
      discrete_scans, search_parameters, lowest_resolution_candidates,
      precomputation_grid_stack_->max_depth(),min_score2);
//说明：源码中定义best_candidate时加了const，但由于我更改后如果出现best_candidate的得分和上一节点最优解得分相同会再次使用分支定界并改变已有的best_candidate，所以我把best_candidate并未定义为常量对象
//如果best_candidate的得分大于筛枝的阈值，则说明筛枝的阈值设置的比较合适
  if (best_candidate.score > min_score2) {

    *my_score=min_score2;
      //把这一节点的最优解得分加入到储存最优解的数组中以便下一个节点使用
    min_score_bag2.push_back(best_candidate.score);
    num_score2++;
      //下述两个操作与源码相同
    *score = best_candidate.score;

    *pose_estimate = transform::Rigid2d(
        {initial_pose_estimate.translation().x() + best_candidate.x,
         initial_pose_estimate.translation().y() + best_candidate.y},
        initial_rotation *
Eigen::Rotation2Dd(best_candidate.orientation));
    return true;
 }
//如果best_candidate的得分等于筛枝的阈值，说明作为筛枝的阈值的上一节点最优解得分偏大
 else if(best_candidate.score==min_score2){
     //这种情况下就再用设定的the_score作为筛枝阈值再进行一次分支定界搜索最优解
   best_candidate = BranchAndBound(
      discrete_scans, search_parameters, lowest_resolution_candidates,
      precomputation_grid_stack_->max_depth(), the_score);
     //如果搜出来的最优解得分等于the_score，则舍弃；如果大于the_score，则进行与上述best_candidate得分直接就大于筛枝阈值一样的操作
      if(best_candidate.score >the_score){
      *my_score=the_score;
      min_score_bag2.push_back(best_candidate.score);
```

```
42          num_score2++;
43          *score = best_candidate.score;
44
45          *pose_estimate = transform::Rigid2d(
46            {initial_pose_estimate.translation().x() + best_candidate.x,
47             initial_pose_estimate.translation().y() + best_candidate.y},
48            initial_rotation *
    Eigen::Rotation2Dd(best_candidate.orientation));
49        return true;
50          }
51          }
52
53
54      }
55      //如果ston为false表示此时正进行新获得的节点和过去的子图建立约束，下述过程和上面新完成
        的子图和过去所有节点建立约束的情况同理
56      else{
57      min_score1=min_score_bag1[num_score1];
58
59      Candidate2D best_candidate = BranchAndBound(
60          discrete_scans, search_parameters, lowest_resolution_candidates,
61          precomputation_grid_stack_->max_depth(), min_score1);
62
63      if (best_candidate.score > min_score1) {
64
65        *my_score=min_score1;
66        min_score_bag1.push_back(best_candidate.score);
67        num_score1++;
68        *score = best_candidate.score;
69
70        *pose_estimate = transform::Rigid2d(
71            {initial_pose_estimate.translation().x() + best_candidate.x,
72             initial_pose_estimate.translation().y() + best_candidate.y},
73            initial_rotation *
    Eigen::Rotation2Dd(best_candidate.orientation));
74        return true;
75      }
76
77      else if(best_candidate.score==min_score1){
78       best_candidate = BranchAndBound(
79          discrete_scans, search_parameters, lowest_resolution_candidates,
80          precomputation_grid_stack_->max_depth(),the_score);
81          if(best_candidate.score > the_score){
82          min_score_bag1.push_back(best_candidate.score);
83        num_score1++;
84        *score = best_candidate.score;
85
86        *pose_estimate = transform::Rigid2d(
```

```
87            {initial_pose_estimate.translation().x() + best_candidate.x,
88             initial_pose_estimate.translation().y() + best_candidate.y},
89            initial_rotation *
      Eigen::Rotation2Dd(best_candidate.orientation));
90        return true;
91          }
92          }
93      }
```

第四步

修改constraint_builder_2d.cc中的ConstraintBuilder2D::ComputeConstraint函数：

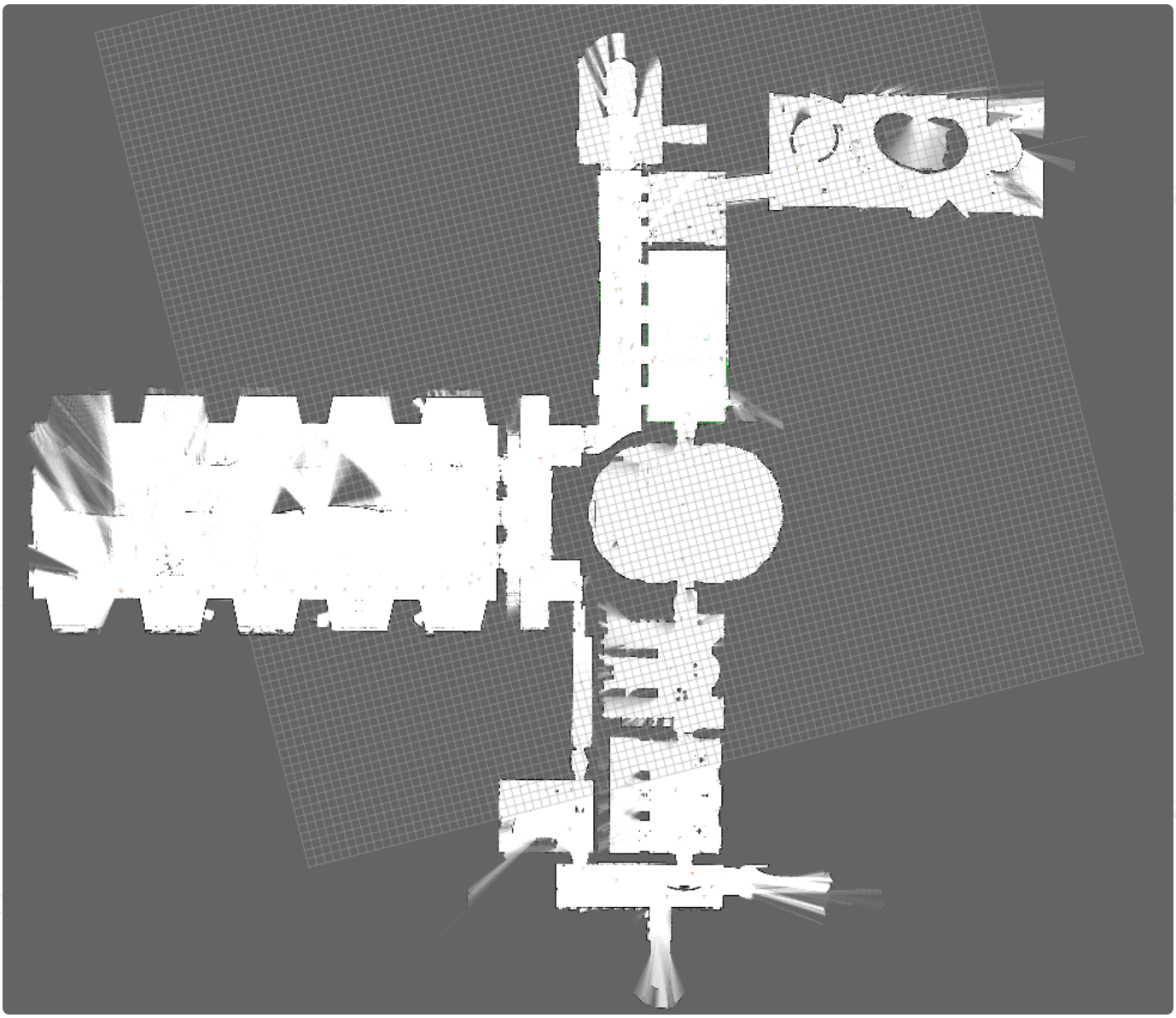
C++ | 复制代码

```
1   //定义变量my_score，设定其初始值为0，便于给下面调用的MatchFullSubmap和Match函数传参数
2   float my_score=0;
3   //将其中的CHECK_GT中的用于比较的另一个变量改为 my_score（在FastCorrelativeScanMatcher2D::MatchWithSearchParameters函数中将用于筛枝的阈值赋值给了它）
4   //此处是为了与前面的更改保持一致，这句话是为了再次验证得到的最优解得分大于筛枝的阈值，因为分支定界中用于筛枝的阈值在我更改后不再是一个固定的值，所以这里也要将第二个变量更改为分支定界中真正使用的筛枝阈值
5   CHECK_GT(score, my_score);
```

7.2.1.4效果

和原来的建图效果基本相同，但经过优化后，在上一激光帧最优解得分不大于这一帧所有解得分的情况下，就可以使用上一激光帧最优解得分作为这一帧分支定界中筛枝的阈值，由于两帧离得比较近，得分比较接近，在分支定界函数中一开始就可以筛去大部分得分不高其下属所有子节点不可能作为最优解的

父节点，这种情况下比只使用一个单一的阈值来筛选更加有效，可以减少部分计算量



7.2.1.5反思不足：

这个优化方法中，分支定界中第一个激光帧的筛枝阈值还是要依靠固定的参数，并且如果上一帧最优解得分不适合作为下一帧分支定界的筛枝阈值的情况下还是要依靠固定的一个参数值进行重新分支定界。

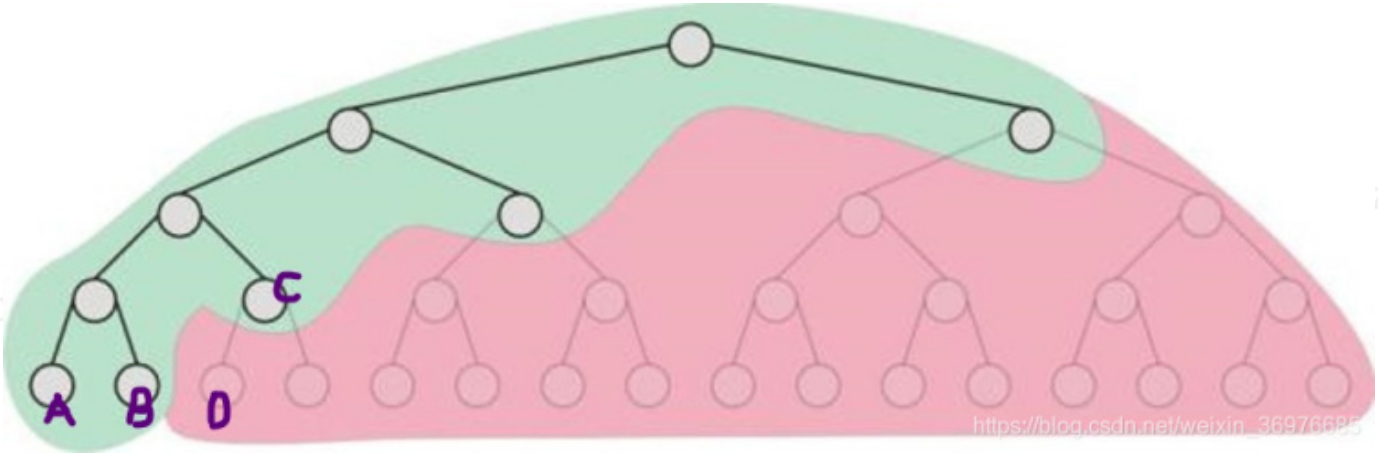
7.2.1.6代码压缩文件：

所有涉及优化的文件见优化文件1（完整src文件夹见src1）

7.2.2分支界定部分的贪心算法优化

7.2.2.1思路来源：

主要的思路是优化建图的时候的分支界定算法，分支界定算法的主要思想：把全部可行的解空间不断分割为越来越小的子集（称为分支），并为每个子集内的解的值计算一个下界或上界（称为定界）。在每次分支后，对凡是界限超出已知可行解值那些子集不再做进一步分支。这样，解的许多子集（即搜索树上的许多结点）就可以不予考虑了，从而缩小了搜索范围。



这些红色的部分就是经过可行的解减枝的部分，这个算法本身已经无可挑剔，毕竟已经从O（N^3）优化到O（2NlogN）了，所以我的优化是在边界条件部分。

7.2.2.2实现方法：

因为在建立SubMap之后和之前的结果产生束缚关联的时候需要用到大量的搜索，并且这些搜索是完全的（虽然已经经过优化），这样做是为了防止建立的新的SubMap产生匹配的小误差进行积累，从而使得产生难以挽回的大误差，但是在面对非常简单的场景的时候，比如一大片平地，或者是一成不变的周围环境的时候，这样的完全搜索尽管也很快，但是也是浪费了算力，所以我在进入递归之前加上了贪心思想，如果当前的子图匹配根节点（应该是最近的一次搜索）的程度已经比他的两个子问题解都要好，那么就完全不进行搜索，虽然这样的情况比较不常见（可以说只能应用于比较简单朴素的地形），但是也是一个小优化。

7.2.2.3修改步骤：

C++ 复制代码

```cpp
Candidate2D FastCorrelativeScanMatcher2D::BranchAndBound(
    const std::vector<DiscreteScan2D>& discrete_scans,
    const SearchParameters& search_parameters,
    const std::vector<Candidate2D>& candidates, const int candidate_depth,
    float min_score) const {
  if (candidate_depth == 0) {
      // Return the best candidate.
      return *candidates.begin();
  }
  Candidate2D best_high_resolution_candidate(0, 0, 0, search_parameters);//讨论分层并计算目标函数值
  best_high_resolution_candidate.score = min_score;//更新下best score
  for (const Candidate2D& candidate : candidates) { //在分支定界中for循环用来分层
    if (candidate.score <= min_score) { //若该值不优，则减枝
      break;
    }
    else if(candidate.score>min_score && candidate.score>candidate.x_index_offset+(1 << (candidate_depth - 1))&&candidate.score>candidate.y_index_offset+(1 << (candidate_depth - 1)))
    {

 best_high_resolution_candidate.score=std::min(best_high_resolution_candidate.score,candidate.score);
        break;
    }
    std::vector<Candidate2D> higher_resolution_candidates;
    const int half_width = 1 << (candidate_depth - 1);
    for (int x_offset : {0, half_width}) {
      if (candidate.x_index_offset + x_offset >
          search_parameters.linear_bounds[candidate.scan_index].max_x) {
        break;
      }
      for (int y_offset : {0, half_width}) {
        if (candidate.y_index_offset + y_offset >
            search_parameters.linear_bounds[candidate.scan_index].max_y) {
          break;
        }
        higher_resolution_candidates.emplace_back(
            candidate.scan_index, candidate.x_index_offset + x_offset,
            candidate.y_index_offset + y_offset, search_parameters);
      }
```

```
      }
      ScoreCandidates(precomputation_grid_stack_->Get(candidate_depth - 1),
                      discrete_scans, search_parameters,
                      &higher_resolution_candidates);
      best_high_resolution_candidate = std::max(
          best_high_resolution_candidate,
      //下面利用递归，继续搜索
          BranchAndBound(discrete_scans, search_parameters,
                         higher_resolution_candidates, candidate_depth - 1,
                         best_high_resolution_candidate.score));
    }
    return best_high_resolution_candidate;
  }

```

7.2.2.4代码压缩文件：

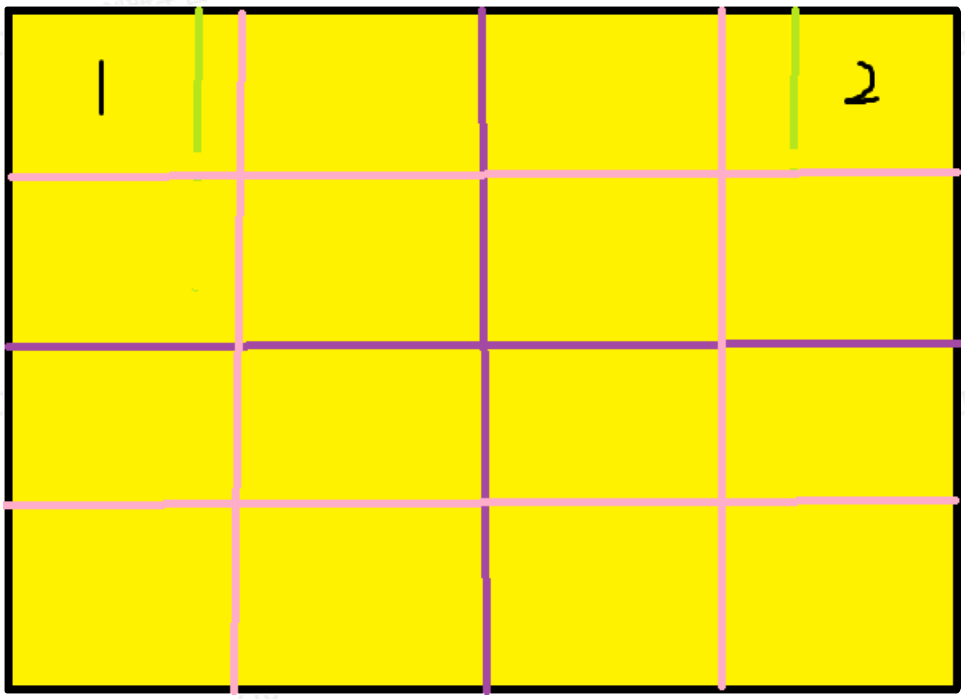
所有涉及优化的文件见优化文件2（完整src文件夹见src2）

7.2.3遇到对称环境处理的改进：

7.2.3.1思路来源：

分支定界过程的分析：

如果建图的时候碰到了对称的空间环境，设置的搜索窗恰好包含了对称环境的两个对称部分，如下图：



哈工大威海创新平台 - 718—刘渝婉(22971967)

两条绿线表示对称的两个位姿，图中展示的每一个格子都是解，以紫色的线为划分，每一个大格子中的左上角为父节点，其右、右下、下以及它自身为它附属的四个子节点（我展示出来的是最简单的一种情况，最大深度仅为1，仅作优化思路说明使用）。现在假设当前的激光帧对应的最优解为这两条绿线表示的位姿之一。

在进行分支定界的时候，进入for循环后先对第一个未被筛枝阈值筛掉的解（假设是最左上角的）进行深度搜索，搜索完返回的最优解为方格1所代表的那个解，此时更新best_high_resolution_candidate为方格1所代表的那个解，再对第二个解（假设是方格2所代表的节点的所属父节点）进行深度搜索，得到的分值最高的解为方格2所代表的那个解，此时将递归函数的返回值（即方格2所代表的解）的得分与之前更新的 best_high_resolution_candidate（方格1所代表的解）的得分作比较，由于这两个位姿对称，所以很有可能二者的得分相同，根据C++的max函数的定义可知，如果比较的两个对象等价，则返回传入的两个变量中的第一个变量，在这个例子中即会返回方格1所代表的解。这样就会产生偏差，导致计算出来的post_estimate与真实的不同，计算约束也会出现两个对称的位姿计算的次数有偏差，很有可能方格2所代表的解才是最优解，但却被舍弃了。

解的几何关系：

在遇到对称位置的上一帧的位姿落在对称轴上并且和两个对称位姿的距离完全相等的概率极小，所以可以利用上一帧所在的最佳位姿的坐标（即最优解的坐标），得到的两个对称的位姿哪一个离上一帧所在的最佳位姿的坐标近，就很有可能是这一帧机器人的真实位姿。

7.2.3.2实现方法：

利用上一帧的最优解对迭代搜索部分的语句进行改进，将迭代的结果的解的得分与当前best_high_resolution_candidate的得分先比较一下大小，如果二者大小相同，就分别计算一下两个解和上一激光帧的最优解的距离，距离较近的赋值给最优解，距离相同的话最优解不变；如果两者得分不相同，则进行正常的根据得分的大小、选取得分高的解作为最优解。

7.2.3.3修改步骤：

由于第一帧所在位姿就是对称的位姿之一的概率很小，所以没有考虑第一帧所在位姿就是对称的位姿的情况。

第一步

由于也要利用上一激光帧离这一激光帧较近的特性，所以也需要用两个数组分别储存新完成的子图和过去所有节点建立约束以及新获得的节点和过去的子图建立约束这两个过程中节点的最优解得分，以及两个整型变量分别表示这两个数组元素的索引，**第一步的修改步骤与“七、2（1）的修改步骤【1】”完全相同**，因此这里不再赘述。

第二步

准备工作：

在fast_correlative_scan_matcher_2d.cc文件中：

首先在文件开头定义全局的数组和全局的整型变量：

```C++
//定义可变长数组用于存储各个激光帧的最优解
std::vector<Candidate2D> historical_candidate1;
std::vector<Candidate2D> historical_candidate2;
//定义整型变量用于表示historical_candidate元素的索引
int num_candidate1=0;
int num_candidate2=0;
```

在FastCorrelativeScanMatcher2D::MatchWithSearchParameters函数中增加语句：

```C++
if(ston){
  //将最优解加入数组存储
    historical_candidate1.push_back(best_candidate);
    num_candidate1++;
    }else{
    historical_candidate2.push_back(best_candidate);
    num_candidate2++;
    }
```

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```
  if (best_candidate.score > min_score) {
  if(ston){
  //将最优解加入数组存储
    historical_candidate1.push_back(best_candidate);
    num_candidate1++;
    }else{
    historical_candidate2.push_back(best_candidate);
    num_candidate2++;
    }
    *score = best_candidate.score;
    *pose_estimate = transform::Rigid2d(
        {initial_pose_estimate.translation().x() + best_candidate.x,
         initial_pose_estimate.translation().y() + best_candidate.y},
        initial_rotation *
Eigen::Rotation2Dd(best_candidate.orientation));
    return true;
  }
```

第三步

修改分支定界函数：

在FastCorrelativeScanMatcher2D::BranchAndBound函数中将原有递归语句修改：

哈工大威海创新平台 - 718—刘渝婉(22971967)

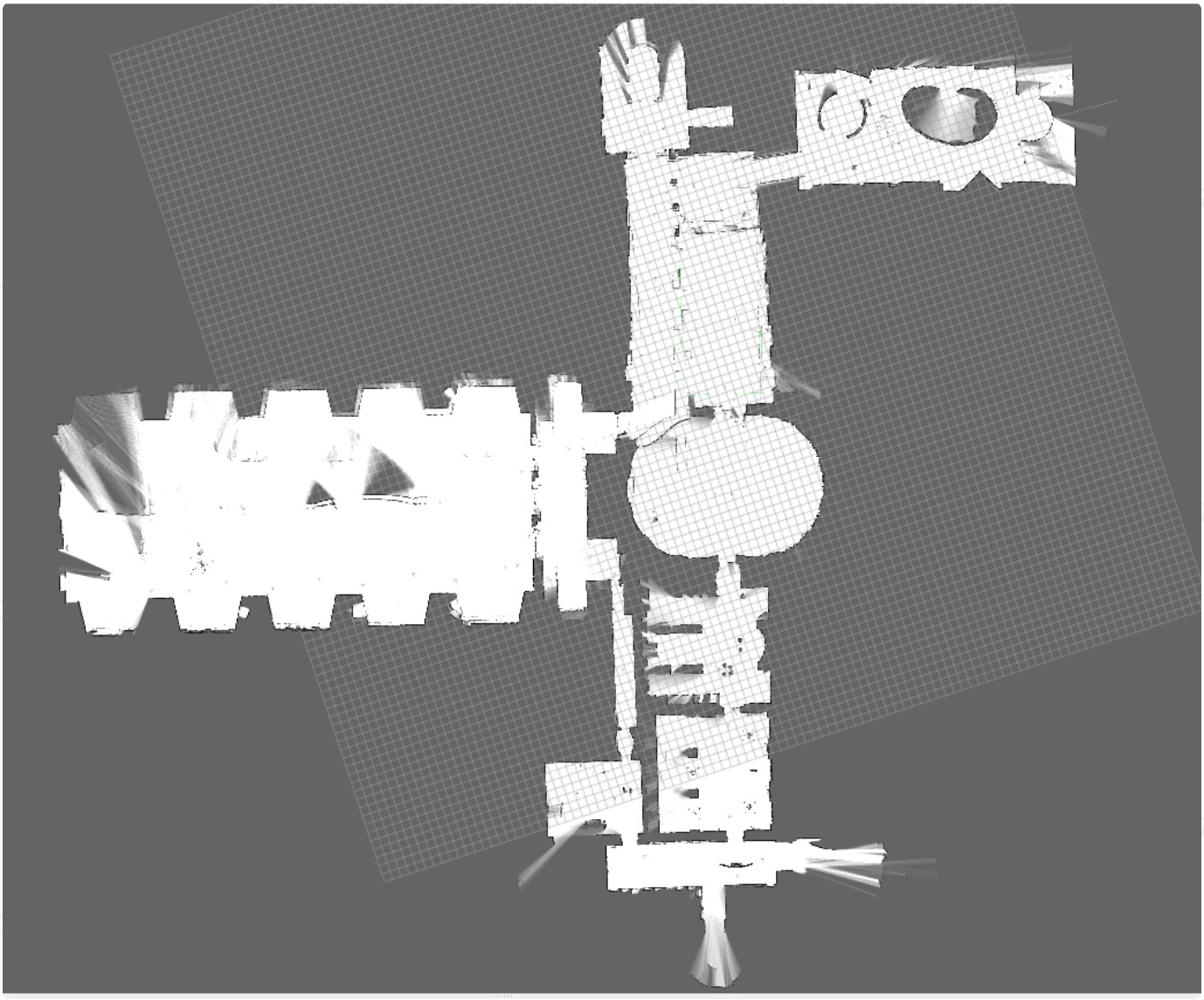
C++ | 复制代码

```cpp
//将递归得到的结果暂时储存在temporary_candidate中
Candidate2D temporary_candidate=BranchAndBound(discrete_scans, search_parameters,
                        higher_resolution_candidates, candidate_depth - 1,
                        best_high_resolution_candidate.score,ston);


  //如果ston为true，表示正在进行新完成的子图和过去所有节点建立约束
  if(ston){
      //判断一下是否是第一个激光帧，由于第一个激光帧所在位姿就是对称的位姿之一的概率很小，所以没有考虑第一帧所在位姿就是对称的位姿的情况
  //不是第一个激光帧才往下进行
      if(num_candidate1!=0){

  //若两个解得分相同
    if(temporary_candidate.score==best_high_resolution_candidate.score){
        //获取上一激光帧最优解
        Candidate2D last_candidate=historical_candidate1[num_candidate1-1];
        //计算和上一个激光帧最优解位置坐标的距离
        double a=1.0*(last_candidate.x-temporary_candidate.x);
        double b=1.0*(last_candidate.y-temporary_candidate.y);
        double c=1.0*(last_candidate.x-best_high_resolution_candidate.x);
        double d=1.0*(last_candidate.y-best_high_resolution_candidate.y);

        double distance_t=a*a+b*b;
        double distance_b=c*c+d*d;
        //如果递归结果与上一个激光帧最优解位置坐标的距离较小则说明很有可能是最优解
        if(distance_t<distance_b){

            best_high_resolution_candidate=temporary_candidate;
             }
        //若两个解得分不同，按照原来的将分数高的赋值给best_high_resolution_candidate
        }else{


    best_high_resolution_candidate = std::max(best_high_resolution_candidate,
       temporary_candidate);
     }

  }
     // 如果ston为false表示正进行新获得的节点和过去的子图建立约束，算法步骤和上面进行新完成的子图和过去所有节点建立约束时相似
```

```
  }else{
  if(num_candidate2!=0){


    if(temporary_candidate.score==best_high_resolution_candidate.score){

        Candidate2D last_candidate=historical_candidate2[num_candidate2-
1];
        double a=1.0*(last_candidate.x-temporary_candidate.x);
        double b=1.0*(last_candidate.y-temporary_candidate.y);
        double c=1.0*(last_candidate.x-best_high_resolution_candidate.x);
        double d=1.0*(last_candidate.y-best_high_resolution_candidate.y);

        double distance_t=a*a+b*b;
        double distance_b=c*c+d*d;
        if(distance_t<distance_b){

            best_high_resolution_candidate=temporary_candidate;
             }

        }else{


    best_high_resolution_candidate =
std::max(best_high_resolution_candidate,
       temporary_candidate);
     }

  }
  }
```

7.2.3.4效果：

建图效果比原来稍微差了一些，出现了一定程度的漂移与重影



7.2.3.5反思不足：

如果上一帧恰好处在两个对称位姿中间，这样两者离上一帧最优解位置距离完全相同，这种情况未作处理；并且优化时的第一帧也有可能为对称的两位姿中的一个，也未处理此种情况。

7.2.3.6代码压缩文件：

所有涉及优化的文件见优化文件3（完整src文件夹见src3）

7.2.4分支定界源码筛枝阈值设定优化：

7.2.4.1思路来源：

分支定界中先以一个阈值min_score筛去一部分的父节点，以免所有节点进行深度搜索导致计算量过大。

预算图：

每一个节点的得分都是根据预算图（PrecomputationGridStack2D）得到的。预算图是在原分辨率的地图上生成的地图，原分辨率地图作为最高分辨率地图，最高分辨率地图中的不同的固定数目的格子合成一个格子便形成了不同分辨率的地图，新形成的地图每个格子的得分是对应原分辨率地图中组成它的几个格子中最大的得分。深度较大的父节点的得分是依据低分辨率地图得到的，深度较小的子节点的得分是依据高分辨率地图得到的，子节点对应的格子即为其父节点对应的格子细分后的格子。

源码中的best_high_resolution_candidate：

因此可以得出，所有父节点的得分一定大于等于其下属子节点的得分，在分支定界的算法中应用了for循环对没被阈值筛掉的父节点依次进行深度搜索，并且不断的将表示最优解的变量best_high_resolution_candidate随着向下搜索更新，best_high_resolution_candidate的初始值是人工可调的参数min_score。

7.2.4.2实现方法：

我的想法是将用于筛掉父节点的min_score改为best_high_resolution_candidate，在父节点1已经深度搜索完更新了best_high_resolution_candidate后，在进行父节点2的深度搜索前，将它的得分和best_high_resolution_candidate作比较，如果它的得分小于等于best_high_resolution_candidate就会被筛掉就不必再往下进行深度搜索（因为父节点2的得分一定大于等于下属子节点，如果它的得分都不比best_high_resolution_candidate大，那么就算对父节点2进行深度搜索至深度为0得到其下属子节点的得分最高的解再与best_high_resolution_candidate比较也会因值小被筛掉，那么这个深度搜索便是在效果没有变化的情况下增大了计算量）

7.2.4.3修改步骤：

将筛枝的if条件中的min_score改为best_high_resolution_candidate.score

哈工大威海创新平台 - 718—刘渝婉(22971967)

C++ 复制代码

```cpp
Candidate2D FastCorrelativeScanMatcher2D::BranchAndBound(
    const std::vector<DiscreteScan2D>& discrete_scans,
    const SearchParameters& search_parameters,
    const std::vector<Candidate2D>& candidates, const int
candidate_depth,
    float min_score) const {
  if (candidate_depth == 0) {
    // Return the best candidate.
    return *candidates.begin();
  }

  Candidate2D best_high_resolution_candidate(0, 0, 0, search_parameters);
  best_high_resolution_candidate.score = min_score;
  for (const Candidate2D& candidate : candidates) {
      //将筛枝的if条件中的min_score改为best_high_resolution_candidate.score
    if (candidate.score <= best_high_resolution_candidate.score) {
      break;
    }
    std::vector<Candidate2D> higher_resolution_candidates;
    const int half_width = 1 << (candidate_depth - 1);
    for (int x_offset : {0, half_width}) {
      if (candidate.x_index_offset + x_offset >
          search_parameters.linear_bounds[candidate.scan_index].max_x) {
        break;
      }
      for (int y_offset : {0, half_width}) {
        if (candidate.y_index_offset + y_offset >
            search_parameters.linear_bounds[candidate.scan_index].max_y)
{
          break;
        }
        higher_resolution_candidates.emplace_back(
            candidate.scan_index, candidate.x_index_offset + x_offset,
            candidate.y_index_offset + y_offset, search_parameters);
      }
    }
    ScoreCandidates(precomputation_grid_stack_->Get(candidate_depth - 1),
                    discrete_scans, search_parameters,
                    &higher_resolution_candidates);
    best_high_resolution_candidate = std::max(
        best_high_resolution_candidate,
        BranchAndBound(discrete_scans, search_parameters,
                       higher_resolution_candidates, candidate_depth - 1,
                       best_high_resolution_candidate.score));
  }
```

```
    return best_high_resolution_candidate;
  }

```

7.2.4.4效果：

建图效果与原来相同，从理论上可以减少计算量，如果遇上父节点分数大于原来用于筛选的min_score但是小于等于已经更新的最优解best_high_resolution_candidate的得分的情况，就可以直接舍弃避免再对这一子节点不可能成为最优解的父节点再次进行深度搜索增大计算量。

7.2.4.5代码压缩文件：

所有涉及优化的文件见优化文件4（完整src文件夹见src4）

7.2.5已排除的错误的优化思路：

建立一个存储每个节点的pose_estimate的数组避免重复计算pose_estimate：

7.2.5.1初步想法：

看源码的时候看到PoseGraph2D::ComputeConstraintsForNode函数中无论是对新的节点和过去完成的子图之间求约束还是对新的子图和过去的节点之间求约束，都用到了ComputeConstraint(node_id, submap_id)，在这个函数中无论是局部窗口搜索还是全局窗口搜索计算约束的式子都应该是：

C++ 复制代码

```
const transform::Rigid2d constraint_transform =
      ComputeSubmapPose(*submap).inverse() * pose_estimate;
```

用到的pose_estimate是根据分支定界找出来的最优解来赋值的。
所以我想到新得到的节点会求一次pose_estimate，那么所有节点都有对应的pose_estimate，那么在计算过去的节点和新子图之间的约束就可以不用再对过去的节点求一次pose_estimate了，这样可以减少计算量。

7.2.5.2错误原因：

经过进一步阅览代码发现，MatchWithSearchParameters函数对pose_estimate赋值时：

```C++
*pose_estimate = transform::Rigid2d(
        {initial_pose_estimate.translation().x() + best_candidate.x,
         initial_pose_estimate.translation().y() + best_candidate.y},
        initial_rotation *
Eigen::Rotation2Dd(best_candidate.orientation));
```

用到了initial_pose_estimate，而调用MatchWithSearchParameters函数的Match和MatchFullSubmap相应位置的参数分别是 initial_pose_estimate和center，Match函数的initial_pose_estimate被调用它的ConstraintBuilder2D::ComputeConstraint赋值为initial_pose，而这个initial_pose的计算过程是：

```C++
const transform::Rigid2d initial_pose =
      ComputeSubmapPose(*submap) * initial_relative_pose;
```

它与当前要计算约束的submap有关。
则同一个节点和不同的submap计算约束的时候，其pose_estimate并不相同，是和submap有关的，所以每次确实得重新计算一遍pose_estimate

8.视频及优化代码压缩包网盘链接

8.1视频压缩包：

链接：https://pan.baidu.com/s/1XKz_qvptNzhL9MtOQ07E5Q
提取码：2022

8.2代码压缩包：

（含有涉及优化的文件以及优化文件所在完整的src文件夹）
链接：https://pan.baidu.com/s/1wUMusPuYab5B6bErvXe6LA
提取码：2331